



**Beni-Suef University
Faculty of Engineering
Communication and Electronics Engineering**

OTA Updates with Real Time Diagnostics via CAN

Project members

Ahmed Ragab Mogoda

Amar Ragab Youssef

Mahmoud El Dinary Mohamed

Supervisor

Dr. Mohamed Mourad

Academic Year: 2024 – 2025

July 2025

ABSTRACT

This thesis presents the design, implementation, and validation of an intelligent vehicle control system capable of Over-The-Air (OTA) firmware updates and Real-Time Diagnostics over a CAN-Bus network. The proposed architecture comprises two STM32F103 ECUs—the Main ECU for update management and the App ECU for sensor data acquisition and motor control—and a Raspberry Pi 4 hosting a Flask-based GUI and acting as the OTA client.

The system leverages FreeRTOS on each STM32 to orchestrate multiple concurrent tasks, including CAN-frame handling, diagnostics via Unified Diagnostic Services (UDS), and secure update reception with RSA-based verification. On the Raspberry Pi, SocketCAN and Server-Sent Events (SSE) enable live streaming of vehicle telemetry (temperature, distance, speed) to a responsive web interface. Firmware images are stored on a cloud server (AWS S3/Firebase) and fetched by the Raspberry Pi via a RESTful API. The Bootloader on each ECU handles chunked reception over CAN, flash writing, and fail-safe rollback.

Comprehensive testing demonstrates reliable CAN communication at 500 kbps, seamless OTA updates under load, and conformance with ISO 14229 Diagnostic services. The results confirm that the system provides a scalable, secure, and user-friendly solution for continuous vehicle monitoring and maintenance. Potential extensions include CAN-FD support and integration of cloud-based analytics for predictive Diagnostics.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to **Prof. Dr. Mohamed Mourad** for his invaluable guidance, continuous support, and insightful feedback throughout every stage of this work. His expertise in embedded systems and automotive electronics inspired and motivated me to push the boundaries of this project.

I am equally grateful to the staff and engineers at the Communication and Electronics Engineering for granting me access to their laboratories and equipment, and for their practical advice on hardware integration and CAN-Bus configuration. Special thanks go to my colleagues and friends **Ahmed Ragab Mogoda** and **Ammer Ragab Youssef** and **Mahmoud El Dinary Mohamed** for their collaborative spirit, brainstorming sessions, and late-night troubleshooting that greatly enriched this work.

This project has been made possible in part by the generous support of the Information Technology Industry Development Agency (**ITIDA**). Their funding and resources were instrumental in procuring key hardware components and facilitating the cloud-server infrastructure.

I would also like to acknowledge the support of my family, whose patience and encouragement carried me through the challenges of design, implementation, and testing. Lastly, I thank all the open-source communities and documentation authors whose tools (FreeRTOS, Flask, SocketCAN, etc.) and examples were instrumental in realizing the smart-vehicle control system.

Table of Contents

Abstract.....	ii
Acknowledgement.....	iii
Table of Contents.....	iv
List of Figures.....	viii
List of Abbreviations.....	ix
Nomenclatures.....	ix
Chapter 1: Introduction	2
1.1 Background & Motivation	2
1.2 Project Objectives	3
1.3 Scope & Limitations	3
1.4 Organization of the Thesis	5
Chapter 2: Technical Background	6
2.1 Automotive Firmware Systems	7
2.2 Firmware in Automotive Systems	9
2.3 Firmware Traditional Flashing Techniques	9
2.3.1 Off –Circuit Programming.....	10
2.3.2 In-Circuit programming	11
2.3.3 The Evolution of OTA updates	11
2.4 Diagnostic Services	13
2.4.1 Traditional Diagnostic Methods.....	13
2.4.2 Evolution of Wireless Diagnostics	14
2.4.3 Diagnostic Trouble Codes (DTCs)	15
2.5 ISO UDS	16
2.5.1 Introduction to ISO Standards in Automotive	16
2.5.2 Overview of ISO 142292	16
2.5.3 Key Features of UDS ISO 14229	17
2.5.4 Diagnostic Services Provided by UDS	17
2.5.5 Communication Protocols Supporting UDS	18
2.5.6 Benefits of Using UDS in Automotive Diagnostics	18
2.6 Communication protocols Brief	19
2.6.1 CAN protocol.....	19
2.6.2 UART (Universal Asynchronous Receiver/Transmitter)	20
2.7 RTOS	21
2.7.1 Why using RTOS	21
2.7.2 Brief History of RTOS and Embedded Systems	21

Chapter 3: Hardware Architecture	24
3.1 STM32F103	25
3.2 Raspberry Pi 4	26
3.3 CAN Hat (MCP2515 + TJA1050).....	27
3.4 CAN Transceiver (SN65HVD01)	28
3.5 Sensors	29
3.5.1 Temperature Sensor (LM35)	29
3.5.2 Ultrasonic Sensor	29
3.6 Actuators & Motor Drivers (L298N)	30
3.7 HMI Touchscreen	32
3.8 Bluetooth Module	32
 Chapter 4: CAN Network	33
4.1 Introduction	34
4.1.1 Before CAN	34
4.1.2 A typical wiring harness for a car	35
4.1.3 The Evolution of In-vehicle Network Architectures	36
4.1.4 Why Serial Communication?	39
4.2 CAN Overview	40
4.2.1 What is CAN?	40
4.2.2 Features of CAN protocol	41
4.2.3 Comparison of automotive bus systems	42
4.3 The basic structure of a CAN bus automotive system	43
4.3.1 Physical Layer	44
4.3.2 Boundary conditions.....	45
4.3.3 Integration	45
4.4 CAN BUS advantages and disadvantages	45
4.2.1 CAN bus advantages	45
4.2.2 CAN bus disadvantages	46
4.2.3 CAN bus main properties.....	46
4.5 CAN Bus Frames	47
4.5.1 Can Network Message Format	49
4.5.2 CAN in Action.....	50
 Chapter 5: Real-Time Operating System (FreeRTOS)	53
5.1 RTOS Fundamentals	54
5.2 Task Design & Priorities	56
5.2.1 CAN Receive Task	60

5.2.2 CAN Transmit Task	61
5.2.3 Control Task (Motor Safety)	62
5.2.4 Diagnostics Task (UDS)	63
5.2.5 OTA Update Task	64
5.3 Inter-Task Communication (Queues, Notifications)	67
 Chapter 6: Unified Diagnostic Services (UDS) on CAN	70
6.1 UDS Standard (ISO 14229)	71
6.2 Diagnostic Session Control (0x10)	72
6.3 Security Access (0x27)	74
6.4 Read Data By Identifier (0x22)	76
6.5 Read & Clear DTC (0x19 / 0x14)	78
6.6 Request Download (0x34)	80
6.7 DTC Storage & Management in Flash	82
 Chapter 7: Bootloader & OTA Firmware Update	86
7.1 Flash Memory Layout & Partitioning	87
7.2 Bootloader Design	89
7.2.1 Metadata & Option Bytes	89
7.2.2 CAN-based Chunk Reception	90
7.2.3 Chunk Erase & Write	90
7.2.4 Update Verification (RSA)	91
7.3 Fallback & Fail-Safe Mechanisms	92
7.4 Bootloader ↔ Application Handoff	96
 Chapter 8: Cloud-Based Firmware Server	100
8.1 Server Architecture & REST API	101
8.2 Version JSON Schema & Hosting	104
8.3 Secure Storage (e.g. AWS S3, Firebase)	105
8.4 Firmware Download & Integrity Checks	106
8.5 Extending for Multiple Device Types	108
 Chapter 9: Raspberry Pi Software Stack	110
9.1 SocketCAN Interface	111
9.2 Sensor Listener & Dispatcher	112
9.3 Diagnostics Client (UDS over CAN)	113
9.4 OTA Client (Chunked Upload)	114
9.5 Server Client (Firmware Version Check & Download)	116

Chapter 10: Web GUI Design	118
10.1 Technology Choices (Flask + HTML/CSS/JS + SSE)	119
10.2 Template Layout & Static Assets	120
10.3 Live Speed Streaming	121
10.4 Diagnostics Panel & Auto-Hide Behavior	122
10.5 OTA Update Workflow & Progress Bar	123
10.6 Responsive Design & UX Considerations	125
 Chapter 11: System Integration & Deployment	 128
11.1 Raspberry Pi Auto-Start (systemd Services)	129
11.2 CAN Interface Initialization	130
11.3 Security & Access Control	131
11.4 Logging & Monitoring	131
11.2 STM32 ECUs Integration.....	132
11.2 .1 App ECU (STM32F103)	132
11.2.2 Main ECU (STM32F103)	137
11.3 CAN Bus Wiring & Topology.....	139
11.4 System Overview & Project Images.....	140
11.4.1 System Overview.....	140
11.4.2 Real Time Photo.....	140
 Chapter 12: Results & Discussion.....	 143
12.1 Key Findings.....	144
12.2 Limitations & Challenges.....	145
12.3 Comparison with Existing Solutions.....	146
12.4 Challenges Faced & Lessons Learned.....	147
 Chapter 13: Conclusions & Future Work.....	 149
13.1 Summary of Contributions.....	145
13.2 Lessons Learned.....	151
13.3 Potential Extensions (e.g. CAN-FD, Cloud Analytics)	152
13.4 Final Remarks.....	153

LIST OF FIGURES

Figure 2.1 Different ECUs functions in moder cars.....	7
Figure 2.2 Mercedes M156 ECU.....	8
Figure 2.3 Off-Circuit Programmer.....	10
Figure 2.4 ISP ST-Link Programmer.....	11
Figure 2.5 Over the air updates mechanism aka. FOTA.....	12
Figure 2.6 Different types of OBD tools.....	13
Figure 2.7 OBD Connector in a car.....	14
Figure 2.8 Most important applied networking and diagnostic technologies over the past 15 years.....	16
Figure 2.9 Uart Frame Structure.....	20
Figure 2.9 GE 225 Computer.....	22
Figure 2.10 : end of help command output from RT-11SJ display.....	22
Figure 3.1 STM32F103C8T6 Pinout.....	25
Figure 3.2 Raspberry Pi.....	26
Figure 3.3 CAN Hat.....	27
Figure 3.4 CAN BUS Transceiver mcp2551.....	28
Figure 3.5 High-speed CAN BUS and Low-speed CAN BUS.....	28
Figure 3.6 Photographs and pin configuration diagrams of the LM35.....	29
Figure 3.7 Close-up view of the HC-SR04.....	29
Figure 3.8 DC Motors.....	30
Figure 3.9 Motor Driver (L298N).....	31
Figure 3.10 Connection of 7-inch capacitive touchscreen.....	32
Figure 3.11 Bluetooth Module.....	32
Figure 4.1 Networking Before and After CAN.....	34
Figure 4.2 Traditional old-style vehicle electronics.	35
Figure 4.3 wiring harness for a car.....	36
Figure 4.4 In vehicle network.....	36
Figure 4.5 IVNs Classical architecture.....	37
Figure 4.6 In the Domain IVN architecture, domain controllers organize the ECUs under them.....	37
Figure 4.7 the Zonal INV architecture.....	38
Figure 4.8 CAN Bus with two nodes.....	41
Figure 4.9 CAN bus-based automotive wiring system.....	43

Figure 4.10 CAN is a two-wire bus.....	44
Figure 4.11 CAN BUS Architecture.....	45
Figure 4.12 Difference between OSI model AND CIA.....	47
Figure 4.13 Data Frame.....	47
Figure 4.14 Remote Frame.....	48
Figure 4.15 Error Frame.....	48
Figure 4.16 The standard CAN frame format.....	49
Figure 4.17 The Extended CAN Frame Format.....	50
Figure 4.18 TRANSMIT MESSAGE FLOWCHART	51
Figure 4.19 CAN RECEIVE FLOWCHAR.....	52
Figure 5.1 Task priority levels in FreeRTOS illustrating that tasks with higher priority preempt those with lower priority to ensure real-time responsiveness.....	60
Figure 5.2 Different task states in FreeRTOS showing how tasks transition between Running, Ready, Blocked, and Suspended states during scheduling.....	60
Figure 5.3 Detailed flowchart illustrating the CAN Receive Task operations in FreeRTOS, including message reception	61
Figure 5.4 Flowchart illustrating the CAN Transmit Task operation within an RTOS environment. The diagram shows message queuing, interaction with CAN controller	62
Figure 5.6 Diagram illustrating the distinction between Onboard and Outboard components in a vehicle.....	64
Figure 5.7 Diagram illustrating the Over-The-Air (OTA) update process in automotive embedded systems. The diagram highlights key stages including update	66
Figure 6.1 ISO-TP (ISO 15765-2) Transport Layer used for UDS over CAN.....	71
Figure 6.2 UDS Protocol Stack as per ISO 14229 layered architecture.....	72
Figure 6.3 Typical UDS Diagnostic Session Request/Response Exchange This figure illustrates a basic request 10 03 and positive response 50 03, along with potential negative response codes (e.g., 7F 10 12) returned in case of failure.....	74
Figure 6.4 shows the full Security Access sequence including seed request, key generation, and positive/negative responses.....	76
Figure 6.5 Read Data By Identifier (0x22) Request/Response Exchange This diagram shows the request for a single DID.....	78

Figure 6.6 UDS DTC Read and Clear Sequence	
This diagram illustrates a typical sequence where the tester sends 19 02 FF to read DTCs and later sends 14 FF FF FF to clear them, with expected positive	80
Figure 6.7UDS Request Download (0x34) Service Flow	
This diagram illustrates the sequence of messages exchanged when initiating	82
Figure 6.8 DTC Storage Structure in Flash Memory (AUTOSAR-based)	
This diagram illustrates how fault codes.....	84
Figure 7.1 Flash memory layout for OTA-enabled STM32 system. Memory is partitioned into bootloader, metadata, and application regions.....	88
Figure 7.2Bootloader functional flow: from metadata parsing, chunk reception via CAN, erase/write to flash, RSA or CRC verification, and jump to the application firmware.....	92
Figure 7.3 Flowchart of fallback and fail-safe mechanism in an embedded OTA update system. The process begins with a system reset,.....	95
Figure 7.4 Mechanism for vector table remapping and jump-to-application in STM32 bootloader. The bootloader updates	98
Figure 8.1 Architecture of the Cloud Firmware Server.....	101
Figure 8.2 JSON Schema Structure for Firmware Metadata.....	103
Figure 8.3 Secure Firmware Upload to Firebase Storage	107
Figure 9.1 Typical CAN communication layers – SocketCAN vs traditional drivers.....	111
Figure 9.2 Data flow from sensor acquisition to dispatch using Observer and Reactor	111
Figure 9.3 OSI layer mapping of CAN and UDS protocols.....	114
Figure 9.4 OSI layer mapping of CAN and UDS protocols.....	115
Figure 9.5 OTA chunked upload process flow – handshake, upload loop, and finalization.	117
Figure 10.1 Layers of a GUI based on a windowing system.....	121
Figure 10.2 System architecture of live speed streaming via SSE	122
Figure 10.3 System architecture of live speed streaming via SSE.....	123
Figure 11.1 System Overview	140
Figure 11.2.....	140
Figure 11.3.....	140

LIST OF ABBREVIATIONS

ADC	Analog to Digital Converter
API	Application Programming Interface
CAN	Controller Area Network
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
GUI	Graphical User Interface
HMI	Human Machine Interface
HTTP	Hypertext Transfer Protocol
ID	Identifier
IOT	Internet of Things
JSON	JavaScript Object Notation
LED	Light Emitting Diode
OTA	Over-The-Air Update
RTOS	Real-Time Operating System
RSA	Rivest-Shamir-Adleman
SPI	Serial Peripheral Interface
S3	Amazon Simple Storage Service
TCP/IP	Transmission Control Protocol / Internet Protocol
UART	Universal Asynchronous Receiver/Transmitter
UDS	Unified Diagnostic Services

VCC Voltage at the Common Collector

NOMENCLATURES

T Temperature – Kelvin (K)

VCC Supply Voltage – Volts (V)

I Current – Ampere (A)

R Resistance – Ohm (Ω)

L Length – Meter (m)

f Frequency – Hertz (Hz)

t Time – Seconds (s)

D Data Size – Bytes

ID Message Identifier – —

CAN_H High Line Voltage – Volts (V)

CAN_L Low Line Voltage – Volts (V)

CHAPTER 1

INTRODUCTION

1 Chapter 1: Introduction

1.1 Background & Motivation

The automotive industry is rapidly evolving towards the concept of a "smart car," where modern vehicles are becoming much more than just a means of transportation. They have transformed into complex, connected systems that heavily rely on software and electronics. This evolution has led to an increasing need for effective and continuous software updates and maintenance of electronic systems. In the past, software updates and fault repairs required visits to service centers, which consumed significant time and effort for both users and manufacturers. With the growing complexity of these systems, immediate and accurate diagnosis has become crucial to ensure vehicle safety and performance. This highlights the importance of Over-the-Air (OTA) updates and real-time diagnostics via the CAN Bus network. OTA updates enable manufacturers to send software updates, bug fixes, and performance enhancements directly to vehicles without the need for manual intervention. This not only saves time and money but also ensures that vehicles always run on the latest software versions, thereby enhancing safety, efficiency, and user experience. Furthermore, the ability to diagnose faults and monitor vehicle status in real-time via the CAN Bus allows for early detection of issues, reducing the likelihood of major breakdowns and contributing to effective preventive maintenance. This project aims to address these challenges by developing an integrated system that combines OTA updates and real-time diagnostics using the CAN Bus. This system will provide a comprehensive solution for smart vehicle control, ensuring continuous updates and self-diagnosis capabilities, thereby improving their reliability and performance

1.2 Project Objectives

This project aims to achieve the following main objectives:

1. Develop an Over-the-Air (OTA) Update System: Build a robust and reliable mechanism for remotely updating the software of Electronic Control Units (ECUs) in the vehicle, reducing the need for manual intervention.
2. Implement Real-Time Diagnostics via CAN Bus: Enable continuous monitoring of vehicle status and fault diagnosis using the CAN Bus protocol, providing live data on the performance of various components.
3. Integrate UDS (Unified Diagnostic Services) Protocol: Utilize UDS as a standard protocol for diagnostic communication and update management between different control units.
4. Design a Graphical User Interface (GUI): Develop a user-friendly interface to display real-time vehicle data, manage update processes, and show diagnostic results.
5. Achieve Interaction Between Different Components: Ensure seamless integration and effective communication between STM32F103 units (App ECU and Main ECU), Raspberry Pi, and sensors via the CAN Bus network.
6. Provide an Integrated Smart Vehicle Control Solution: Offer a comprehensive system that enhances the smart vehicle's capabilities in self-updating and self maintenance, contributing to improved safety and efficiency.

1.3 Scope & Limitations

Project Scope:

OTA System Development: Includes the design and implementation of a mechanism for updating software on STM32F103 units via CAN Bus, with support for loading .bin files and storing them in Flash memory.

UDS Implementation: Involves the integration of essential UDS services (e.g., Session Control, Security Access, Read Data by Identifier, Clear DTC) for diagnostics and update management.

GUI Development: Encompasses the development of a web interface using Flask and HTML/CSS/JS to display live data, manage updates, and show diagnostics.

Hardware Integration: Covers the connection of STM32F103 units, Raspberry Pi, and sensors via CAN Bus, and setting up SocketCAN on Raspberry Pi.

FreeRTOS Usage: Includes task organization and synchronization on STM32F103 units using the FreeRTOS real-time operating system.

Project Limitations:

Hardware Scope: The project focuses on using STM32F103 and Raspberry Pi as primary platforms and may not cover compatibility with all types of control units or other platforms.

Update Security: While the project includes mechanisms for update verification, advanced aspects of update security (such as strong encryption or complex digital signatures) may be limited in this initial scope.

Diagnostic Complexity: The project focuses on basic UDS services and a limited number of Diagnostic Trouble Codes (DTCs). It may not cover all complex diagnostic scenarios or all types of sensors.

Experimental Environment: The system is developed and tested in a laboratory environment or as a prototype and may not be fully tested in a real vehicle environment with all operating conditions.

Scalability: Although the design considers some aspects of scalability, the project does not aim to build a production-ready system on a large scale that requires handling thousands of vehicles simultaneously.

1.4 Organization of the Thesis

This book is divided into the following chapters, each covering a specific aspect of the project:

Chapter 1: Introduction: Provides an overview of the project, its motivations, objectives, and scope.

Chapter 2: Technical Background: provides the necessary background knowledge required to comprehend the context, development, and implementation of the system.

Chapter 3: Hardware Architecture: Details the physical components used in the project, such as STM32F103 units, Raspberry Pi, and CAN Transceiver, as well as sensors.

Chapter 4: CAN Bus Networking: Explains the CAN protocol, how to set up communication between different devices, configure SPI for MCP2515, and set up CAN on STM32.

Chapter 5: Real-Time Operating System (FreeRTOS): Illustrates the basics of FreeRTOS, task design, and inter-task communication mechanisms.

Chapter 6: Unified Diagnostic Services (UDS) on CAN: Covers the UDS standard, various diagnostic services used, and their implementation.

Chapter 7: Bootloader & OTA Firmware Update: Explains the bootloader design, Flash memory partitioning, and the mechanism for receiving and storing updates via OTA.

Chapter 8: Cloud-Based Firmware Server: Describes the server architecture, how to host update files, and version verification mechanisms.

Chapter 9: Raspberry Pi Software Stack: Details the software components running on Raspberry Pi, such as SocketCAN, and diagnostic and OTA clients.

Chapter 10: Web GUI Design: Explains the technology choices for the web interface, interface design, live data display, and OTA update workflow.

Chapter 11: System Integration & Deployment: Discusses how to integrate all system components, auto-start settings, and security considerations.

CHAPTER 2

TECHNICAL BACKGROUND

2 Chapter 2: Technical Background

2.1 Overview of Automotive ECUs

ECUs are embedded systems responsible for controlling various electrical systems and subsystems in modern vehicles. Each ECU is designed to handle specific functions, such as engine management, transmission control, airbag deployment, and infotainment systems. These units communicate with each other through a network, typically using protocols such as the Controller Area Network (CAN) bus, to ensure coordinated operation and optimal vehicle performance.

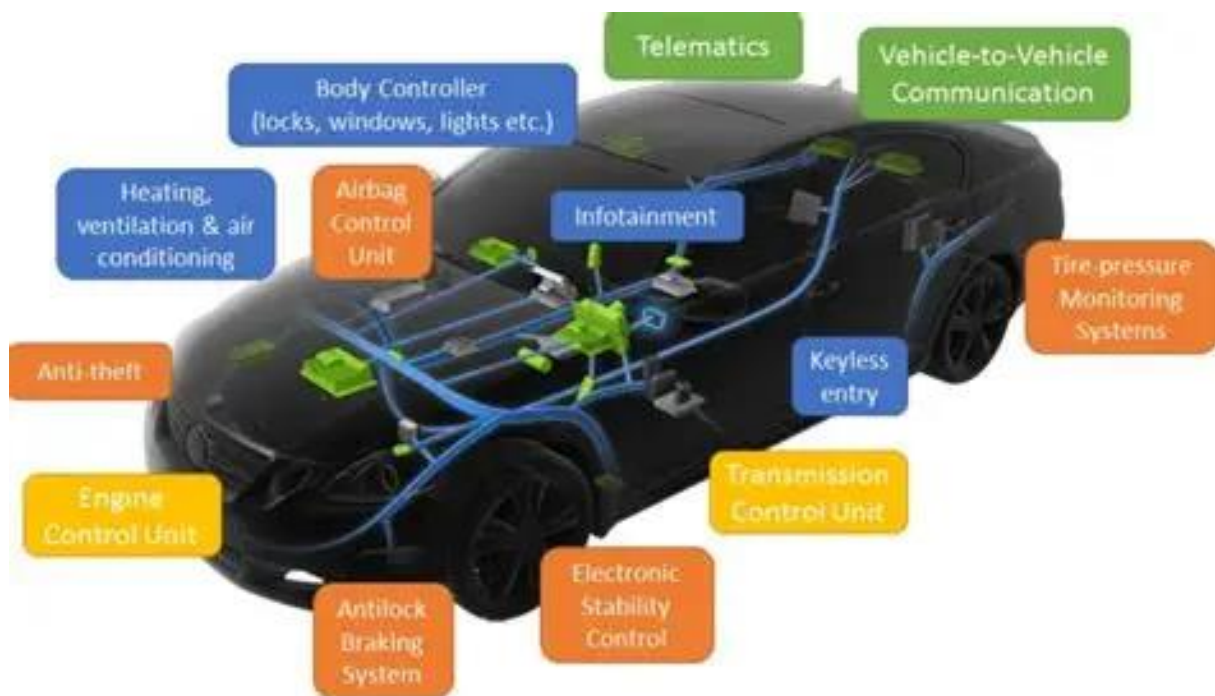


Figure 2.1 : Different ECUs functions in moder cars

In modern vehicles, there can be as many as 70 to 100 ECUs, each dedicated to different tasks. Some of the primary types of ECUs and their functionalities include:

- **Engine Control Unit (ECU):** Manages engine operations, including fuel injection, ignition timing, and emission controls, to ensure efficient performance and compliance with environmental regulations.
- **Transmission Control Unit (TCU):** Controls the operation of the transmission system, optimizing gear shifts for fuel efficiency and performance.
- **Brake Control Unit (BCU):** Manages the anti-lock braking system (ABS) and electronic stability control (ESC) to enhance vehicle safety during braking.
- **Airbag Control Unit (ACU):** Monitors crash sensors and deploys airbags in the event of a collision to protect occupants.
- **Body Control Module (BCM):** Manages various body-related functions such as lighting, climate control, and central locking.
- **Infotainment Control Unit:** Controls the vehicle's entertainment system, navigation, and connectivity features.

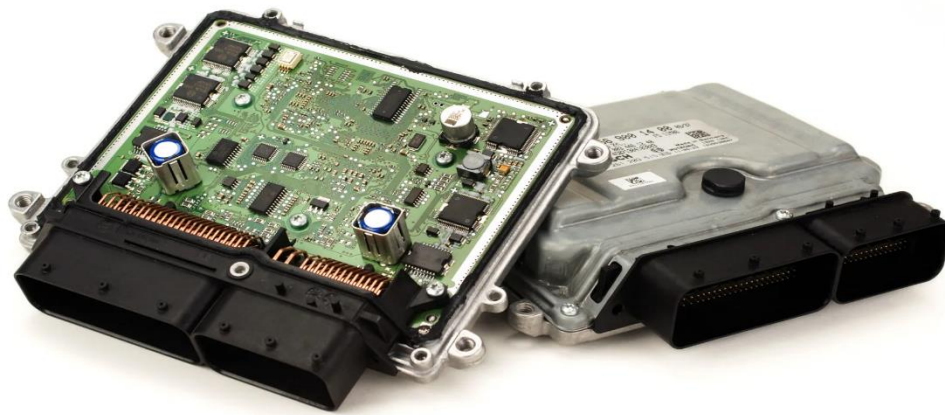


Figure 2.2 : Mercedes M156 ECU

The integration and communication between these ECUs are crucial for the overall functionality and safety of the vehicle. They must work seamlessly to provide reliable and efficient driving experience.

2.2 Firmware in Automotive Systems

Firmware is specialized software embedded in hardware devices, such as ECUs described before and used in automotive systems. Firmware aims to provide low-level control and enable the hardware to perform its intended functions. In automotive systems, firmware acts as the intermediary between the vehicle's hardware components and higher-level software applications, ensuring that each ECU operates correctly and communicates effectively with other units. However, managing and updating firmware presents several challenges:

- **Complexity:** Modern vehicles have a large number of ECUs with complex interdependencies, making firmware management a challenging task.
- **Security:** Ensuring that firmware updates are secure and free from malicious code is critical to prevent cyber-attacks.
- **Compatibility:** Firmware updates must be compatible with existing hardware and software configurations to avoid functionality issues.
- **Downtime:** Traditional methods of updating firmware require physical access to the vehicle, resulting in downtime and inconvenience for vehicle owners.
- **Version Control:** Keeping track of different firmware versions across multiple ECUs and vehicle models adds to the complexity of firmware management.

Addressing these challenges is essential for maintaining the reliability, safety, and performance of modern vehicles. The development of OTASync aims to tackle these issues by providing a robust solution for remote firmware updates and diagnostics, enhancing the overall efficiency and convenience of managing automotive firmware systems as it is more like impossible to disassembly the car.

2.3 Firmware Traditional Flashing Techniques

Flashing techniques are critical processes in the development and maintenance of automotive firmware. These techniques involve programming the firmware onto the

Electronic Control Units (ECUs) either before or after installation into the vehicle. Two primary methods are used: off-circuit programming and in-circuit programming.

2.3.1 Off –Circuit Programming

Off-circuit programming, also known as out-of-circuit programming, involves removing the ECU or its memory component from the vehicle or its circuit board. Then connected to a specific device or burner. The firmware is then programmed onto the ECU using specialized programming devices or burner. This method has several key characteristics:

- **Device Removal:** The ECU or its memory chip must be physically removed from the vehicle or the circuit board.
- **Programming Equipment:** Specialized programming devices, such as EPROM programmers or dedicated ECU flashing tools, are used to write the firmware.
- **Manual Handling:** The process requires manual handling and can be labor intensive.
- **Controlled Environment:** Typically performed in a controlled environment, such as a workshop or laboratory, ensuring optimal conditions for programming.



Figure 2.3 : Off-Circuit Programmer

While off-circuit programming can be reliable and allows for thorough testing before reinstallation, it is time-consuming and not suitable for rapid updates or large-scale systems like automotive systems.

2.3.2 In-Circuit programming

In-circuit programming (ICP) involves programming the firmware onto the ECU while it remains installed in the vehicle or circuit board. This method offers several advantages:

- **Non-Invasive:** The ECU does not need to be removed, making the process less invasive and more efficient.
- **Convenience:** Firmware updates can be performed directly on the vehicle, reducing the need for disassembly.
- **Flexibility:** Allows for easier updates and modifications, particularly useful during development and testing phases.

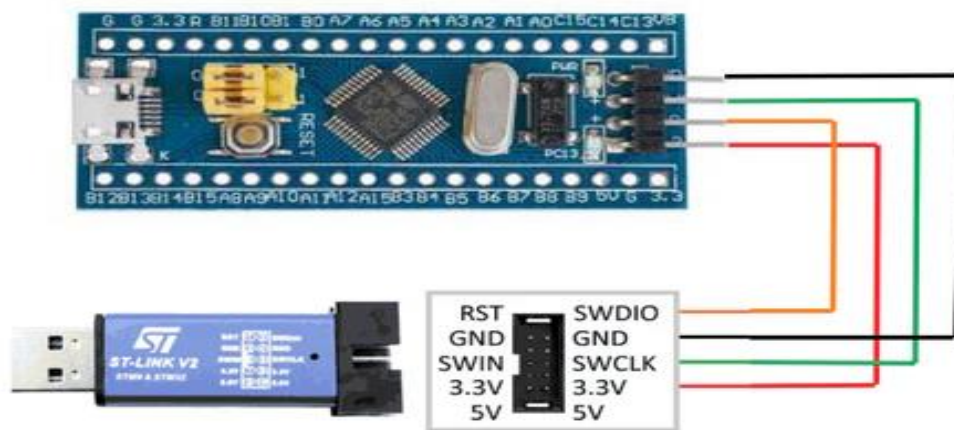


Figure 2.4 : ISP ST-Link Programmer

ICP typically uses specific communication interfaces such as JTAG, SPI, or CAN bus to transfer the firmware from a programming device or a computer to the ECU. This method streamlines the update process but can still be limited by the need for physical access to the vehicle and specific programming tool for each ECU.

2.3.3 The Evolution of OTA updates

As vehicles become increasingly complex and connected, traditional flashing techniques face several limitations:

- **Time and Labor Intensive:** Physical access to the vehicle and manual intervention are required, making the process time-consuming and labor intensive.
- **Scalability Issues:** Updating firmware on a large fleet of vehicles is challenging and costly.
- **Customer Inconvenience:** Vehicle owners must visit service centers for updates, leading to downtime and inconvenience.

To address these limitations, Over-the-Air (OTA) updates have emerged as a powerful solution. OTA updates enable remote delivery of firmware updates directly to the vehicle via wireless communication channels. The primary benefits of OTA updates include:

- **Convenience:** Updates can be performed remotely and anytime without requiring physical access to the vehicle.
- **Efficiency:** Reduces the time and labor associated with traditional flashing techniques.
- **Scalability:** Enables simultaneous updates across a large number of vehicles anywhere.
- **Enhanced Customer Experience:** Minimizes downtime and inconvenience for vehicle owners, as updates can be scheduled and performed automatically.

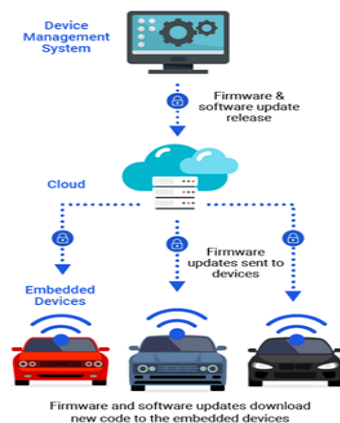


Figure 2.5: Over the air updates mechanism aka. FOTA

The adoption of OTA updates necessitates robust communication protocols and security measures to ensure the integrity and reliability of the updates. The development of OTASync leverages these advancements to provide a comprehensive solution for remote

firmware updates and diagnostics, enhancing the overall efficiency and effectiveness of automotive firmware management.

2.4 Diagnostic Services

2.4.1 Traditional Diagnostic Methods

Traditional vehicle diagnostics have long relied on direct, physical connections between diagnostic tools and the vehicle's on-board systems. These methods typically involve the following steps:

- **Physical Connection:** Mechanics connect diagnostic tools to the vehicle's OBD-II (On-Board Diagnostics) port using cables. The OBD-II port provides access to various ECUs and sensors within the vehicle.
- **Code Reading:** The diagnostic tool reads Diagnostic Trouble Codes (DTCs) stored in the vehicle's ECUs. These codes provide information about detected malfunctions and issues.
- **Data Analysis:** Technicians analyze the retrieved DTCs and other sensor data to identify the root cause of the vehicle's problems. This may involve further manual inspections and tests.



Figure 2.6 : Different types of OBD tools

- **Physical Access Requirement:** Diagnostics can only be performed when the vehicle is physically present at a service center, which can be inconvenient for vehicle owners.
-

- **Time-Consuming:** The process of connecting tools, reading codes, and interpreting data can be time-consuming, leading to longer service times.
- **Limited Data Access:** Traditional methods often provide limited access to real-time data, restricting the ability to perform comprehensive diagnostics.
- **Increased Downtime:** Vehicle owners may experience increased downtime as they wait for diagnostics and subsequent repairs.



Figure 2.7: OBD Connector in a car

2.4.2 Evolution of Wireless Diagnostics

The transition to wireless diagnostic methods represents a significant advancement in vehicle maintenance and repair. Wireless diagnostics leverage modern communication technologies to remotely access and analyze vehicle data. Key benefits of wireless diagnostics include:

- **Remote Access:** Technicians can remotely access a vehicle's diagnostic data without requiring physical presence, enabling quicker and more convenient diagnostics.
- **Real-Time Data:** Wireless diagnostics provide access to real-time data from the vehicle's sensors and ECUs, allowing for more accurate and timely analysis.
- **Reduced Downtime:** Vehicle owners experience less downtime as diagnostics can be performed remotely, and necessary repairs can be planned in advance.
- **Scalability:** Fleet operators can perform diagnostics on multiple vehicles simultaneously, improving efficiency and reducing operational costs.

- **Enhanced Customer Experience:** Vehicle owners benefit from faster service and fewer disruptions, leading to increased satisfaction and loyalty. Wireless diagnostics utilize various communication technologies, such as cellular networks, Wi-Fi, and Bluetooth, to establish connections between the vehicle and diagnostic systems. This connectivity allows for continuous monitoring and proactive maintenance, further enhancing vehicle reliability and performance.

2.4.3 Diagnostic Trouble Codes (DTCs)

Diagnostic Trouble Codes (DTCs) are standardized codes stored in a vehicle's ECUs when malfunctions or issues are detected. These codes play a crucial role in vehicle diagnostics by providing specific information about the nature and location of the problem. The process of working with DTCs involves several steps:

- **Logging DTCs:** When a sensor or system in the vehicle detects an issue, a corresponding DTC is generated and logged in the ECU. Each DTC consists of a letter followed by four digits (e.g., P0301), where the letter indicates the system (P for powertrain, B for body, C for chassis, and U for network) and the digits provide specific details about the fault.
- **Reading DTCs:** Technicians use diagnostic tools to read the stored DTCs from the vehicle's ECUs. Wireless diagnostic systems can remotely retrieve these codes, eliminating the need for physical connections.
- **Interpreting DTCs:** Each DTC corresponds to a specific issue, and technicians use reference manuals or software to interpret the codes. For example, a code P0301 indicates a misfire in cylinder 1 of the engine. Interpreting DTCs accurately is essential for diagnosing and resolving vehicle problems effectively.

DTCs are vital for efficient vehicle diagnostics as they provide a standardized method for identifying and addressing issues. By leveraging wireless diagnostic services, technicians can access and interpret DTCs more quickly and accurately, leading to faster and more effective repairs.

2.5 ISO UDS

2.5.1 Introduction to ISO Standards in Automotive

International Organization for Standardization (ISO) standards play a crucial role in the automotive industry by establishing uniform guidelines and protocols that ensure safety, reliability, and interoperability among vehicle components and systems. ISO standards cover various aspects of automotive design, production, and maintenance, providing a framework that manufacturers and suppliers can follow to achieve consistency and quality.

2.5.2 Overview of ISO 142292

ISO 14229, also known as Unified Diagnostic Services (UDS), is an international standard that specifies a set of diagnostic services used for vehicle diagnostics and communication between diagnostic tools and ECUs. UDS is a comprehensive protocol that supports a wide range of diagnostic functions, including reading and clearing diagnostic trouble codes (DTCs), Monitoring and controlling on-board systems, and performing firmware updates.

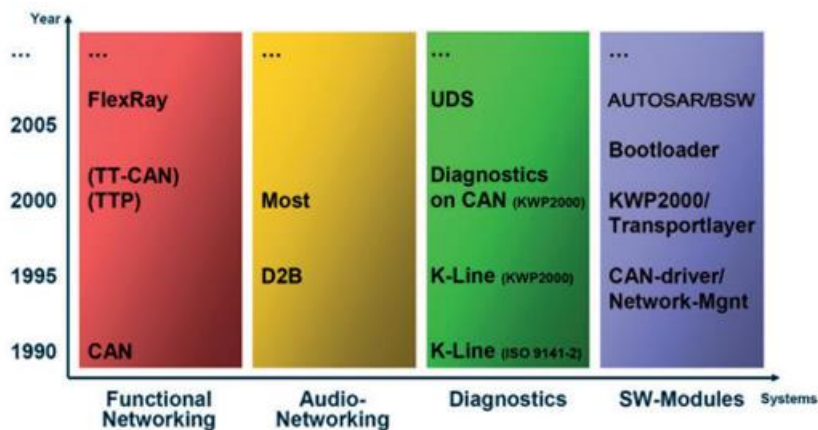


Figure 2.8 : Most important applied networking and diagnostic technologies over the past 15 years

2.5.3 Key Features of UDS ISO 14229

- **Standardized Diagnostic Services:** UDS defines a standardized set of diagnostic services that can be used across different vehicle manufacturers and models, ensuring interoperability and simplifying the diagnostic process.

- **Extended Diagnostic Functions:** UDS supports a broad spectrum of diagnostic functions beyond basic fault code reading, including system control, parameter setting, and routine testing.
- **Flexibility and Scalability:** The protocol is designed to be flexible and scalable, allowing it to be adapted to various vehicle architectures and communication networks.

2.5.4 Diagnostic Services Provided by UDS

UDS offers a wide range of diagnostic services categorized into different groups based on their functions:

- **Session Control:** Services that manage diagnostic sessions, allowing the transition between different diagnostic states (e.g., default session, programming session).
- **Security Access:** Services that provide security mechanisms to prevent unauthorized access to critical vehicle functions.
- **Read/Write Data:** Services for reading and writing data to/from ECUs, including real-time sensor data and configuration parameters.
- **Input/Output Control:** Services that enable control over specific inputs and outputs of the vehicle's systems, useful for testing and calibration.
- **Routine Control:** Services for executing specific routines or tests, such as emission tests or component activation.
- **Fault Memory Services:** Services that manage the storage, retrieval, and clearing of diagnostic trouble codes (DTCs) from the vehicle's fault memory.

2.5.5 Communication Protocols Supporting UDS

UDS can operate over various automotive communication protocols, enhancing its applicability and flexibility:

- **Controller Area Network (CAN):** UDS is commonly implemented over the CAN bus, leveraging its robustness and widespread use in automotive applications.
- **Local Interconnect Network (LIN):** UDS can also be used over the LIN bus for lower-speed communication in vehicle subsystems.
- **FlexRay:** For applications requiring higher data rates and fault tolerance, UDS can operate over the FlexRay protocol.
- **Ethernet:** Emerging automotive Ethernet networks support UDS, providing high-speed data transfer for advanced diagnostic and infotainment systems.

2.5.6 Benefits of Using UDS in Automotive Diagnostics

- **Interoperability:** UDS ensures compatibility across different diagnostic tools and vehicle models, simplifying the diagnostic process for technicians.
- **Efficiency:** The protocol supports a wide range of diagnostic functions, allowing for comprehensive vehicle diagnostics and maintenance.
- **Security:** UDS provides robust security measures to protect against unauthorized access and ensure the integrity of vehicle systems.
- **Scalability:** The flexible and scalable nature of UDS makes it suitable for various vehicle architectures and communication networks, supporting the evolving needs of the automotive industry.

2.6 Communication protocols Brief

2.6.1 CAN protocol

The Controller Area Network (CAN) is a multi-master, message-based serial bus protocol developed by Bosch in the mid-1980s for automotive and industrial applications. It allows microcontrollers and devices ("nodes") to communicate over a two-wire twisted pair without a dedicated host CPU.

Why We Use CAN:

Deterministic Real-Time Communication: Guaranteed timely delivery of high-priority messages.

Robustness: Differential signaling (CAN_H/CAN_L) provides strong noise immunity in electrically noisy environments.

Scalability: Easily add or remove nodes without changing the physical topology.

Cost-Effectiveness: Simple wiring harness (just two wires plus ground) reduces complexity and weight.

Reliability: Built-in error detection and automatic retransmission ensure data integrity.

Key Advantages:

1. Multi-Master Arbitration : Any node can initiate transmission; the lowest-ID (highest-priority) message wins arbitration without data collision.

2. Error Detection & Confinement : Five complementary error checks (bit-monitoring, bit-stuffing, frame check, ACK check, CRC) plus automatic fault isolation (error-passive/bus-off).

3. Flexible Data Rates : Standard speeds from 125 kbps up to 1 Mbps; newer CAN FD extends to 8 Mbps.

4. Standardized Higher-Layer Protocols : ISO-TP for segmented transport, UDS (ISO 14229) for diagnostics, and CANopen or J1939 for industrial/vehicle networks.

2.6.2 UART (Universal Asynchronous Receiver/Transmitter)

UART is a widely used serial communication protocol that enables asynchronous data exchange between devices. It is often employed in embedded systems for communication between microcontrollers, sensors, and peripheral devices.

How UART Works

- **Asynchronous Communication:** UART transmits data asynchronously, meaning there is no shared clock signal between the transmitter and receiver. Instead, both devices agree on a common baud rate (data transmission speed).
- **Data Frame Structure:** A typical UART data frame includes a start bit, data bits (usually 8 bits), an optional parity bit, and one or more stop bits. The start bit signals the beginning of data transmission, while the stop bits indicate the end.
- **Simple Wiring:** UART communication requires only two wires: TX (transmit) and RX (receive), making it straightforward to implement.

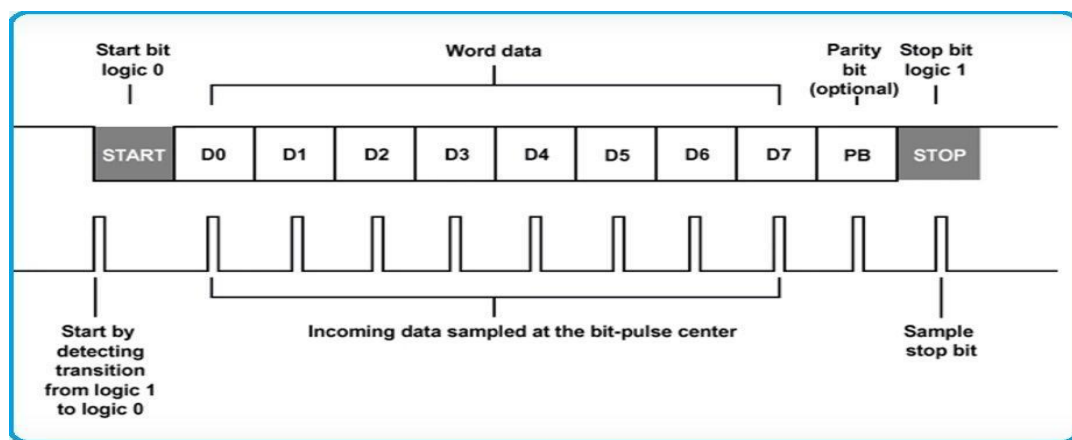


Figure 2.9 : Uart Frame Structure

2.7 RTOS

A Real-Time Operating System (RTOS) is a specialized operating system designed to manage hardware resources, run applications, and process data in real-time. Unlike general-purpose operating systems, an RTOS is built to provide predictable and deterministic response times, ensuring that critical tasks are executed within a specified time frame. This is particularly important in systems where timing and reliability are crucial, such as automotive, aerospace, industrial control, and telecommunications.

2.7.1 Why using RTOS

The primary reasons for using an RTOS in embedded systems include:

- **Deterministic Timing:** Ensures that high-priority tasks are executed within predictable time limits, essential for real-time applications.
- **Multitasking:** Efficiently manages multiple tasks, allowing them to run concurrently without interference.
- **Resource Management:** Provides robust mechanisms for managing hardware resources like CPU, memory, and I/O devices.
- **Inter-Task Communication:** Facilitates communication and synchronization between tasks, enhancing system coordination and reliability.
- **Reliability and Stability:** RTOSs are designed to be reliable and stable, minimizing the risk of system crashes and failures in critical applications.

2.7.2 Brief History of RTOS and Embedded Systems

The symbiotic relationship between Real-Time Operating Systems (RTOS) and embedded systems boasts a rich history, characterized by continuous adaptation to meet the demands of ever-evolving technology.

Early Foundations (1960s): The origins of RTOS can be traced back to the 1960s, coinciding with the introduction of computer systems specifically designed for real-time control applications. Pioneering efforts in this domain include the General Electric Real-Time Executive (GERTS), developed in the early 1960s for the GE 225 computer. These early RTOS solutions primarily focused on functionalities crucial for industrial automation and aerospace control systems.



Figure 2.10 : GE 225 Computer

Expansion with Minicomputers (1970s): The 1970s witnessed the rise of minicomputers, offering a more affordable and compact alternative to mainframe systems. This era spurred the development of a wider array of RTOS options, such as the RT-11 operating system used on Digital Equipment Corporation's PDP-11 minicomputers. These RTOS solutions catered to real-time tasks like process control and data acquisition in various industries.



Figure 2.11 : end of help command output from RT-11SJ display

The Birth of Commercial RTOS (1980s): A significant turning point arrived in the 1980s with the advancement of microprocessors. This decade saw the introduction of the first commercially available embedded RTOS kernel, the VRTX (Virtual Real-Time Executive), developed by Hunter & Ready Inc. in 1982. Designed for 8-bit and 16-bit microprocessors, VRTX found applications in automotive systems and consumer electronics, paving the way for widespread adoption of RTOS in embedded systems.

Continuous Development and Diversification (1990s-Present): The subsequent decades witnessed parallel advancements in microprocessor capabilities and the rise of the internet. This confluence of factors fueled the evolution of RTOS. The 1990s saw the development of more sophisticated RTOS solutions like VxWorks and QNX, capable of handling complex real-time applications in telecommunications, network routing, and multimedia. Today, the RTOS landscape is a diverse ecosystem offering a multitude of options tailored to the ever-expanding functionalities and demands of embedded systems.



Figure 16 : QNX logo one of most used RTOS currently

CHAPTER 3 HARDWARE ARCHITECTURE

3 Chapter 3: Hardware Architecture

3.1 STM32F103

The STM32F103xx medium-density performance line family incorporates the high-performance Arm® Cortex®-M3 32-bit RISC core operating at a 72 MHz frequency, high-speed embedded memories (Flash memory up to 128 Kbytes and SRAM up to 20 Kbytes), and an extensive range of enhanced I/Os and peripherals connected to two APB buses. All devices offer two 12-bit ADCs, three general purpose 16-bit timers plus one PWM timer, as well as standard and advanced communication interfaces: up to two I2Cs and SPIs, three USARTs, an USB and a CAN. The devices operate from a 2.0 to 3.6 V power supply. They are available in both the -40 to $+85^{\circ}\text{C}$ temperature range and the -40 to $+105^{\circ}\text{C}$ extended temperature range. A comprehensive set of power-saving mode allows the design of low-power applications.

The STM32F103xx medium-density performance line family includes devices in six different package types: from 36 pins to 100 pins. Depending on the device chosen, different sets of peripherals are included, the description below gives an overview of the complete range of peripherals proposed in this family.

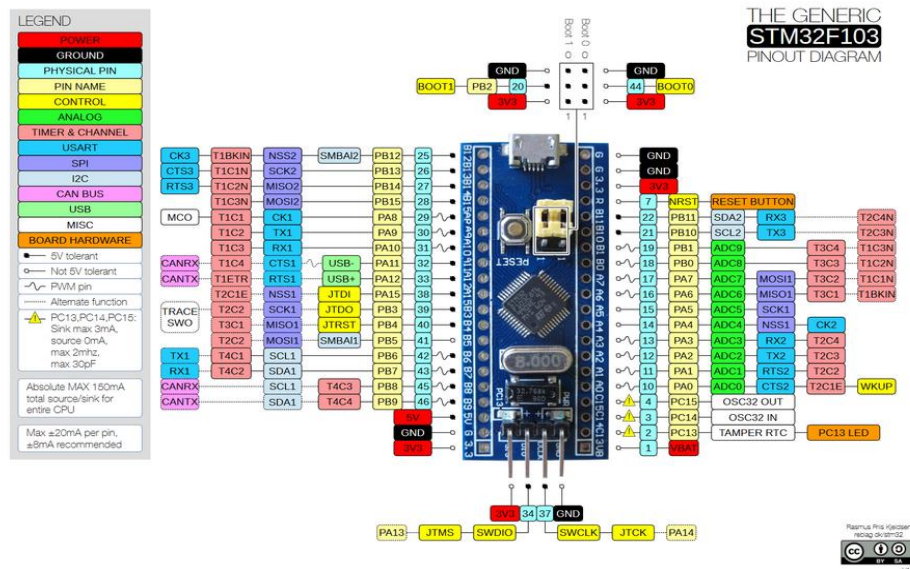


Figure 3.1.: STM32F103C8T6 Pinout

3.2 Raspberry Pi

What is a Raspberry Pi?

The Raspberry Pi is a small, single-board computer offered at an affordable price. It has become popular across a wide range of projects and applications, from hobbyist home automation to industrial uses. Raspberry Pi boards can run various operating systems and provide GPIO pins (General-Purpose Input/Output) that allow easy integration with sensors, actuators, and other hardware components. They come in different models with varying specifications, yet they all share the same core design philosophy: low cost, small form factor, and versatility.

Raspberry Pi OS (formerly known as Raspbian) is a Unix-like operating system based on the Debian GNU/Linux distribution, specifically tailored for the Raspberry Pi family of single-board computers distributed by the Raspberry Pi Foundation. Raspberry Pi OS has been heavily optimized for ARM-based CPUs found on the Pi and uses a modified LXDE desktop environment with the Openbox stacking window manager. It remains under active development, with ongoing efforts to improve stability and performance for as many Debian packages as possible on the Raspberry Pi.

Flashing Raspberry Pi OS onto the Raspberry Pi

The first step after obtaining a Raspberry Pi is to flash a micro-SD card with Raspberry Pi OS, insert it into the board, and then power on the device to boot into the OS

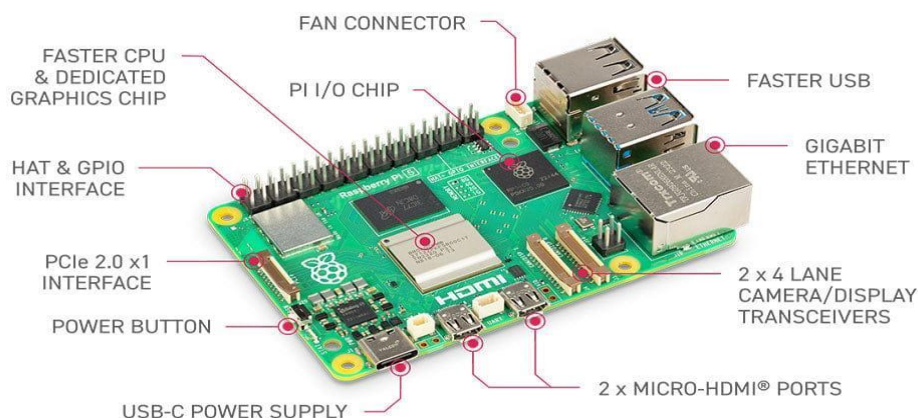


Figure 3.2: Raspberry Pi

3.3 CAN Hat

The RS485 CAN HAT will enable your Pi to communicate with other devices stably in long-distance via RS485/CAN functions.



Figure 3.3 : CAN Hat

The functionality of the CAN module involves handling all message reception and transmission on the CAN bus. When sending a message, it is first loaded into the appropriate message buffer and control registers. Sending operations can be initiated by setting the corresponding bits in the control registers via the SPI interface or by using the transmit enable pin. Communication status and errors can be checked by reading the respective registers. Any messages detected on the CAN bus undergo error checking, and then are matched with user-defined filters to determine whether the message should be moved to one of the two receive buffers. As the Raspberry Pi itself does not support the CAN bus, we can use the CAN controller with the SPI interface, along with a transceiver, to realize CAN functionality.

Microchip's MCP2515 is a CAN protocol controller that fully supports the CAN V2.0B technical specification. The device can send and receive both standard and extended data frames as well as remote frames. With two acceptance mask registers and six acceptance filter registers built in, the MCP2515 can filter out unwanted messages, reducing the overhead on the main microcontroller (MCU). The MCU connects to the device via the SPI interface, that is, the Raspberry Pi connects to the chip through the SPI interface. For Raspberry Pi, using this chip does not require writing drivers; instead, simply enabling the kernel driver in the device tree is sufficient for usage.

3.4 CAN Transceiver (SN65HVD01)

CAN Transceiver: It converts the data from the CAN controller to the CAN bus levels and converts the data from the CAN bus levels to a suitable level that the CAN controller uses.



Figure 3.4: CAN BUS Transceiver mcp2551

The transceiver drives or detects the dominant and recessive bits by the voltage difference between the CAN_H and CAN_L lines. The nominal dominant differential voltage is between 1.5V to 3V and the recessive differential voltage is always 0V. The CAN transceiver actively drives to the logical 0 (dominant bits) voltage level and the logical 1 (recessive bits) are passively returned to 0V by the termination resistor. The idle state will always be at the recessive level (logical 1). Individually, CAN_H will always be driven towards supply voltage (VCC) and CAN_L towards 0V when transmitting to the dominant (0). But in a practical case, supply voltage (VCC) or 0V cannot be reached due to the transceiver's internal diode drop. CAN H/L will not be driven when transmitting a recessive (1) where the voltage will be maintained at $V_{CC}/2$.

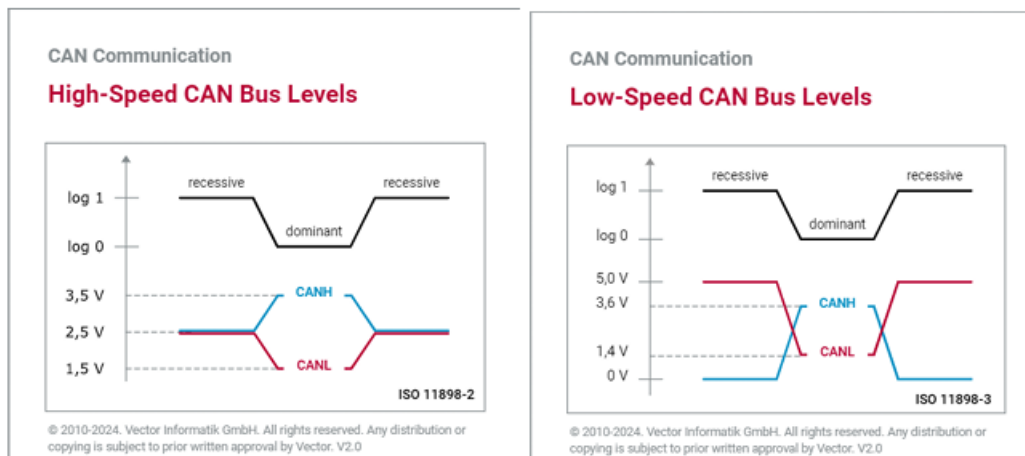


Figure 3.5: High-speed CAN BUS and Low-speed CAN BUS.

3.5 Sensors

3.5.1 Temperature Sensor (LM35)

The LM35 is a precision integrated analog temperature sensor developed by Texas Instruments. It provides an output voltage that is linearly proportional to the temperature in degrees Celsius. This sensor offers several advantages over traditional thermistors, as it does not require any external calibration or signal conditioning circuitry. The LM35 is factory-calibrated to provide a typical accuracy of $\pm 0.5^\circ\text{C}$ at room temperature (25°C).

It operates across a wide temperature range from -55°C to $+150^\circ\text{C}$, making it suitable for various industrial, automotive, and embedded applications. One of the main benefits of using the LM35 is its low self-heating due to minimal current consumption (typically less than $60\mu\text{A}$), which results in higher measurement accuracy. The output voltage increases by 10 mV per degree Celsius, which allows for straightforward conversion to temperature using simple arithmetic

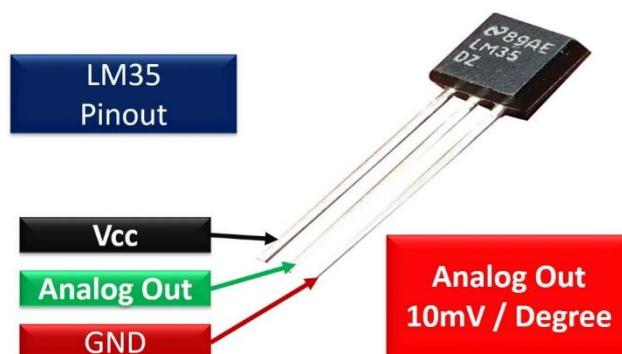


Figure 3.6 : Photographs and pin-configuration diagrams of the LM35

3.5.2 Ultrasonic Sensor

The HC-SR04 is a commonly used ultrasonic distance measurement sensor, especially in embedded systems and robotic applications. It operates by emitting an ultrasonic pulse and measuring the time taken for the echo to return after reflecting off a nearby object.

This time delay is then used to calculate the distance to the object using the known speed of sound in air (approximately 343 m/s at room temperature).

The sensor consists of two components:

A transmitter (TRIG), which emits a burst of 40 kHz ultrasonic waves.

A receiver (ECHO), which detects the reflected wave from an object

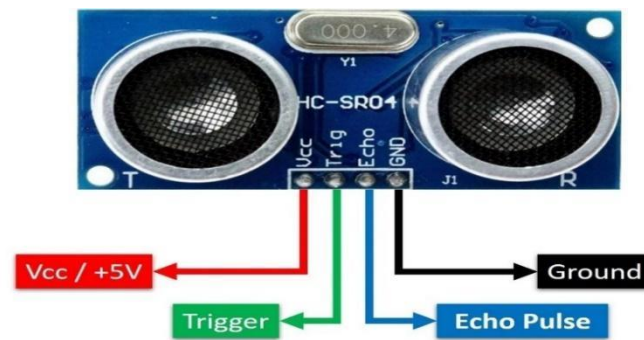


Figure 3.7 : Close-up view of the HC-SR04

3.6 Actuators & Motor Drivers (L298N)

DC Motors (Actuators) : DC motors are widely used in embedded and robotic systems due to their simplicity, ease of control, and reliability. In this project, two brushed DC motors are used to drive the miniature vehicle's left and right wheels. These motors convert electrical energy into mechanical rotational motion, which directly translates into vehicle movement. By varying the direction and speed of the current applied to each motor, the vehicle can move forward, reverse, or perform differential steering (turning left or right by adjusting motor speed individually). Each motor operates at a nominal voltage of 6–12 V, drawing a current typically between 300–500 mA under load. For reliable and independent control



Figure 3.8 : DC Motors

Motor Driver (L298N)

The L298N is a dual full-bridge driver IC capable of controlling two DC motors independently. It integrates two H-bridge circuits, allowing bidirectional current flow through the motors, which is essential for forward and reverse motion. The IC is capable of delivering up to 2 A per channel at voltages up to 46 V, making it ideal for controlling small to medium-sized DC motors.

The L298N module includes an onboard 5 V voltage regulator, enabling it to supply logic voltage to the microcontroller if needed. It features:

1. Two enable inputs (ENA, ENB) used to activate each H-bridge independently.
2. Four logic inputs (IN1–IN4) for controlling motor direction.
3. Two outputs (OUT1–OUT4) connected directly to motor terminals.
4. A logic power input (VSS), motor power input (VS), and ground (GND).

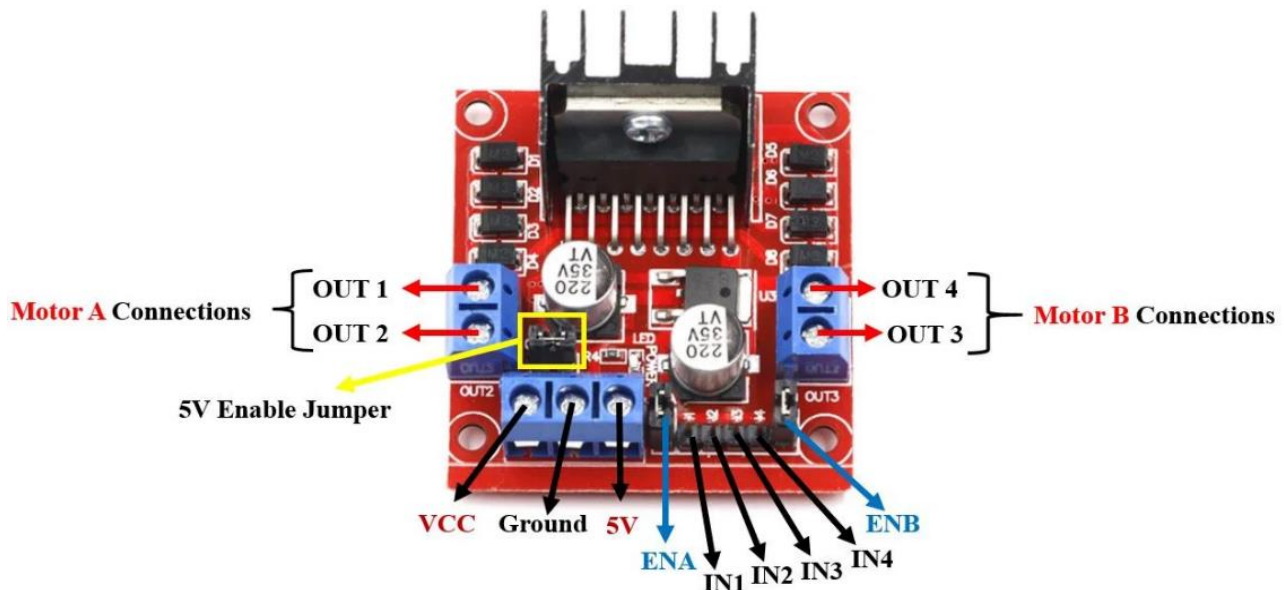


Figure 3.9 : Motor Driver (L298N)

CHAPTER 4

CAN NETWORK

4 Chapter 4: CAN Network

4.1 Introduction

4.1.1 Before CAN

Today's vehicles are complex machines incorporating mechanical and electronic parts. The number of electronic components used in vehicles has rapidly increased in recent years. As a result of the increase in safety, comfort, and performance requirements, we see many more electronic components being added to modern vehicles. Henceforth, there has been an increasing demand to connect these electronic components in such a way that they can communicate with each other reliably, safely, and in real time.

In the past, electronic units used to be connected in a complex way, with hundreds of wires running to distinct parts of a vehicle. Consequently, it was difficult to trace an electronic fault. The history of the CAN bus starts when there were only a few electronic components in automobiles and automotive companies preferred point-to-point wiring systems, where data lines were drawn individually from each component. With the expansion of the automobile industry, more electronic components were incorporated into designs to meet increasingly sophisticated demands.

CAN Introduction

Networking

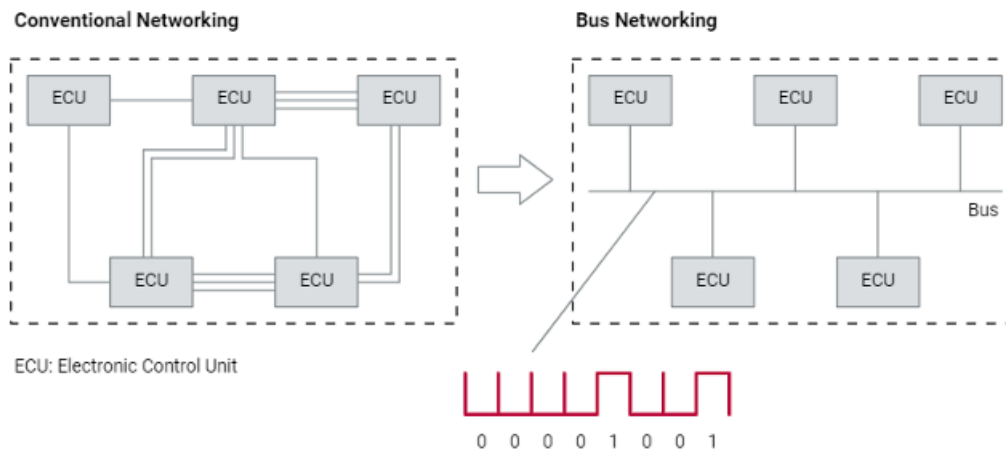


Figure 4.1: Networking Before and After CAN

There was no coordination between various parts of the electronics, as each electronic part was controlled independently of the others. Maintenance and repair of the vehicle electronics were extremely difficult, as in many cases it was not easy to locate and change a faulty component. Passenger safety has also become one of the most important considerations in today's vehicles. In the past decade, safety equipment has evolved and moved from physical to electronic assisted safety, starting with braking and type technology, through collision protection and airbags, and most recently, on to safety-related driver assistance systems. The latest modern vehicles are smart and intelligent machines and are equipped with many sensors that can evaluate the surroundings and display useful and safety-related information to drivers. These sensors form intelligent local networks together with actuators, displays, and fast digital processors, such as high-speed purpose-built microcontrollers. A traditional old-style vehicle electronic system with sensors and actuators interconnected to each other in a complex manner. One of the major problems in this type of design is maintenance. The wiring was so complex that it was nearly impossible to trace and rectify faults.

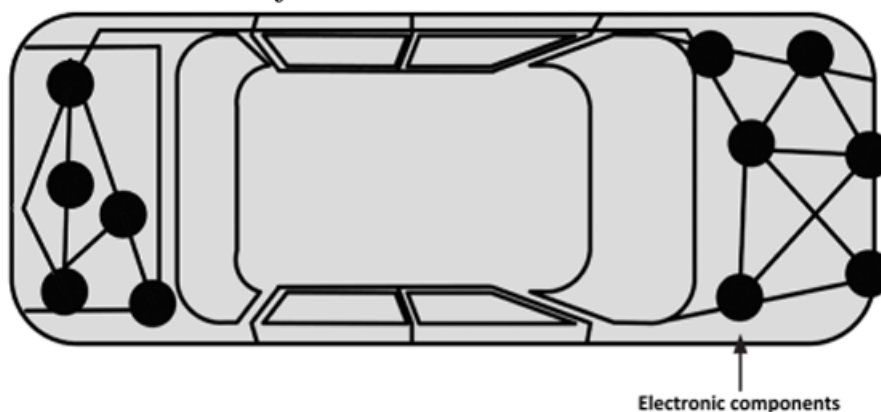


Figure 4.2 : Traditional old-style vehicle electronics.

4.1.2 A typical wiring harness for a car

the wiring that connects the electronic and electric components is so complex and extensive that, were it laid end to end connection, all those cables would stretch over a mile. This cable Obviously, complexity and weight both add significant costs, as well as energy consumption while driving



Figure 4.3 : wiring harness for a car

4.1.3 The Evolution of In-vehicle Network Architectures

The development of CAN began when more and more electronic devices were implemented into modern motor vehicles. A modern vehicle, even if not fully featured, already has 70 to 100 ECUs, with over 2500 signals to transmit internally. To manage increased complexity and higher data rates as new versions of existing protocols find their way into vehicle networks (CAN FD, Ethernet).

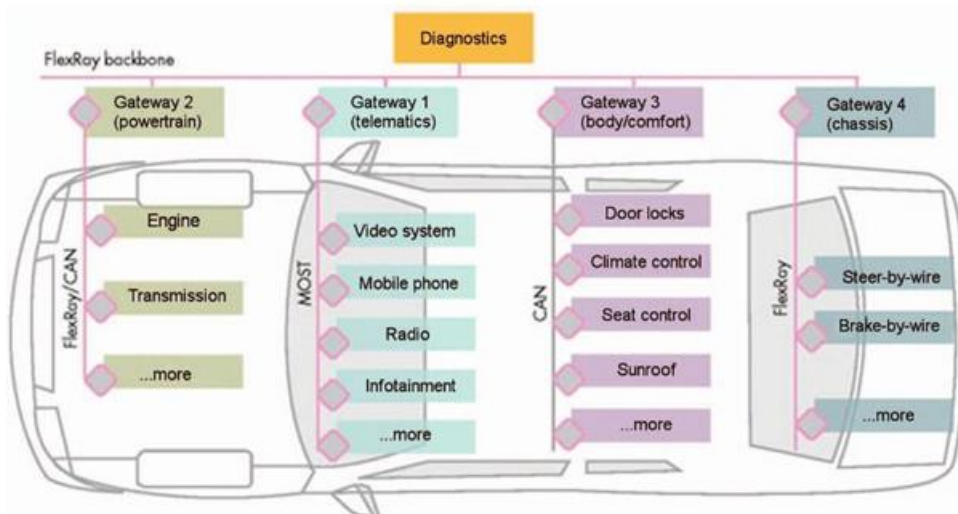


Figure 4.4 : In vehicle network

The drive for fuel-efficient and safer vehicles opened the door to electronic control in vehicles, which in turn led to the deployment of In-vehicle Networks (IVN). IVNs have become the backbone of modern vehicles. The volume of data flowing through these networks is increasing exponentially with demands for electric vehicles, advanced driver assistance systems (ADAS), radar, lidar, infotainment systems, cameras and

vehicle-to-vehicle communications systems. To meet this need, the automotive industry working with technology suppliers has developed specialized communications protocols and application specific extensions to existing network technologies, standardized under the aegis of organizations like ISO and IEEE, and it continues to investigate new topologies and protocols to improve performance, increase reliability and lower the costs of IVNs.

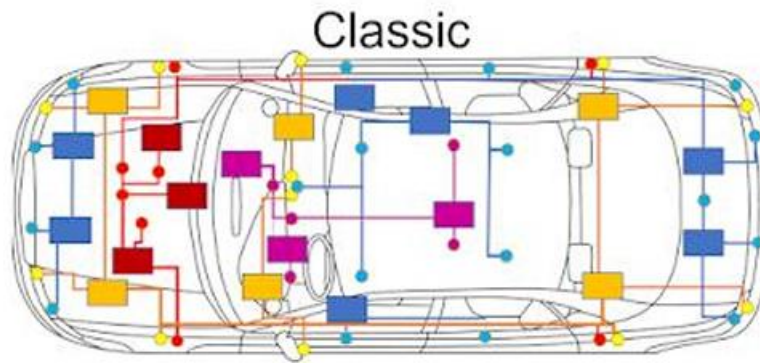


Figure 4.5 : IVNs Classical architecture

In the Classic architecture, different Engine Control Units (ECUs) all in different locations in the vehicle are connected to each other. Normally, they are placed near to the sensors or the motor depending on the function of the ECU. The result is that everything in the IVN is connected by a huge, heavy and costly cable set. The number of electronic control units (ECUs) in vehicles is continuing to rise. A current mid-range vehicle might have 70 ECUs, while a luxury vehicle might have as many as 150. Connecting these devices is challenging, and vehicle manufacturers seek to consolidate capabilities into fewer devices to reduce complexity and cost. Hence, the evolution of the Domain architecture is currently used in newer vehicles.

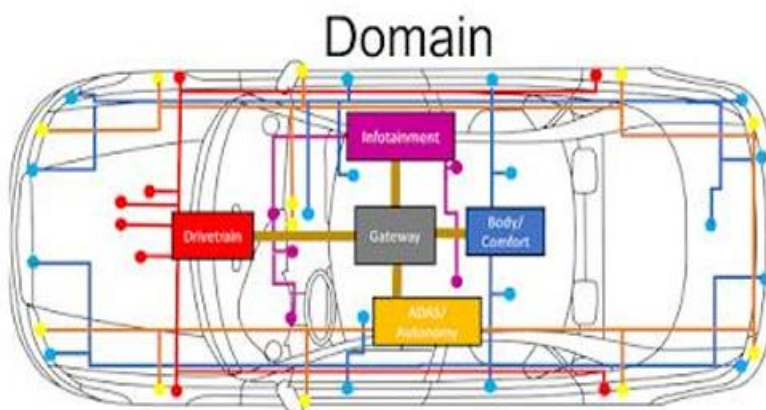


Figure 4.6 : In the Domain IVN architecture, domain controllers organize the ECUs under them

The ECUs are more centralized and are divided into functional groups or “domains”. For example, there is one domain for infotainment functions, another domain for the drivetrain, etc., all connected to each other through a gateway. The different domains may use different physical layers, which means different bus systems. Some may use Ethernet while others use CAN. The gateway handles translation between the different domains and ensures smooth, safe and correct communication between all ECUs. The “next generation” IVN architecture is called Zonal architecture. In the Zonal architecture, main controllers called zone controllers are in different sections of the car (e.g., front left, front right). The sensors and actuators are connected directly to the zone controller, and because the zone controller is close to the interfaced devices, the cable lengths required to connect them are relatively short, resulting in less weight and expense. This architecture reduces the number of wires in the harness, and there are fewer ECUs overall replaced by much more powerful, centralized ECUs for the different regions of the car.

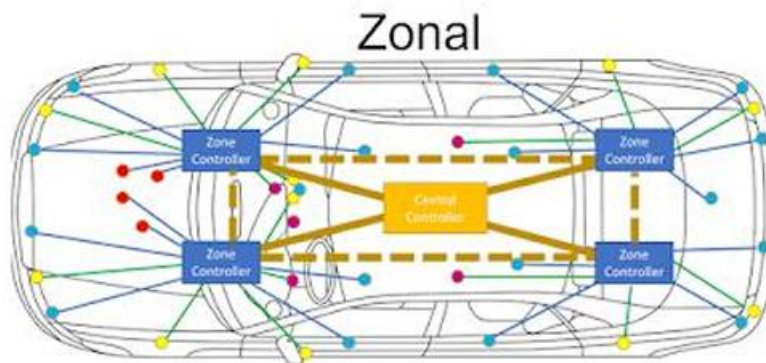


Figure 4.7 : the Zonal INV architecture

A central controller connected to the zone controllers over a high-speed data “backbone” performs data fusion and higher-level decision-making functions. The backbone also serves to provide the data redundancy needed especially for autonomous driving. Because of the volume of data exchanged between the controllers, as well as the need to connect to external high-speed networks, higher speed and higher throughput are essential to this architecture, especially for the central controller and backbone functions. It is necessary to be able to move data at speeds well above 10 Mbit/S. All IVN designs involve multiple buses because some operations do not require a high data rate and the attendant bandwidth. The power train or body/comfort domains are normally slower networks. In most current architectures, only a limited number of functions, like infotainment or ADAS systems, absolutely require high bandwidth. Some functions that do not need a high data rate are very important nonetheless and need a high priority. For example, safety functions require a very high priority, but only have a medium bandwidth requirement. So, for many functions, a data rate of 10 Mbit/S is as high as required. Yet, until recently, few protocols operated in the gap between 1 and 10 Mbit/S.

4.1.4 Why Serial Communication?

1. Simplicity and Cost: Serial communication requires fewer wires (typically just one or two for data transmission) compared to parallel communication, which reduces the complexity and cost of cabling and connectors.

2. Long Distance Transmission: Serial communication is better suited for long distance data transmission. The reduced number of wires minimizes signal degradation and interference, allowing data to be transmitted over longer distances without significant loss of quality.

3. Data Integrity: Serial communication protocols often include error detection and correction mechanisms, ensuring higher data integrity during transmission. This is particularly important in environments with high electromagnetic interference.

4. Synchronization: Many serial communication protocols include synchronization mechanisms that help ensure that data is correctly interpreted by the receiving device. This is essential for maintaining data accuracy, especially in asynchronous communication.

5. Flexibility and Scalability: Serial communication can be easily scaled to support multiple devices using multiplexers, daisy-chaining, or bus networks (e.g., RS-485). This flexibility makes it suitable for complex systems with multiple components.

6. Speed and Efficiency: Advances in serial communication technologies, such as USB, SPI, and I2C, have significantly increased data transfer rates. Modern serial protocols can achieve high-speed communication, making them efficient for various applications.

7. Compatibility: Serial communication standards like RS-232, RS-485, and USB are widely used and supported by many devices and systems, ensuring broad compatibility and ease of integration.

8. Reduced Crosstalk and Interference: The serial transmission of data, often coupled with differential signaling (e.g., RS-485), reduces crosstalk and electromagnetic interference, resulting in more reliable data transmission.

4.2 CAN Overview

4.2.1 What is CAN?

CAN is short for 'controller area network'. A controller area network is an electronic communication bus defined by the ISO 11898 standards. Those standards define how communication happens, how the wiring is configured, and how messages are constructed, among other things. Collectively, this system is referred to as a CAN bus. The Controller Area Network (acronym: CAN) structure was originally developed for use in passenger cars. Today, sophisticated CAN controller chips are available from over 20 manufacturers, and CAN is finding applications in other fields such as medical, aerospace, process control, automation, and so on. With the establishment of CAN within the Automation (CIA) association back in 1992, manufacturers and users rallied to exchange ideas and develop the CAN standards and specifications. Vehicle network systems (or networks) were classified in 1994 by the Society of Automotive Engineers (SAE). According to this classification, bus systems were classified based on their bandwidth (i.e., data rate) and functions of the network system. The classification divides bus networks into four: Class A, Class B, Class C, and Class D.

- Class A networks are low-speed, low-cost networks with data rates of less than 10 kbps. These systems are mainly used in the body of the car.
- Class B networks operate between 10 and 125 kbps and are used for information exchange, e.g., instrument cluster, vehicle speed, and so on.
- Class C networks operate between 125 and 1 Mbps and are used in a wide range of applications, such as engine control.
- Class D networks operate above 1 Mbps, and they are mainly used in telematics applications.

There are many automotive network systems (or bus systems, since the vehicle networks are in the form of bus wires), some developed by vehicle manufacturers on their own, and some developed jointly with semiconductor manufacturers. Popular vehicle network systems include:

- CAN bus
- LIN bus
- FLEXRAY
- MOST
- And others...

4.2.2 Features of CAN protocol

CAN is a serial, two-wire multi-master bus developed by Robert Bosch GmbH in the 1980s. It is one of the most widely used automotive electronic system buses today. The physical layer of CAN consists of a pair of twisted cables. CAN provides dependable, robust, and fast communication up to 1 Mbps (with a 40 m bus length). CAN 2.0A is the original CAN version consisting of the fields:

- Start of Frame bit
- 18-bits header (having 11-bit message identifier)
- 0-8 bytes data
- 15-bit Cyclic Redundancy Check (CRC)
- 3-bit acknowledgement slot
- 7-bit of End of Frame

The CAN bus is based on the CSMA/CR (Carrier Sense Multiple Access/Collision Resolution) "mechanism" to prevent frame collisions during transmissions on the bus. Each CAN node monitors the bus and when the node detects that the bus is idle, it may start transmitting data. If other nodes on the bus attempt to send data at the same time, arbitration will take place and the node with the highest priority (i.e., lowest message identifier) will win the arbitration and send its own data. The CAN bus has a simple error detection-and-recovery mechanism. Receiving nodes check the integrity of the messages by looking at the CRC fields. If an error is detected, the other nodes on the bus are informed through error flag messages. shows a typical CAN bus implementation with two nodes, A and B. CAN is a Class A/B type network.

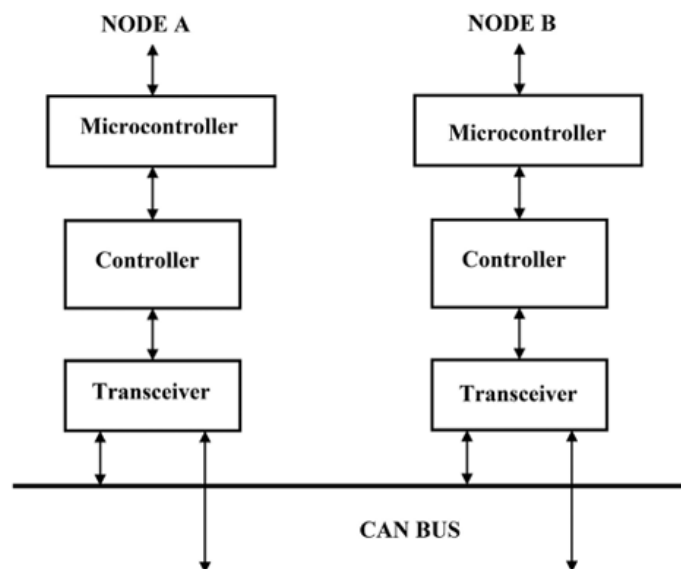


Figure 4.8 : CAN Bus with two nodes

4.2.3 Comparison of automotive bus systems

Currently, one of the most used bus systems is the CAN bus, which can be used at speeds up to 1 Mbps. In general, the LIN bus is used in low-speed automation, such as wiper motor control, rain sensors, etc. CAN is used in engine control, clutch control, and so on. FLEXRAY is used in very high-performance and safety-critical applications. This is a relatively new bus structure and is hoped to be the automotive standard of the future. MOST is generally used in automotive multimedia, and navigation applications. This bus is optical-based and is tolerant to electrical noise. Byteflight is also a new automotive network system and is well suited to high-demand real-time safety critical applications, such as airbag control. Byteflight is based on fiber optics and as such is much more tolerant to electromagnetic radiation and electrical noise. Intellibus, finally, offers high bandwidth and is mainly used in aerospace applications. It is not as safe and dependable as Byteflight.

	Class	General	Bandwidth	Application
CAN	A, B	- Twisted pair - Low-cost - Widely used - Multiple masters	1 Mbps	- Body - Engine
LIN	A	- Low-cost - Low-speed - Master/slave - Single wire	10 kbps	- Body - Comfort
FlexRay	D	- Very high-speed - Future standard - Master/slave (for synchronization)	10 Mbps	- Engine - Safety
MOST	D	- Data efficient - Standard for multimedia - Master/slave (for synchronization) - Fiber-optic	Mbps	Multimedia
Byteflight	D	- High-speed - Master/slave - Fiber-optic	10 Mbps	Safety critical
Intellibus	D	- High-speed	15 Mbps	- Body - Engine - Safety

Table 1: Automotive bus systems

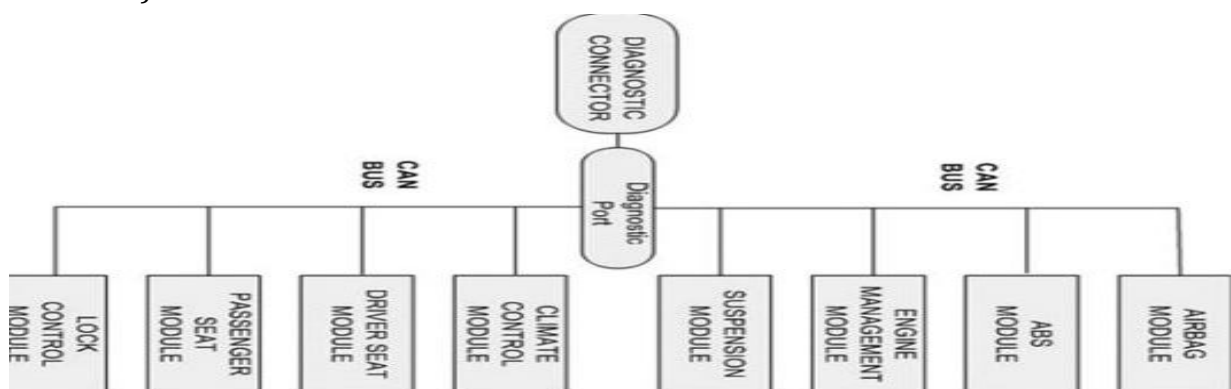
CAN is one of the most dominant bus protocols used in passenger cars. Since the year 2000, CAN usage in vehicles has rapidly increased from approximately 120 million CAN devices to around 800 million devices in the year 2007. Currently, this figure is over tens of billions of devices. CAN and CAN-based higher-level protocols such as CANopen and DeviceNet are becoming more widely accepted by North American manufacturers and it is expected that these protocols will have very high penetrations in most of the industrial and commercial automation markets. CAN will be used in the following diverse applications within the next five to ten years:

- Passenger cars
- Buses
- Trains
- Space applications
- Etc.

An interesting and important application of the CAN bus will almost certainly be in the medical industry. Medical equipment and devices such as X-ray machines, ultrasound, radiotherapy machines, CAT scanners, MRI machines, and so on could be designed and developed to make use of the CAN bus for their internal communication structures. This approach will simplify the design of machines as well as increase their safety and reliability, also helping to improve ease of maintenance.

4.3 The basic structure of a CAN bus automotive system

In a CAN bus-based automobile, all units are connected to each other over a two-wire bus system. The two wires of the bus are named CAN_L and CAN_H. There is full coordination between all the units, and the ECU is responsible for controlling each unit, making sure that all units operate as expected. CAN is a two-wire bus system with resistive connectors at each end of the bus. All the units (or nodes, or CAN-stations) are connected rigidly to the bus using CAN connectors (e.g., T-connectors, End-connectors, and so on).



In general, information is received from various sensors (e.g., engine temperature sensor) located in various parts of the vehicle. The sensor information is then passed to a microcontroller, and then to the CAN bus via a CAN controller and transceiver. The function of the CAN transceiver is to make physical connection to the actual CAN bus for communication over the bus. The CAN controller is under the control of the microcontroller and performs the CAN protocol specific functions.

4.3.1 Physical Layer

An unshielded twisted two-wire line is the physical transmission medium used most frequently in applications (Unshielded Twisted Pair UTP), over which symmetrical signal transmission occurs. Typically, UTPs have wire cross sections between 0.34 mm² and 0.6 mm². Line resistance should be less than 60 mΩ.

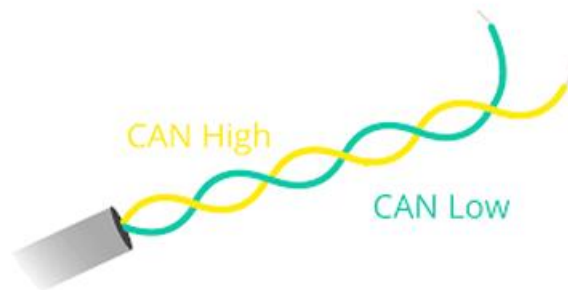


Figure 4.10 : CAN is a two-wire bus.

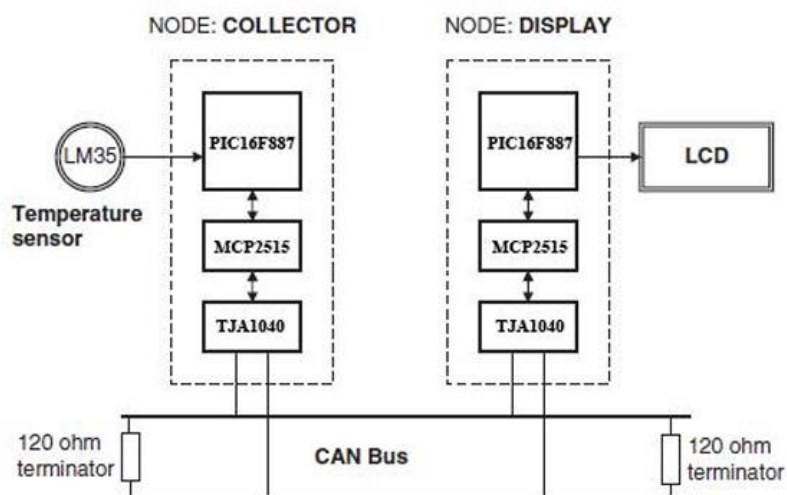


Figure 4.11 : Sensors on the CAN bus

4.3.2 Boundary conditions

The maximum data rate is 1 Mbit/s. A maximum network extension of about 40 meters is allowed. At the ends of the CAN network, bus termination resistors contribute to preventing transient phenomena (reflections). ISO 11898 specifies the maximum number of CAN nodes as 32.

4.3.3 Integration

Regardless of the CAN controller type, CAN controllers may be integrated, or they may — as shown in the diagram — be used as a stand-alone chip component. In this case, the microcontroller treats the CAN controller like a memory chip. The stand-alone variant is more flexible, but the on-chip variant has the advantage of requiring less space, and communication between the microcontroller and the CAN controller is faster and more reliable.

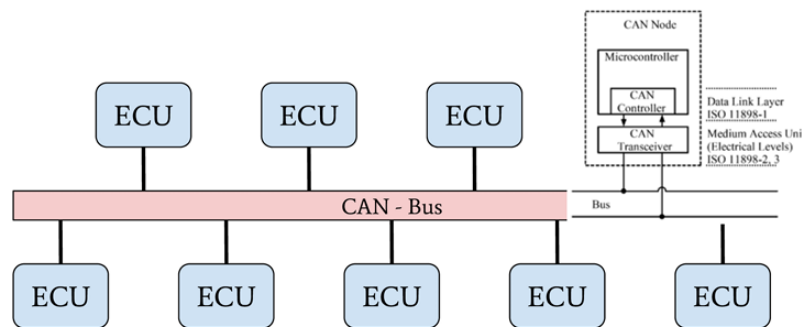


Figure 4.12 : CAN BUS Architecture

4.4 CAN BUS advantages and disadvantages

4.4.1 CAN bus advantages

The main benefits of the CAN bus can be summarized as follows:

- Low wiring complexity
- Easy to manage twisted-pair wire bus
- New nodes can be easily added and removed
- Breakdown of a node does not affect the other nodes in the system
- Centralized control
- All devices on the bus can read the same message
- Fail-safe against electromagnetic radiation
- Deals effectively with errors
- Easy diagnostic and maintenance

4.4.2 CAN bus disadvantages

Some of the disadvantages of the CAN bus include:

- the data rate can be low in some applications (limited to 1 Mbps).
- expensive to implement as special controllers are required.
- complete system shutdown if the main CAN trunk wire is damaged.

4.4.3 CAN bus main properties

CAN bus communication is not like the popular Client-Master type of communication. In the CAN bus, all nodes have the same rights, and they can transmit as well as receive data at suitable times. The main properties of the CAN bus can be summarized below:

- Two-wire twisted-cable bus
- Bus-terminated with resistors at each end
- Up to 1 Mbps speed on the bus (for a 40-m length bus). Higher data rates for shorter bus lengths
- Transfer of up to 8 data bytes at any time
- Messages sent and received as frames under a clearly defined robust protocol
- Multi-master priority-based access to the busAny one node can transmit while other nodes listen
- Bus arbitration if two or more nodes attempt to transmit at the same time
- Broadcast-type message transfer where all nodes can receive the same message
- No node addresses. Nodes accept or reject data on the bus based on message acceptance filtering
- Remote data request where a node can request data from another node • Error detection and signaling
- Automatic retransmission of messages if failed to transmit (because of the bus arbitration)
- Automatic deactivation of nodes that present consistent errors

4.5 CAN Bus

Frames Messages on CAN bus is sent and received using frames. A frame is basically like a packet in a TCP/IP-type network, where the actual data is encapsulated with control data.

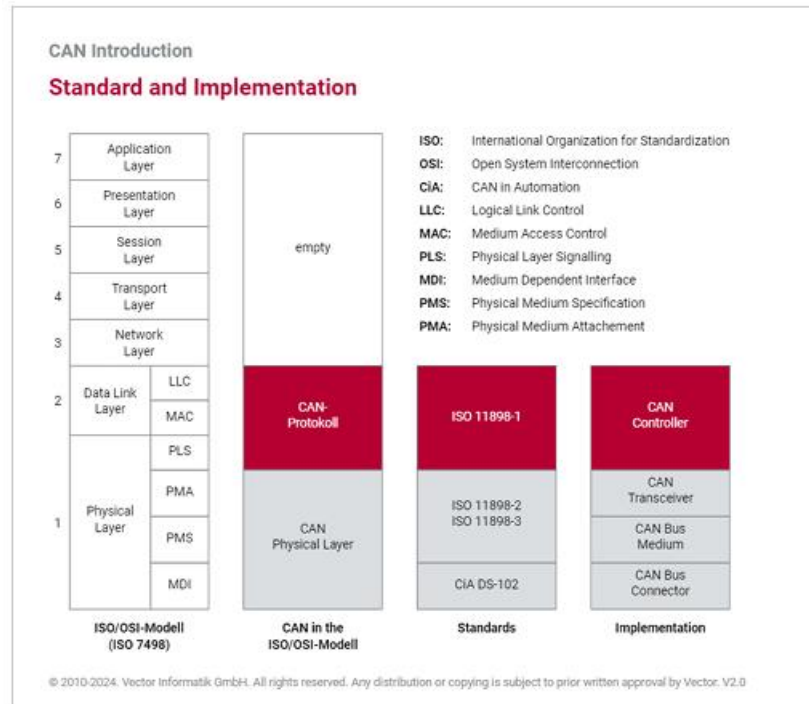


Figure 4.13 : Difference between OSI model AND CIA

There are four message frames in CAN bus:

- **Data frames:** These are most used frames and used when a node transmits information to any or all other nodes in the system. Data Frames consist of fields that provide additional information about the message as defined by the CAN specification. Embedded in the Data Frames are Arbitration Fields, Control Fields, Data Fields, CRC Fields, a 2-bit Acknowledge Field, and an End of Frame.

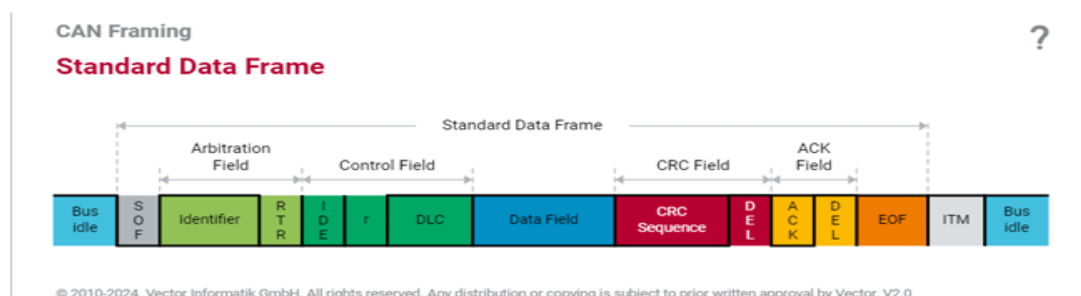


Figure 4.14: Data Frame

- **Remote Frame:** Used by a node to request transmission of a message (i.e., data) from another node.

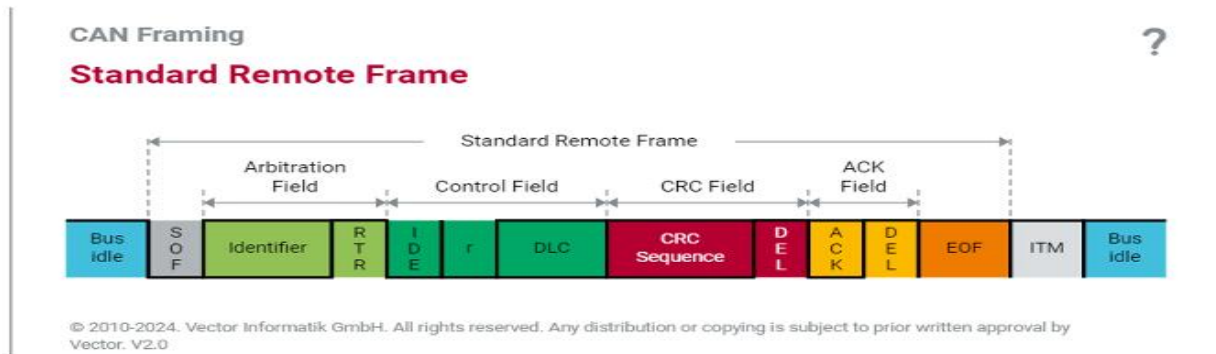


Figure 4.15 : Remote Frame

- **Extended frame:** The purpose of the remote frame is to seek permission for the transmission of data from another node. This is like the data frame without the data field and the RTR bit is recessive.
- **Error frames:** If the transmitting or receiving node detects an error, it will immediately abort transmission and send an error frame consisting of an error flag made up of six dominant bits and an error flag delimiter made up of eight recessive bits. The CAN controller ensures that a node cannot tie up a bus by repeatedly transmitting the error frame.

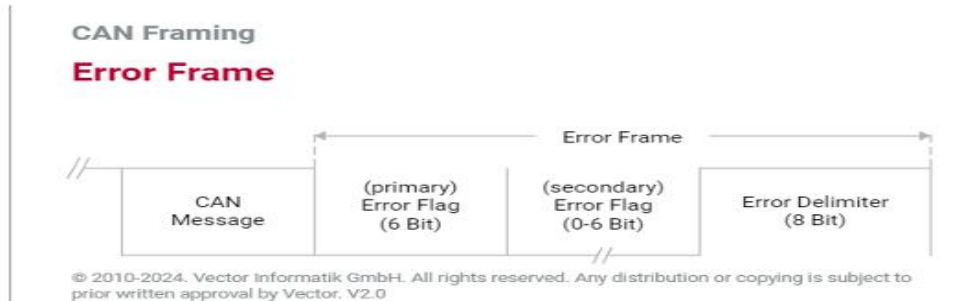


Figure 4.16: Error Frame

- **Overload frame:** It is like the error frame but used for providing an extra delay between the messages. An Overload frame is generated by a node, when it becomes too busy and is not ready to receive.

4.5.1 Can Network Message Format

CAN devices send data across the CAN network in packets called frames. These frames can be differentiated based on identifier fields. A CAN frame with 11-bit identifier fields is called Standard CAN and with a 29-bit identifier, fields called the extended frame.

1. Standard frame:

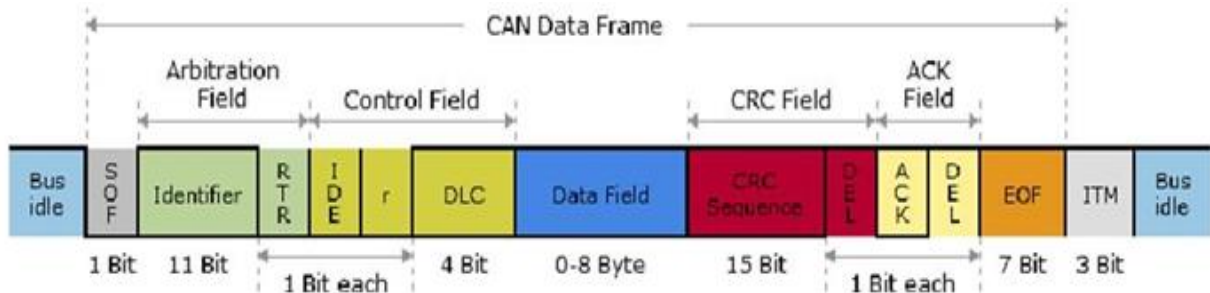
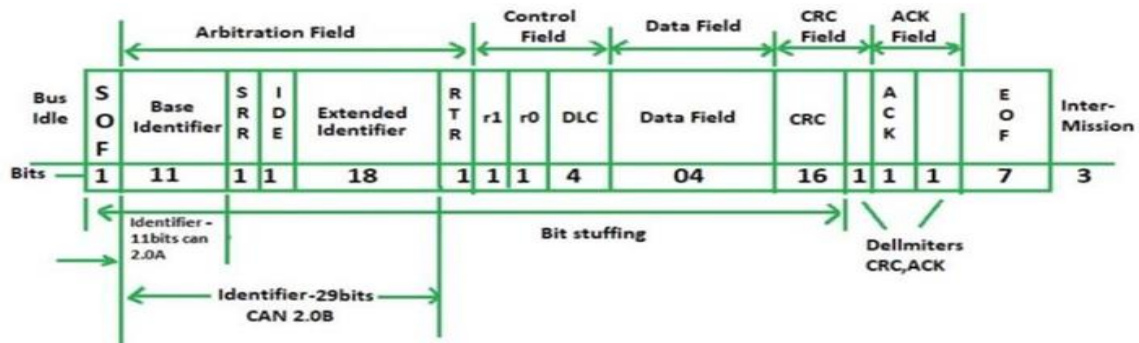


Figure 4.17 : The standard CAN frame format

- SOF (start-of-frame) bit – indicates the beginning of a message with a dominant (logic 0) bit.
- Arbitration ID – identifies the message and indicates the message's priority. Frames come in two formats -- standard, which uses an 11-bit arbitration ID, and extended, which uses a 29-bit arbitration ID.
- RTR (remote transmission request) bit – serves to differentiate a remote frame from a data frame. A dominant (logic 0) RTR bit indicates a data frame. A recessive (logic 1) RTR bit indicates a remote frame.
- IDE (identifier extension) bit – allows differentiation between standard and extended frames.
- R0 – Reversed bit. Not used currently and kept for future use.
- DLC (data length code) – indicates the number of bytes the data field contains.
- Data Field – contains 0 to 8 bytes of data.
- CRC (cyclic redundancy check) – contains a 15-bit cyclic redundancy check code and a recessive delimiter bit. The CRC field is used for error detection.
- ACK (Acknowledgement) slot – It comprises of the ACK slot and the ACK delimiter. When the data is received correctly the recessive bit in the ACK slot is overwritten as a dominant bit by the receiver.
- EOF (End of Frame) – the 7-bit field marks the end of a CAN frame (message) and disables.

2. Extended frame:



29-bit CAN Frame Format

Figure 4.18 : The Extended CAN Frame Format

It is the same as an 11-bit identifier with some added bits. SRR (Substitute Reverse Request) – The SRR bit is always transmitted as a recessive bit to ensure that, in the case of arbitration between a Standard Data Frame and an Extended Data Frame, the Standard Data Frame will always have priority if both messages have the same base (11 bit) identifier. R1– It is another bit not used currently and kept for future use.

4.5.2 CAN in Action

For the next couple of flow chart, we will illustrate how to transmit and receive a message based on our blue pill board, black pill board, raspberry pi board and our CAN unit Driver. a) Transmitting a CAN message: Transmitting a message typically requires loading the identifier, RTR, ID, Data length, and the Data into Transmit structure.

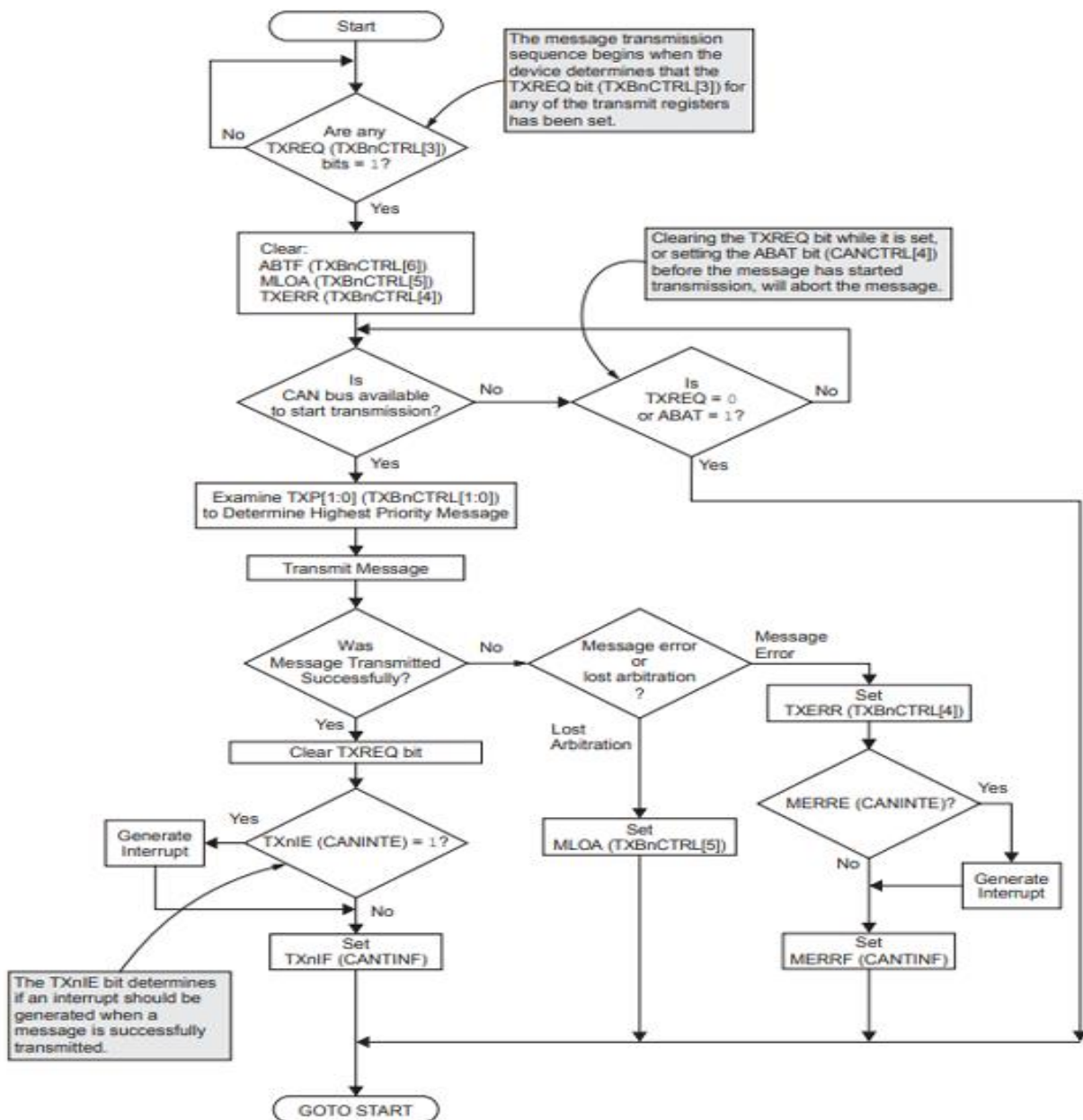


Figure 4.19 : TRANSMIT MESSAGE FLOWCHART

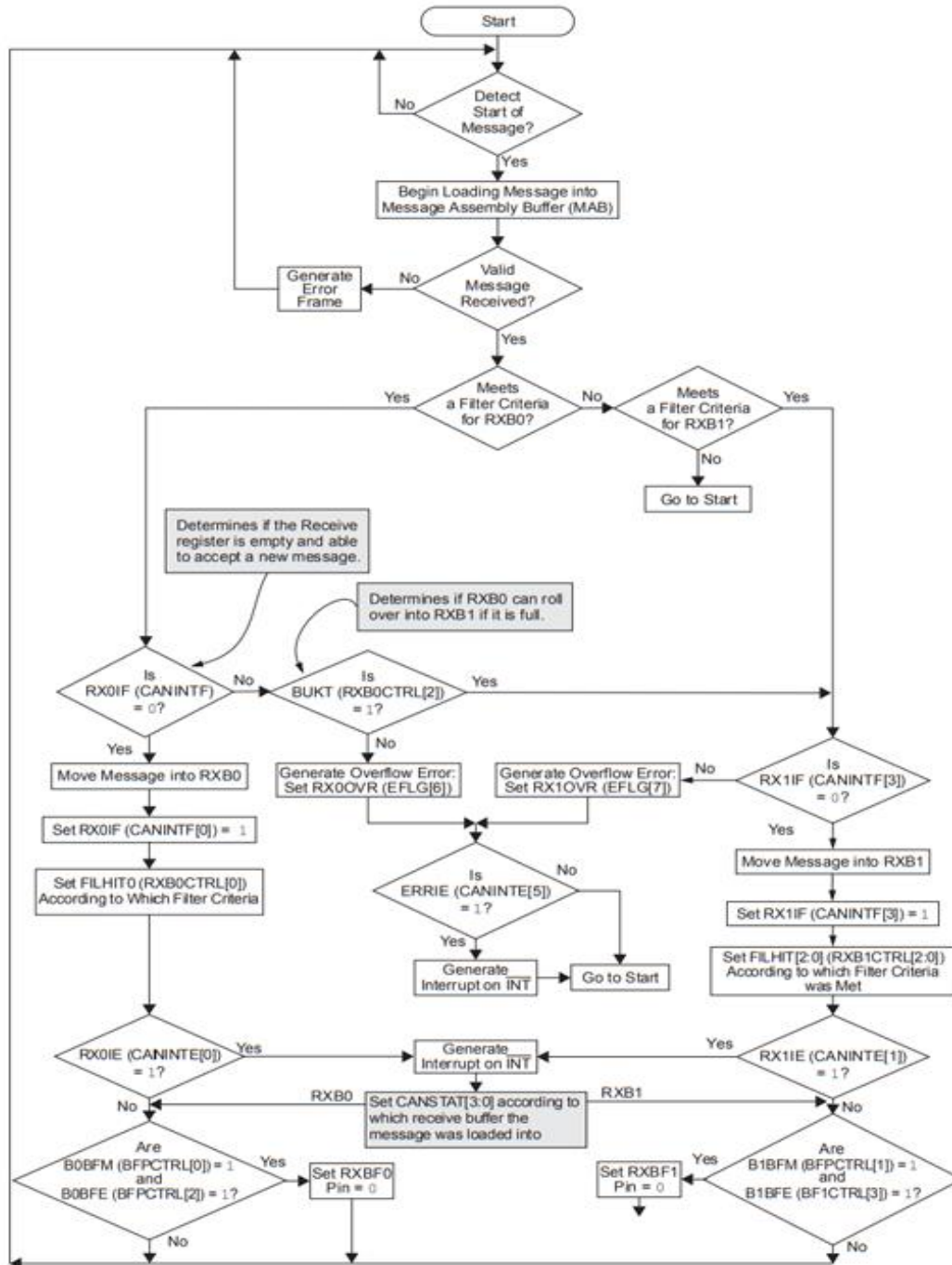


Figure 4.20 : CAN RECEIVE FLOWCHAR

CHAPTER 5

REAL-TIME OPERATING SYSTEM (FRERTOS)

5 Chapter (5): Real-Time Operating System (FreeRTOS)

5.1 RTOS Fundamentals

Real-Time Operating Systems (RTOSs) are essential software components in embedded systems that require **predictable timing behavior** and **deterministic task execution**. Unlike General Purpose Operating Systems (GPOS), which focus on maximizing resource utilization and user interactivity, RTOSs are designed to guarantee that high-priority tasks are executed **within strict timing constraints**, regardless of system load.

In this project, **FreeRTOS** has been adopted as the main RTOS due to its portability, scalability, and wide support across microcontroller architectures such as the ARM Cortex-M family. FreeRTOS is a **lightweight, open-source** kernel that enables preemptive multitasking, inter-task communication, and efficient scheduling — making it highly suitable for embedded automotive systems.

Key Differences Between RTOS and GPOS:

Feature	RTOS	GPOS
Timing Behavior	Deterministic (hard/soft real-time)	Non-deterministic
Task Scheduling	Priority-based, preemptive	Fairness-oriented (time-sharing)
Interrupt Latency	Minimal and predictable	Variable and dependent on load
Footprint	Small (few KBs)	Large (MBs to GBs)

Use Case	Embedded, safety-critical systems	Desktop, server, mobile platforms
-----------------	-----------------------------------	-----------------------------------

Core Features of FreeRTOS:

1. **Preemptive Multitasking:** Supports context switching between tasks based on priority.
2. **Task Management:** Developers can define multiple tasks with independent stacks and execution logic.
3. **Synchronization Mechanisms:** Semaphores, mutexes, and direct-to-task notifications for coordinating access to shared resources.
4. **Inter-task Communication:** Queues and stream buffers allow safe data exchange between tasks.
5. **Portability:** Supports various microcontroller architectures including STM32 (ARM Cortex-M3).
6. **Scalability:** Allows developers to enable only required components, minimizing overhead.

By integrating FreeRTOS into the system architecture, the software achieves **modularity, timing predictability, and safe task isolation** — which are crucial in automotive applications like CAN message handling, motor safety control, and diagnostics.

Figure 5.1-1: Comparison Between RTOS and GPOS:

RTOS vs GPOS

Mandalay Technological University
Department of Computer Engineering and Information Technology

RTOS	GPOS
Process data typically <u>without buffer delays</u>	random delays into application software and <u>may cause slow responsiveness</u> of an application at unexpected times
deterministic	Non-deterministic
Time sensitive	Time insensitive
Can't use virtual memory	Use virtual memory
Dedicated to single work	Used in multi-user environment
Low interrupt latency	High interrupt latency

Lecture Slides for Textbook *Introduction to Embedded Systems*, Moe Moe Myint, 2017-2018

5.2 Task Design & Priorities

Designing tasks and assigning appropriate priorities are fundamental steps in developing a deterministic real-time system using FreeRTOS. Each task represents an independent thread of execution with its own stack and context, allowing concurrent operation managed by the RTOS scheduler [1].

The scheduler uses task priorities to decide which task should execute at any given time. Higher priority tasks preempt lower priority ones, ensuring that critical functions meet their timing requirements. For instance, in automotive embedded systems, tasks handling motor control and CAN message reception often have higher priorities due to their safety and real-time nature [2].

A well-designed task architecture also addresses issues such as **priority inversion**, where a high-priority task is blocked waiting for a resource held by a lower-priority task. This problem is commonly mitigated by employing **priority inheritance protocols** provided by FreeRTOS mutexes, which temporarily elevate the priority of the task holding the resource [3].

Task design further involves defining appropriate stack sizes to prevent overflow, balancing CPU utilization, and ensuring that each task performs atomic operations where necessary to maintain data integrity.

Practical Example of Task Design and Priority Management in FreeRTOS:

```
#include "FreeRTOS.h"
```

```
#include "task.h"

#include "semphr.h"

// Mutex to protect shared resources
SemaphoreHandle_t xResourceMutex;

// Task function declarations
void MotorSafetyTask(void *pvParameters);
void CANReceiveTask(void *pvParameters);
void DiagnosticsTask(void *pvParameters);
int main(void)
{
    // Create mutex with priority inheritance support
    xResourceMutex = xSemaphoreCreateMutex();

    // Create tasks with assigned priorities
    xTaskCreate(MotorSafetyTask, "MotorSafety", 256, NULL, 3, NULL); // High priority
    xTaskCreate(CANReceiveTask, "CANReceive", 256, NULL, 2, NULL); // Medium
    priority
    xTaskCreate(DiagnosticsTask, "Diagnostics", 256, NULL, 1, NULL); // Low priority

    // Start the scheduler
    vTaskStartScheduler();

    // Should never reach here
    for (;;)
    }

void MotorSafetyTask(void *pvParameters)
```

```
{  
    for (;;)   
    {  
        // Critical task that requires immediate execution  
        // Example: Monitor safety sensors and trigger emergency shutdown  
        // Safety control code here  
        vTaskDelay(pdMS_TO_TICKS(10)); // Delay for 10 ms  
    }  
}  
  
void CANReceiveTask(void *pvParameters)  
{  
    for (;;)   
    {  
        // Wait for incoming CAN data and process it  
        // Take mutex to safely access shared resources  
        if (xSemaphoreTake(xResourceMutex, portMAX_DELAY) == pdTRUE)  
        {  
            // Safely process CAN data here  
            xSemaphoreGive(xResourceMutex);  
        }  
        vTaskDelay(pdMS_TO_TICKS(5)); // Delay for 5 ms  
    }  
}
```

```
void DiagnosticsTask(void *pvParameters)
{
    for (;;)
    {
        // Execute less time-critical diagnostics tasks
        vTaskDelay(pdMS_TO_TICKS(100)); // Delay for 100 ms
    }
}
```

Code Explanation:

- **Mutex (xResourceMutex):** Protects shared resources like CAN data buffers from concurrent access.
- **Priority Inheritance:** FreeRTOS mutex supports temporarily boosting the priority of the task holding the mutex, preventing priority inversion.

- **Priority Assignment:**

MotorSafetyTask is assigned **highest priority (3)** due to its critical safety function.

CANReceiveTask runs at **medium priority (2)** to handle live CAN data.

DiagnosticsTask has the **lowest priority (1)** as it is less time-sensitive.

- **Delays (`vTaskDelay`):** Used to yield CPU time and allow other tasks to run.

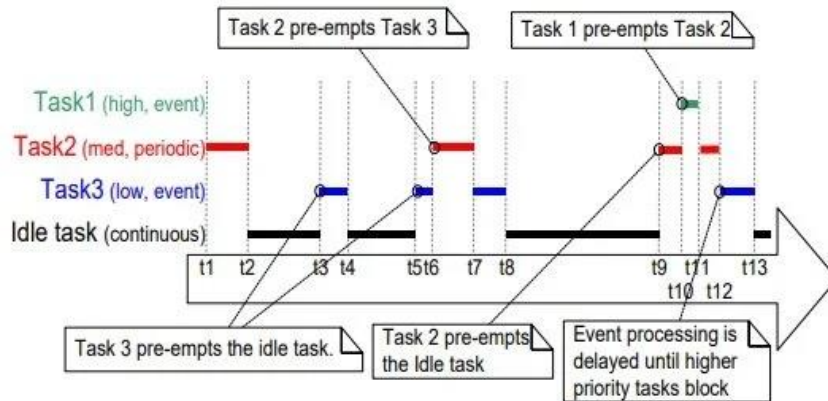


Figure 5.1 : Task priority levels in FreeRTOS illustrating that tasks with higher priority preempt those with lower priority to ensure real-time responsiveness.

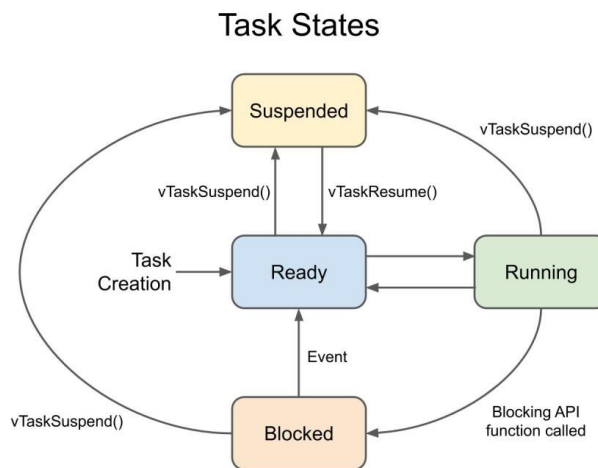


Figure 5.2 : Different task states in FreeRTOS showing how tasks transition between Running, Ready, Blocked, and Suspended states during scheduling.

5.2.1 CAN Receive Task

The **CAN Receive Task** plays a vital role in processing incoming messages from the Controller Area Network (CAN) bus, which is the primary communication medium in automotive systems. This task is assigned a **high priority** to guarantee minimal latency in message handling, as delayed processing could lead to outdated or missed data affecting system behavior.

The task typically waits on a notification or queue signaling the arrival of new CAN frames. Upon reception, it performs parsing and validation of the message contents. Critical data is then forwarded to other system modules or stored in shared memory buffers for subsequent processing.

To ensure thread-safe access to shared resources, synchronization mechanisms such as **mutexes** or **binary semaphores** are employed. This prevents race conditions and data corruption when multiple tasks access the CAN buffers concurrently.

Moreover, the CAN Receive Task is responsible for error detection and reporting, monitoring the CAN controller status for faults such as message errors or bus off conditions. Prompt detection allows the system to take corrective actions, enhancing reliability.

By assigning this task a high priority and implementing robust synchronization, the system ensures real-time responsiveness and data integrity in CAN communication.

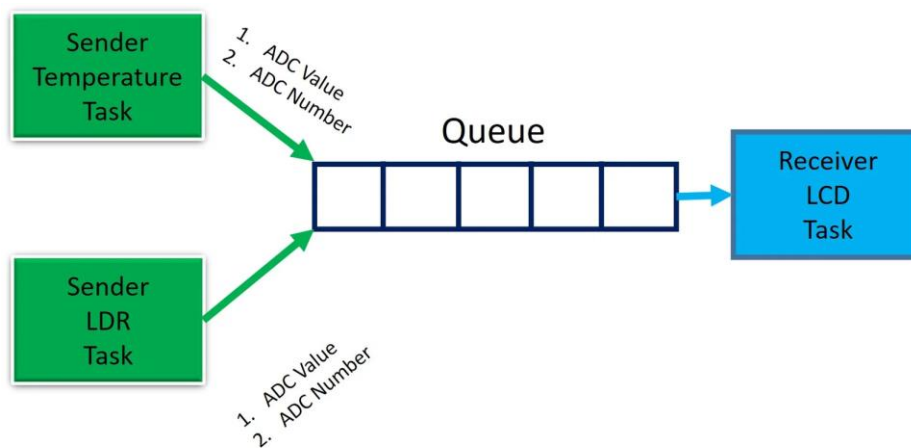


Figure 5.3 : Detailed flowchart illustrating the CAN Receive Task operations in FreeRTOS, including message reception, task notification, parsing, and synchronized access to shared buffers to ensure data integrity.

5.2.2 CAN Transmit Task

The **CAN Transmit Task** is responsible for managing the transmission of messages onto the CAN bus, ensuring that outgoing data packets are sent in a timely and orderly

manner. This task typically runs at a **medium priority**, balancing the need for prompt message delivery with the demands of higher-priority real-time tasks.

The task monitors transmission queues where other system components place messages that need to be sent. It interfaces with the CAN controller hardware via a hardware abstraction layer (HAL), submitting messages to the controller for transmission over the bus.

To maintain data integrity and synchronization with other tasks, the CAN Transmit Task uses inter-task communication mechanisms such as **message queues** or **direct task notifications**. These methods allow safe and efficient message passing between tasks without race conditions.

Error detection and retry mechanisms are integral to this task, ensuring that failed transmissions due to bus errors or collisions are handled appropriately to maintain communication reliability.

By operating at a medium priority and using robust communication protocols, the CAN Transmit Task contributes to the overall real-time performance and reliability of the embedded system.

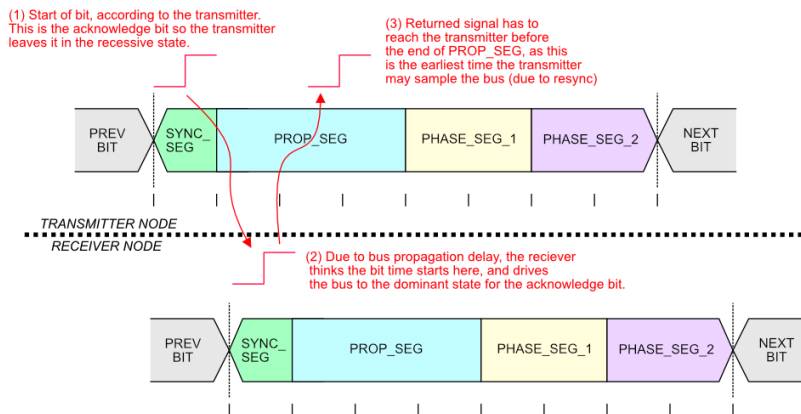


Figure 5.4 : Flowchart illustrating the CAN Transmit Task operation within an RTOS environment. The diagram shows message queuing, interaction with CAN controller hardware, and transmission over the CAN bus, highlighting synchronization mechanisms for safe data handling.

5.2.3 Control Task (Motor Safety)

The **Control Task**, often referred to as the **Motor Safety Task**, is a critical component responsible for continuously monitoring the motor operation parameters and ensuring safe functioning within predefined limits. Given its safety-critical role, this task is typically assigned the **highest priority** to guarantee immediate response to fault conditions.

This task monitors inputs such as motor speed, temperature, and current, processing sensor data to detect anomalies or dangerous conditions. Upon detecting any fault, the task executes predefined safety protocols, which may include shutting down the motor, activating alarms, or triggering fail-safe mechanisms.

Synchronization with other tasks is crucial, especially when accessing shared resources like motor control registers or safety flags. The task uses synchronization primitives such as mutexes and semaphores to maintain data integrity and prevent race conditions.

By prioritizing this task and implementing robust monitoring and control logic, the system achieves high reliability and safety, essential for automotive applications.

5.2.4 Diagnostics Task (UDS)

The **Diagnostics Task** is responsible for implementing the Unified Diagnostic Services (UDS) protocol, which enables communication between the vehicle's Electronic Control Units (ECUs) and diagnostic tools. This task facilitates reading diagnostic trouble codes (DTCs), monitoring system health, and performing routine diagnostics essential for vehicle maintenance and troubleshooting.

5.2.4.1 Main Functions

- **UDS Request Processing:** The task receives diagnostic requests sent over the CAN bus and processes them according to the ISO 14229 standard.
 - **DTC Reading and Clearing:** It allows retrieval of stored diagnostic trouble codes and clearing them upon receiving the appropriate commands.
 - **Diagnostic Session Management:** Manages different diagnostic sessions (default, extended, programming) to control access permissions to various ECU
-

functions.

- **Security Access:** Implements security protocols to prevent unauthorized diagnostic operations.
- **Inter-task Communication:** Coordinates with other tasks such as the CAN Receive Task for message handling and the Control Task for system status updates.

5.2.4.2 Task Design

1. Assigned a medium to high priority to ensure timely processing of diagnostic messages without interfering with critical control functions.
2. Utilizes queues and notifications for efficient message passing and event handling.
3. Designed to be event-driven, enabling prompt response to diagnostic requests while minimizing resource usage.

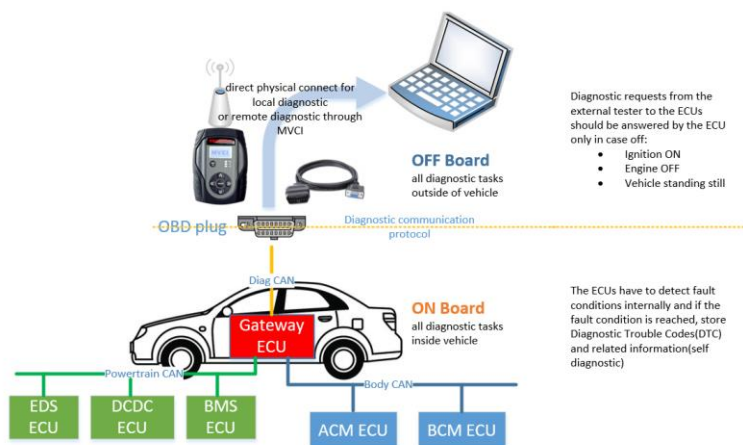


Figure 5.5 : Diagram illustrating the distinction between Onboard and Outboard components in a vehicle. Onboard refers to the electronic control units and sensors installed within the vehicle's internal systems, while Outboard denotes external modules or devices mounted outside the vehicle body for additional functionalities.

5.2.5 OTA Update Task

1. Overview

The OTA (Over-The-Air) Update Task is responsible for managing firmware updates remotely, allowing the system to receive and apply software patches without the need for physical access to the vehicle's Electronic Control Units (ECUs). This functionality enhances maintainability, reduces downtime, and ensures that the system remains secure and up-to-date with the latest features and bug fixes.

2. Primary Functions

2.1 Update Reception: Listens for incoming update packages transmitted over communication interfaces such as CAN, Bluetooth, or Wi-Fi.

2.2 Validation and Security: Verifies the integrity and authenticity of the received update using cryptographic checksums and digital signatures to prevent malicious code injection.

2.3 Storage Management: Handles writing the update package safely to the non-volatile memory (e.g., external flash or internal MCU flash), ensuring data consistency even in case of power loss.

2.4 Update Execution: Coordinates the transition to the new firmware by triggering system resets or bootloader activation sequences as required.

2.5 Error Handling and Rollback: Implements mechanisms to detect failed updates and revert to the previous stable firmware version to maintain system reliability.

3. Robustness and Fault Tolerance

To prevent system failures during the update, the OTA Update Task incorporates robust fault tolerance strategies. These include watchdog timers to recover from unexpected freezes, dual-bank memory schemes allowing rollback to the previous firmware if the update is corrupted, and transactional write operations to avoid partial or inconsistent data writes.

4. Security Considerations

Security is paramount in OTA updates to prevent unauthorized access or malicious tampering. The task enforces secure boot and cryptographic verification of update packages, employing algorithms such as SHA-256 for checksums and RSA/ECC for digital signatures. Additionally, secure communication channels (e.g., TLS over Wi-Fi or encrypted CAN messages) are

used to protect data in transit.

5. Resource Management

Since OTA updates involve handling large data packets and memory writes, the task optimizes CPU and memory usage by employing buffer management techniques and prioritizing real-time tasks to avoid latency spikes. The task also interacts with the file system or flash memory drivers to ensure efficient storage operations.

6. Integration with System Bootloader

The OTA Update Task closely integrates with the bootloader component, which is responsible for switching execution between firmware versions. After successfully storing the new firmware, the task signals the bootloader to validate and activate the update during the next system restart, ensuring a seamless transition.

7. Task Priority and Communication

In the FreeRTOS environment, the OTA Update Task is assigned a medium priority level, balancing the need for timely update processing with critical real-time tasks such as motor control and diagnostics. Inter-task communication mechanisms such as queues and notifications are used to synchronize the OTA process with other system components.

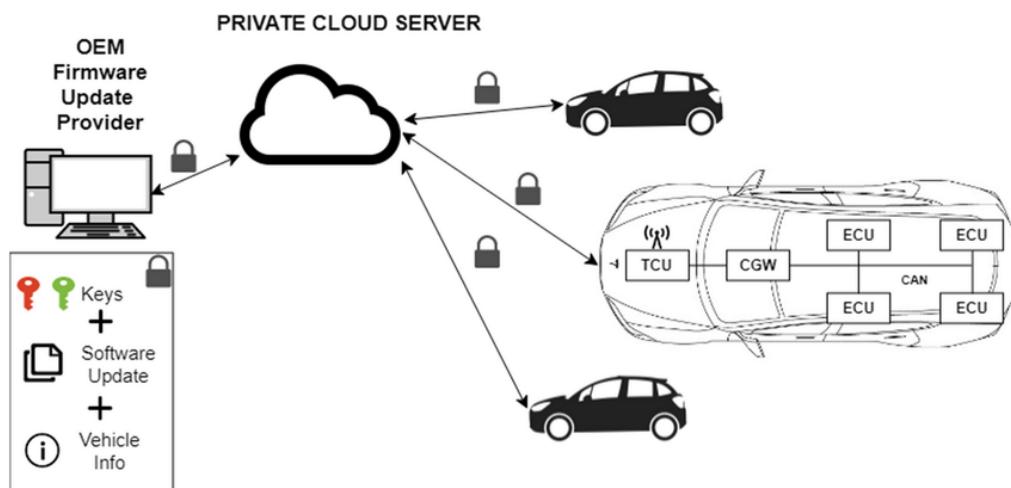


Figure 5.6 : Diagram illustrating the Over-The-Air (OTA) update process in automotive embedded systems. The diagram highlights key stages including update reception,

validation, storage, activation, and error handling to ensure reliable and secure firmware updates.

5.3 Inter-Task Communication (Queues, Notifications)

Efficient inter-task communication is critical in real-time embedded systems to ensure synchronized operation and data exchange among multiple concurrent tasks. In this project, FreeRTOS provides two primary mechanisms for inter-task communication: queues and task notifications, each optimized for different scenarios.

5.3.1 Queues

Queues in FreeRTOS are FIFO (First-In-First-Out) buffers that allow tasks or ISRs (Interrupt Service Routines) to send and receive messages or data safely. They enable asynchronous communication and data buffering, which is essential when the producer and consumer tasks operate at different rates.

- **Usage in This Project:**
The CAN Receive Task uses queues to forward incoming CAN messages to higher-level tasks such as the Diagnostics Task and Control Task. Similarly, the OTA Update Task may receive update commands or data packets through queues.
- **Advantages:**
Queues support sending variable-length data and complex data structures. They also provide built-in thread safety, preventing data corruption during concurrent access.
- **Implementation Details:**
Queues are created with fixed sizes based on expected message volume. Blocking timeouts are configured to balance responsiveness and CPU utilization.

5.3.2 Task Notifications

Task notifications offer a lightweight, low-overhead method for signaling events between tasks. Unlike queues, notifications typically send a single 32-bit value, which can be used as a binary flag, counting semaphore, or message.

- **Usage in This Project:**
Notifications are used to signal task events such as the completion of CAN message transmission, readiness of new diagnostic data, or completion of motor control operations.
- **Advantages:**
Task notifications are faster and consume less RAM compared to queues, making them suitable for high-frequency signaling.
- **Implementation Details:**
Notifications are implemented with specific bits assigned to different events, allowing tasks to wait for multiple event types simultaneously.

5.3.3 Synchronization and Data Integrity

To avoid race conditions and ensure data integrity during shared resource access, synchronization primitives such as mutexes and semaphores are used alongside communication mechanisms.

1. **Mutexes:**
Used to protect shared resources like global variables or hardware interfaces that multiple tasks might access concurrently.
2. **Binary Semaphores:**
Used for signaling between tasks and ISRs, often to unblock a task waiting for an event triggered by an interrupt.

5.3.4 Design Considerations

The communication architecture is designed to optimize system responsiveness and reliability:

1. Task priorities are assigned to ensure time-critical tasks receive CPU time promptly.

2. Queues handle larger, less frequent data transfers, while notifications handle small, frequent signals.
3. Timeouts and blocking periods are configured to prevent task starvation and deadlocks.
4. Communication paths are minimized to reduce latency and complexity.

CHAPTER 6

UNIFIED DIAGNOSTIC

SERVICES (UDS) ON

CAN

6 Chapter (6):Unified Diagnostic Services (UDS) On Can

6.1 – UDS STANDARD (ISO 14229)

The Unified Diagnostic Services (UDS) protocol, standardized under ISO 14229, is a diagnostic communication protocol used in the automotive industry to enable communication between diagnostic testers (external tools) and Electronic Control Units (ECUs) in a vehicle. It is designed to support the diagnostics of complex, distributed automotive electronic systems and is a successor to older protocols like KWP2000 and ISO 9141.

UDS operates over multiple physical layers, but in modern automotive applications, it is most commonly implemented over the Controller Area Network (CAN) using ISO 15765-2 for transport and network layer services. The UDS standard defines a set of diagnostic services, each identified by a unique Service Identifier (SID), which enables access to ECU functionality such as reading data, controlling sessions, accessing memory, downloading software, managing fault codes (DTCs), and more.

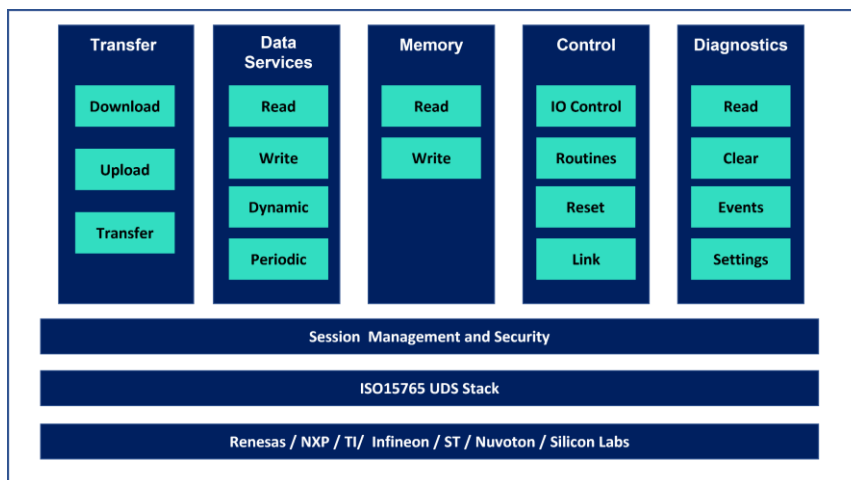


Figure 6.1 : ISO-TP (ISO 15765-2) Transport Layer used for UDS over CAN

This figure illustrates the role of ISO-TP in transporting UDS diagnostic messages over the CAN bus using segmentation and flow control mechanisms.

Each UDS message includes a Service ID (SID) followed by optional parameters such as Data Identifiers (DID), sub-functions, or security seeds/keys. The protocol supports both request/response communication and multi-frame transmission for large data payloads.

A typical UDS implementation follows a layered architecture as per AUTOSAR, with the Diagnostic Communication Manager (DCM) handling UDS services on top of the PDU Router and CAN stack. This abstraction enables flexible software updates, modularity, and separation of concerns between application and communication layers.

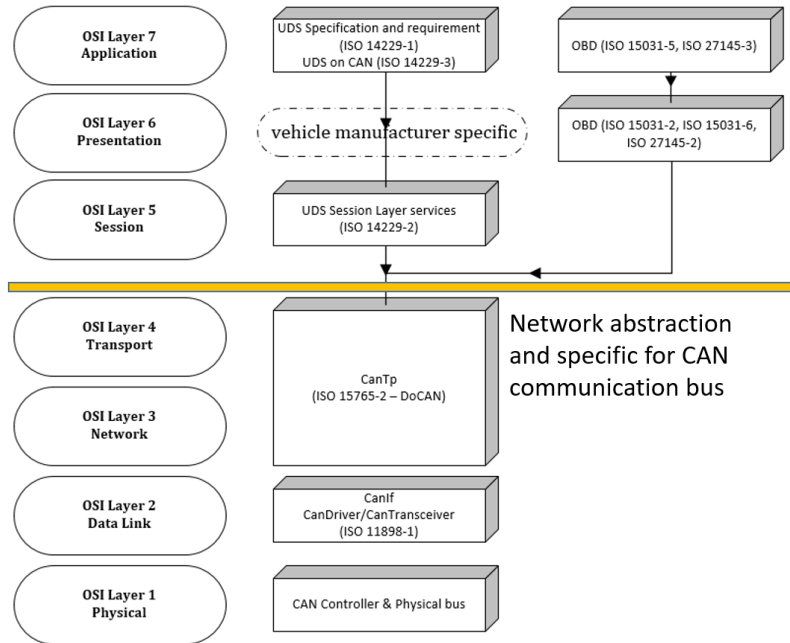


Figure 6.2 : UDS Protocol Stack as per ISO 14229 layered architecture

This diagram shows how the UDS standard is layered on top of ISO 15765-2 (ISO-TP), with each protocol level (Session, Transport, Physical) defined separately according to the ISO 14229 family.

6.2 – DIAGNOSTIC SESSION CONTROL (0X10)

The Diagnostic Session Control (DSC) service, identified by the Service ID 0x10, is a fundamental UDS service defined in ISO 14229-1, responsible for switching the ECU between different diagnostic sessions. Each session provides a specific set of functionalities and access rights, allowing diagnostic tools to perform operations such as reading memory, downloading software, or accessing security-protected data.

UDS defines several standard sessions, including:

Default Session (0x01):

The ECU's normal operation mode, with limited diagnostic access.

Programming Session (0x02):

Enables memory reprogramming or software download operations.

Extended Diagnostic Session (0x03):

Grants access to advanced diagnostic functions such as reading DTCs, I/O control, and advanced monitoring.

OEM-Defined Sessions:

Custom sessions may be implemented by OEMs for proprietary features, using reserved session IDs (e.g., 0x04 – 0x7F).

When a tester requests a session change, it sends a request message formatted as:

css

[0x10] [SessionType]

For example, a request 10 03 asks the ECU to enter the Extended Diagnostic Session. The ECU replies with a Positive Response (0x50) followed by the session type, such as:

50 03

This indicates that the ECU successfully entered the requested session. If the request is rejected, a Negative Response (0x7F) is returned along with the reason (NRC – Negative Response Code), such as 0x12 for "Sub-function not supported".

In addition to changing the session, the ECU may also send a response pending (0x78) if it needs time to process the transition before responding.

Session transitions also affect P2 / P2 timing parameters*, which define the maximum allowed response delay from the ECU. These timing constraints are crucial in time-sensitive communication and are often calibrated during ECU development.

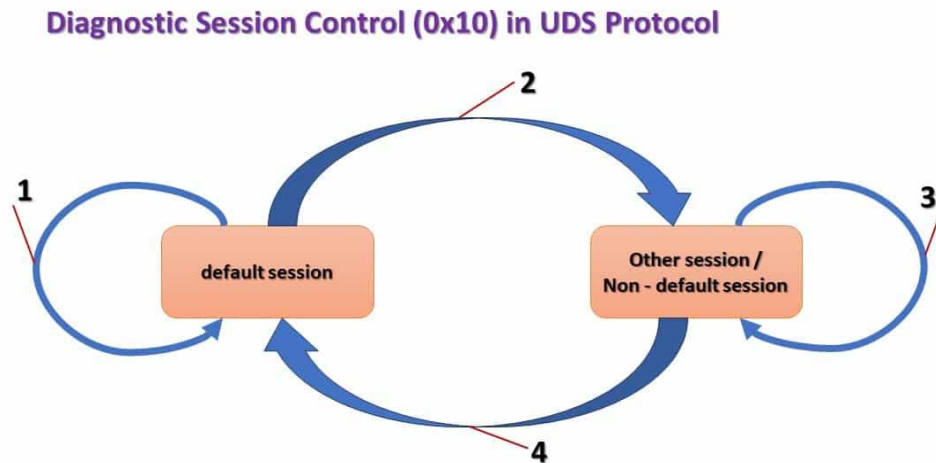


Figure 6.3 : Typical UDS Diagnostic Session Request/Response Exchange
This figure illustrates a basic request 10 03 and positive response 50 03, along with potential negative response codes (e.g., 7F 10 12) returned in case of failure.

6.3 – SECURITY ACCESS (0X27)

The Security Access service, defined by the Service ID 0x27, is one of the most critical diagnostic services in the UDS protocol. It is used to protect access to sensitive ECU operations, such as reprogramming, reading/writing protected memory, or executing advanced diagnostic functions. The mechanism ensures that only authorized testers can perform these operations.

The service follows a challenge-response authentication scheme, where the ECU issues a random "seed", and the tester must respond with the correct "key" calculated using a manufacturer-specific algorithm.

Security Access Levels

UDS allows for multiple levels of security access, typically defined as:

Level 1 (0x01/0x02):

Unlocks access to basic functions like clearing DTCs or extended diagnostics.

Level 3 (0x03/0x04):

Grants access to advanced functions, often related to programming or memory operations.

Levels 5 and beyond (0x05/0x06 ...):

Reserved for OEM-specific operations or high-level protection.

Each level has a requestSeed (odd sub-function) and a corresponding sendKey (even sub-function):

Tester sends 27 01 to request the seed for Level 1

ECU responds with: 67 01 [seed bytes]

Tester calculates the key and sends: 27 02 [key bytes]

ECU responds with: 67 02 (success) or 7F 27 35 (invalid key)

Security Timing & Lockout

To prevent brute-force attacks, ECUs implement security delay timers. After a certain number of failed attempts (e.g., 3 incorrect keys), the ECU enters a lockout state and rejects further access attempts for a predefined time (e.g., 10 minutes).

This delay is handled using NRC 0x36 (Request Sequence Error) or 0x37 (Required Time Delay Not Expired), and the ECU may reject both seed and key requests during this time.

Typical Message Flow

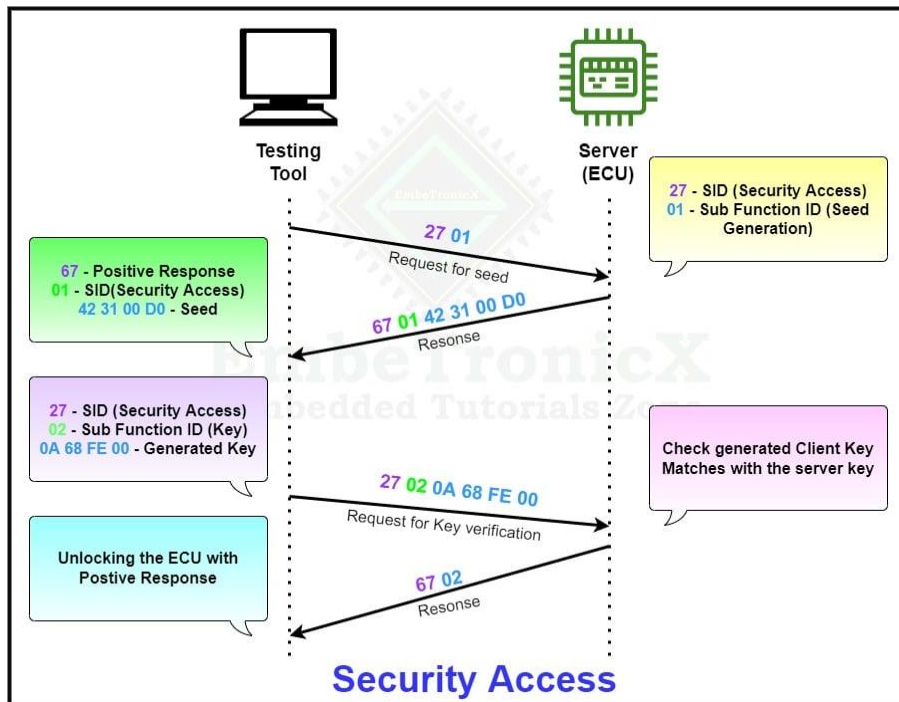


Figure 6.4 : shows the full Security Access sequence including seed request, key generation, and positive/negative responses.

6.4 – READ DATA BY IDENTIFIER (0X22)

The Read Data By Identifier (RDBI) service, defined by the Service ID 0x22, is used to request and retrieve specific internal data from an ECU. This service is commonly used by diagnostic testers to access real-time values, configuration parameters, sensor data, version information, VIN numbers, and other critical vehicle-related data stored inside the ECU.

Each piece of data that can be read via this service is associated with a Data Identifier (DID) — a unique 2-byte hexadecimal code (e.g., F190 for VIN, F187 for ECU serial number).

Request Format

The request message from the tester is formatted as:

[0x22] [DID high byte] [DID low byte]

For example

22 F1 90

This message instructs the ECU to return the Vehicle Identification Number (VIN).

Response Format

If the DID is supported and accessible, the ECU responds with:

62 [DID high] [DID low] [Data bytes]

For example:

62 F1 90 56 46 41 34 58 34 37 39 53 32 49 4E 33

This response contains the ASCII-encoded VIN: VFA4X479S2IN3

Negative Response

If the requested DID is not supported or access is denied (e.g., due to session level or security lock), the ECU responds with a Negative Response Code (NRC) such as:

0x13: Invalid Format

0x31: Request Out of Range

0x33: Security Access Denied

Multi-DID Request

UDS also supports reading multiple DIDs in a single request, such as:

22 F1 90 F1 87

The ECU responds with both data blocks, each prefixed by its DID.

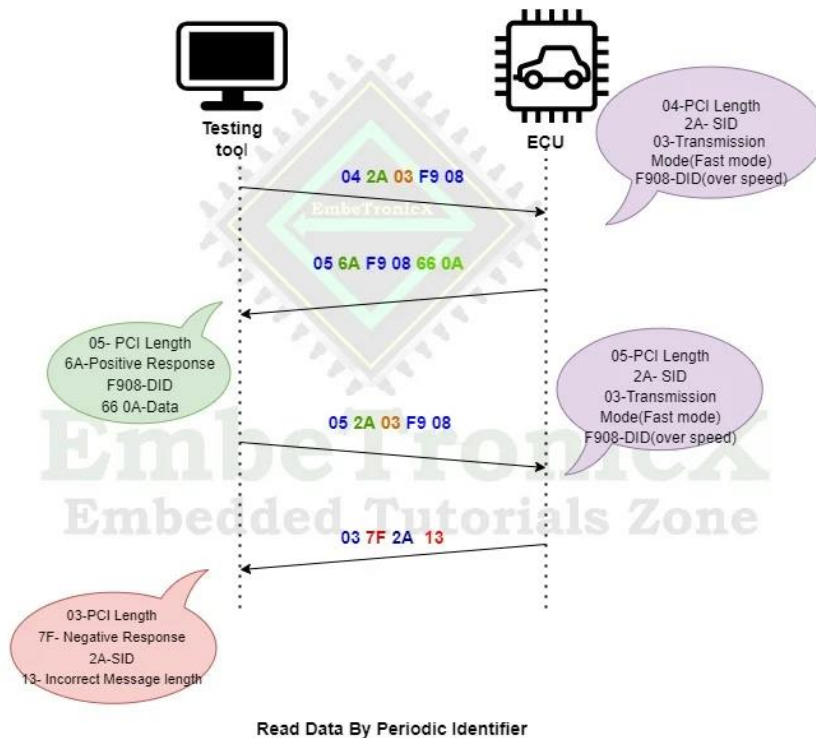


Figure 6.5 : Read Data By Identifier (0x22) Request/Response Exchange

This diagram shows the request for a single DID (e.g., 22 F190) and the corresponding positive response containing the requested data, as well as examples of negative responses.

6.5 – READ & CLEAR DTC (0X19 / 0X14)

One of the primary purposes of UDS is fault diagnostics, and for that, the Diagnostic Trouble Code (DTC) services are essential. These services allow a tester to read stored fault codes from the ECU and clear them once the issue has been resolved.

UDS defines two key services for this:

0x19 – ReadDTCInformation

0x14 – ClearDiagnosticInformation

These services are standardized in ISO 14229-1 and implemented in most modern automotive ECUs.

6.5.1 – Reading DTCs (Service 0x19)

The ReadDTCInformation service allows a tester to retrieve information about faults that have been detected and stored by the ECU. It supports multiple sub-functions, the most commonly used being:

Example – Request:

19 02 FF

This asks the ECU to return all DTCs with any active status bit (since FF is the full status mask).

Example – Positive Response:

59 02 01 23 45 00 02

59 02: Positive response for sub-function 0x02

01 23 45: DTC code (24-bit)

00: Status byte

02: Number of occurrences

If no DTCs are stored, the ECU may return an empty response or a negative response with NRC 0x11 (Service Not Supported).

6.5.2 – Clearing DTCs (Service 0x14)

Once a detected issue has been repaired, the tester can request the ECU to clear the stored DTC memory using service 0x14.

Example – Request:

14 FF FF FF

This requests clearing all DTCs, where FF FF FF is a wildcard for all groups.

Example – Positive Response:

54

A simple one-byte acknowledgment indicating successful erasure of the fault memory.

Note: Some ECUs may require security access before clearing DTCs. If access is denied, the ECU responds with:

7F 14 33

(NRC 0x33: Security access denied)

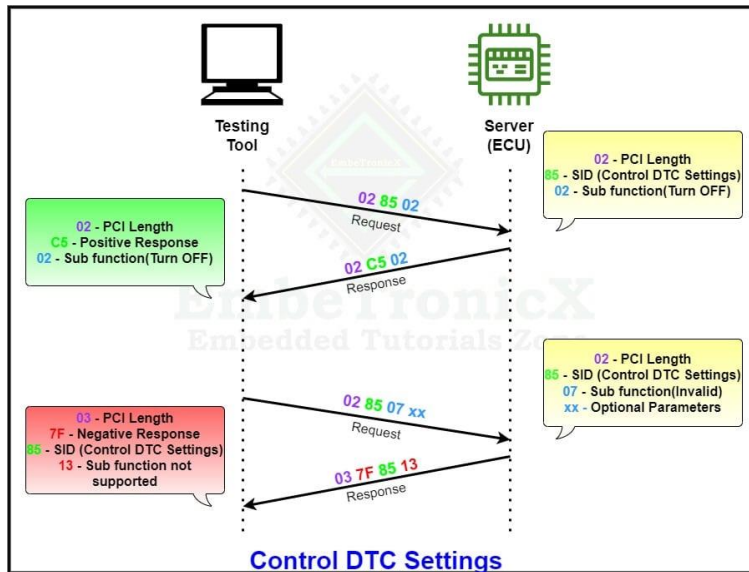


Figure 6.6 : UDS DTC Read and Clear Sequence

This diagram illustrates a typical sequence where the tester sends 19 02 FF to read DTCs and later sends 14 FF FF FF to clear them, with expected positive and negative responses.

6.6 – REQUEST DOWNLOAD (0X34)

The Request Download service, identified by Service ID 0x34, is part of the Unified Diagnostic Services (UDS) protocol defined in ISO 14229-1. It is used by a diagnostic tester to initiate the download of new software, firmware, or calibration data to the ECU.

This service is typically used in reprogramming scenarios and forms the first step in the data transfer sequence, followed by Transfer Data and Request Transfer Exit.

Request Structure

A typical request from the tester includes the following elements:

A data format identifier, indicating whether compression or encryption is used.

The memory address in the ECU where the data will be written.

The size of the data to be downloaded.

For instance, a tester may send a request indicating uncompressed, unencrypted data (DataFormatIdentifier = 0x00) to be written to memory address 0x00100000 with a total size of 32 bytes. The memory address and length are encoded using a format specifier that defines how many bytes represent each value.

Upon receiving the request, the ECU validates:

The diagnostic session (typically it must be in Programming Session),

The security access level,

The address range and memory permissions.

If the request is valid, the ECU responds with a positive acknowledgment, including the maximum number of bytes it can accept per block during the actual transfer phase.

If the request is invalid — for example, if the memory range is not permitted or the session is incorrect — the ECU returns a negative response indicating the reason, such as “conditions not correct” or “security access denied”.

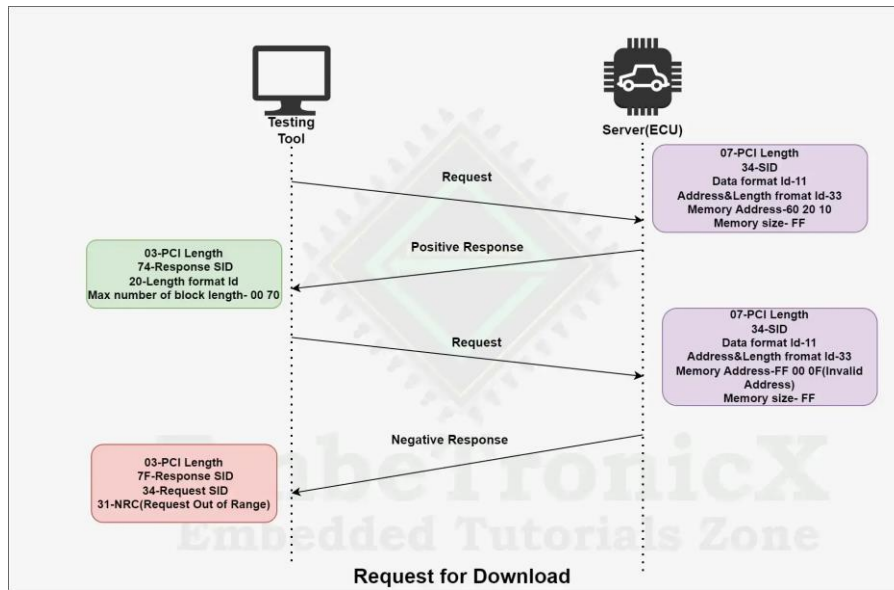


Figure 6.7 : UDS Request Download (0x34) Service Flow

This diagram illustrates the sequence of messages exchanged when initiating a data download to the ECU, including both successful and rejected cases depending on session or access rights.

6.7 – DTC STORAGE & MANAGEMENT IN FLASH

(6.7.1) Introduction to DTC Storage

In modern automotive systems, Diagnostic Trouble Codes (DTCs) are essential for monitoring vehicle health and supporting after-sales diagnostics. When an Electronic Control Unit (ECU) detects a fault in one of its monitored functions, it generates a corresponding DTC and stores it in non-volatile memory, typically Flash memory. This ensures that fault information is retained even after the vehicle is powered off, allowing service technicians to retrieve it later during inspection or repair procedures.

(6.7.2) Structure of Stored Diagnostic Data

The storage of DTCs involves more than just the fault code itself. ECUs typically store associated metadata such as:

The current fault status (e.g., active, confirmed, or intermittent),

The number of occurrences of the fault,

And contextual data recorded at the time of the fault.

This may include environmental and operational parameters like engine temperature, speed, or throttle position. In more advanced systems, timestamps and additional event-specific diagnostic data are also stored.

(6.7.3) Role of Flash Memory

Flash memory is well suited for storing DTCs because of its non-volatile nature, meaning it can retain data without continuous power. However, flash has limited write-erase cycles, which requires the use of memory management techniques. To preserve memory integrity, ECUs often delay writing fault data to flash until the fault is confirmed through repeated detection over a defined threshold.

(6.7.4) Memory Segmentation and Management

Diagnostic memory is typically divided into dedicated segments, such as:

Primary DTC storage area for active and historical faults,

Freeze frame memory that captures system state at the time of fault detection,

And extended data sections for storing aging counters or other supporting diagnostic metrics.

When a fault is cleared by the user or tool, the ECU either marks the DTC as inactive or erases its memory block, depending on the implementation. Write minimization strategies such as data buffering or deferred writing are employed to extend flash memory life and ensure efficient memory use.

(6.7.5) Data Integrity and Security Considerations

To ensure robustness, ECUs incorporate mechanisms such as checksums, cyclic redundancy checks (CRC), or redundant storage areas to avoid corruption of diagnostic data in case of unexpected power loss or hardware failure. Additionally, in systems where safety or security is critical, access to DTC memory may be restricted based on diagnostic session levels or security authentication.

(6.7.6) AUTOSAR-Based DTC Management

In AUTOSAR-compliant systems, DTC storage and management are handled by a dedicated module called the Diagnostic Event Manager (DEM). This module standardizes how faults are detected, recorded, updated, and erased across all ECUs in the vehicle. It also ensures that memory layout, status bit management, and fault tracking follow a consistent and traceable format, improving interoperability between components from different suppliers.

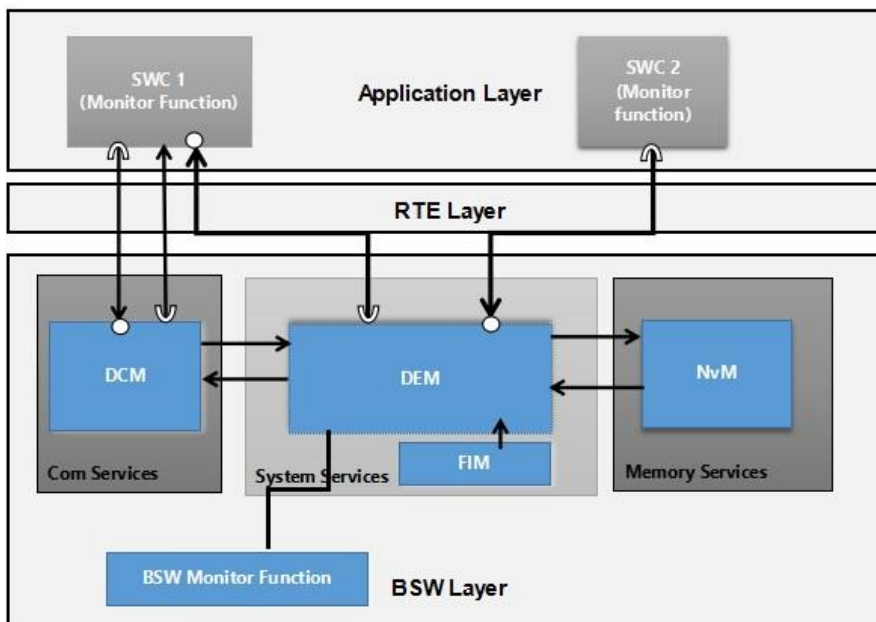


Figure 6.8 : DTC Storage Structure in Flash Memory (AUTOSAR-based)

This diagram illustrates how fault codes, freeze frame data, and extended diagnostic information are organized within the non-volatile memory of an ECU, ensuring structured and secure access to diagnostic records.

6.8 – SUMMARY & SERVICES USED

(6.8.1) Summary

This chapter provided a detailed overview of the Unified Diagnostic Services (UDS) protocol, as standardized in ISO 14229, and its practical relevance in automotive embedded systems. We discussed key services such as session control, access authorization, data reading, fault diagnostics, and memory reprogramming.

Each service was explored not only in terms of its theoretical structure, but also through its role within our system architecture and diagnostic design. Particular emphasis was placed on how these services enable structured, secure communication between the external diagnostic tester and the vehicle's ECUs.

In addition, we examined how DTCs are stored and managed in non-volatile flash memory, including their format, memory segmentation, and integration within AUTOSAR-compliant architectures. These mechanisms ensure that fault data is preserved reliably, traceable, and securely accessible—contributing to the overall robustness and maintainability of the vehicle's electronic system.

Understanding and correctly implementing UDS services is a fundamental requirement in modern automotive software development, especially for safety- and reliability-critical functions such as those addressed in our graduation project.

(6.8.2) Diagnostic Services Used in the Project

The following table summarizes the UDS services that were implemented or referenced in the development of our diagnostic and flashing system:

(6.8.2) Diagnostic Services Used in the Project

The following table summarizes the UDS services that were implemented or referenced in the development of our diagnostic and flashing system:

CHAPTER 7

BOOTLOADER & OTA FIRMWARE UPDATE

7 Chapter (7):Bootloader & OTA Firmware Update

7.1 FLASH MEMORY LAYOUT & PARTITIONING

In embedded systems that support over-the-air (OTA) firmware updates, careful planning of the microcontroller's flash memory layout is essential to ensure safe, reliable, and reversible firmware updates. In this project, the STM32F103 microcontroller is used to host both a custom bootloader and the application firmware, with a dedicated memory region for metadata that facilitates versioning, validation, and rollback.

The flash memory is divided into the following sections:

Bootloader Region

Start Address: 0x08000000

Size: Typically 16KB (can vary depending on features)

Purpose: Contains the bootloader code responsible for receiving firmware chunks over CAN, verifying metadata, managing flash erase/write operations, and transitioning control to the application.

Protection: This section is marked as read-only and is not overwritten during updates.

Metadata Region

Start Address: Just after the bootloader (e.g., 0x08004000)

Size: 1–2 KB

Purpose: Stores metadata such as:

Firmware version

Total size

CRC32 or SHA-256 hash

Validity flags

Update state (e.g., In Progress, Verified, Failed)

This section is updated by the bootloader during and after the update process.

Application Firmware Region

Start Address: e.g., 0x08005000

Size: Remaining flash (minus reserved areas)

Purpose: Stores the main application firmware. The bootloader jumps to this section after successful verification.

Note: This region is erased and rewritten during OTA updates.

(Optional) Backup Application Region

Start Address: Varies (if supported)

Purpose: May be used to store a previous working version of the application in case rollback is needed. Not implemented in this prototype but supported in future designs.

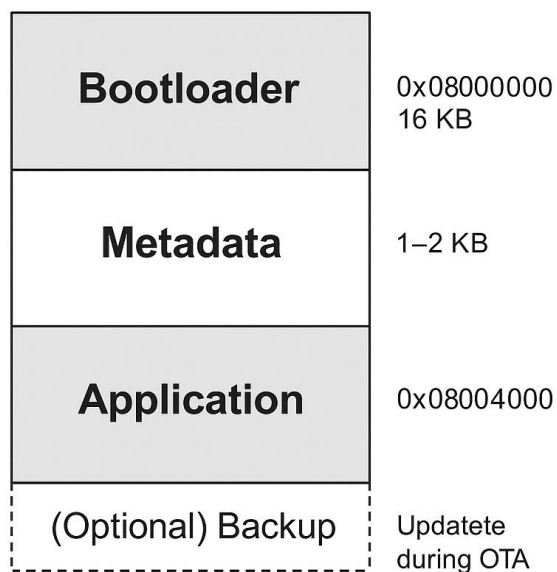


Figure 7.1 : Flash memory layout for OTA-enabled STM32 system. Memory is partitioned into bootloader, metadata, and application regions. The bootloader is protected from modification, while the application region is updated during OTA.

[Adapted based on STM32F103 flash organization]

Key Considerations:

The memory partitioning ensures isolation between the bootloader and the application.

The metadata region acts as a bridge for OTA logic, holding essential information for version control and verification.

In case of an incomplete update or failed verification, the bootloader can prevent execution of the faulty application and stay in recovery mode.

The vector table offset (VTOR) is configured in the bootloader to redirect execution to the application firmware.

7.2 BOOTLOADER DESIGN

The bootloader is a dedicated software module that executes immediately after power-up or system reset, residing in a protected region of the STM32 flash memory. In this project, the bootloader plays a pivotal role in managing OTA firmware updates securely and reliably over the CAN bus. It ensures that firmware upgrades are received, validated, written to memory, and handed off to the application in a controlled and fail-safe manner.

Unlike monolithic firmware designs, this bootloader is modular, event-driven, and fault-tolerant. Its tasks are clearly segmented into stages, each governed by decision logic, status flags, and integrity checks. This architecture allows the system to recover gracefully from interruptions, ensuring system robustness in field deployments.

The design and functionality of the bootloader are detailed across the following subsections.

7.2.1 METADATA & OPTION BYTES

A dedicated metadata structure is reserved in flash memory, acting as the communication bridge between the bootloader and application firmware. This metadata is essential for identifying the firmware's state, version, and validity. It typically contains the following fields:

Firmware Version: Enables the bootloader to detect downgrades or redundant updates.

Firmware Size (in bytes): Used to calculate the expected number of CAN packets.

Hash or CRC32 Checksum: Ensures data integrity by allowing the bootloader to verify the correctness of the written firmware.

Firmware State Flag: Indicates whether the image is `VALID`, `IN_PROGRESS`, `PENDING_VERIFICATION`, or `INVALID`.

Application Start Address: Guides the bootloader on where to jump after verification.

The STM32 also provides Option Bytes, a special configuration area in flash used to enforce boot-time behavior:

BOOT0 / BOOT1 Configuration: Determines whether the system boots from flash, system memory, or SRAM.

Read and Write Protection: Write protection is enabled on the bootloader region to prevent accidental overwrite.

RDP Level: (Readout Protection) restricts access to flash memory, enhancing security against unauthorized reads.

Together, metadata and option bytes enable a secure and predictable update process, even under harsh or unstable power conditions.

7.2.2 CAN-BASED CHUNK RECEPTION

Due to the 8-byte data payload limitation of standard CAN frames, the firmware binary must be fragmented into smaller chunks, which are individually transmitted and acknowledged. The reception process is as follows:

Initialization Packet: Contains total firmware size, version, and hash.

Data Chunk Packets: Each chunk includes:

Sequence number

Data payload

Optional checksum

Acknowledgment: The bootloader sends ACK or NACK after each chunk, enabling retransmission if needed.

Progress Tracking: The bootloader uses sequence numbers to ensure in-order reception and to detect missing or duplicated chunks.

This chunked reception model offers resilience against temporary disconnections and aligns with UDS-style updates using services such as 0x34 (Request Download) and 0x36 (Transfer Data) in future versions.

7.2.3 CHUNK ERASE & WRITE

Before a new firmware image can be written, the relevant flash sectors must be erased. STM32F1 devices require erasing by 1KB or 2KB pages depending on flash density. The bootloader implements the following logic:

Sector Tracking: Tracks which sectors have been erased to avoid redundant operations.

Partial Writes: Allows writing chunk-by-chunk instead of buffering the entire image in RAM.

Flash Programming: Each chunk is programmed directly into its flash offset using STM32 HAL functions.

Write Verification: Optionally reads back the written chunk to confirm integrity.

This incremental approach to flash writing reduces RAM usage and provides the ability to resume interrupted updates by comparing received chunks to existing data on reboot.

7.2.4 UPDATE VERIFICATION (RSA OR CRC)

After receiving all firmware chunks, the system performs post-write verification to ensure authenticity and integrity before execution:

CRC32 Verification: A fast, lightweight method to verify data correctness. The computed CRC of the written image is compared with the one from metadata.

SHA-256 Digest (Optional): Offers stronger integrity validation in case of tampering.

RSA Signature Check (Future Work): The final firmware hash can be signed using a private RSA key. The bootloader would then use a stored public key to verify the digital signature.

Final Status Update: If verification succeeds, the firmware is marked as **VALID**. Otherwise, it is flagged as **INVALID**, and the bootloader halts or enters recovery mode.

This layered verification approach balances simplicity (for prototyping) with extensibility (for production security).

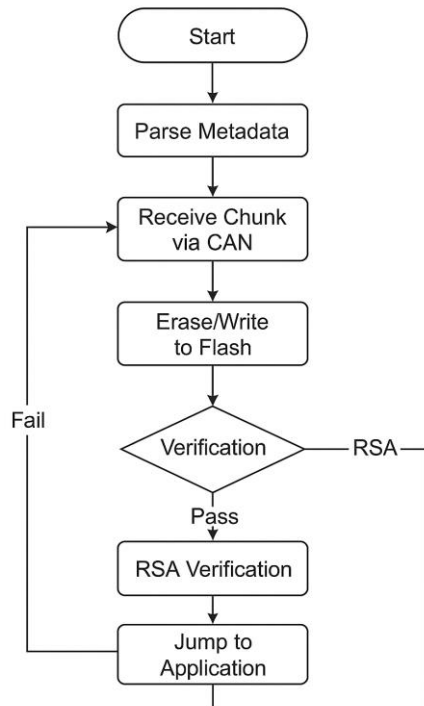


Figure 7.2 : Bootloader functional flow: from metadata parsing, chunk reception via CAN, erase/write to flash, RSA or CRC verification, and jump to the application firmware.

7.3 FALLBACK & FAIL-SAFE MECHANISMS

In the context of embedded systems that support remote firmware upgrades, robustness is not optional—it is a core requirement. The ability of the system to withstand interruptions, corruption, or unforeseen errors during firmware updates is critical for long-term reliability and field survivability. Incomplete updates can result in system instability, boot failures, or permanent malfunction, particularly in mission-critical or safety-sensitive domains such as automotive, medical, or industrial automation.

To mitigate these risks, the proposed bootloader architecture implements a comprehensive fallback and fail-safe mechanism that is both state-aware and integrity-driven. This mechanism ensures that no firmware image is executed unless its correctness is explicitly verified, and that the system can always recover to a known safe state even in the presence of faults.

7.3.1 PERSISTENT METADATA AND UPDATE STATE TRACKING

A key feature of the system's reliability lies in its use of persistent metadata stored in a reserved flash section. This metadata acts as a checkpoint system, tracking the current stage of the update process and storing information critical for recovery. Among the fields maintained in the metadata block are:

Firmware Version Number – Enables the system to prevent downgrades or repeated application of the same image.

Expected Firmware Size – Allows the bootloader to confirm that all chunks have been received.

Hash or CRC32 Value – Used to verify data integrity after the update.

Firmware Validity Flag – Encodes the state of the firmware (e.g., `VALID`, `IN_PROGRESS`, `PENDING_VERIFICATION`, `INVALID`, or `ROLLBACK_REQUIRED`).

This structured representation allows the bootloader to make deterministic decisions based on the most recent valid state, even after power cycles or resets. For example, if the flag is set to `IN_PROGRESS` and a power failure occurs, the bootloader will avoid executing the partially written image and instead re-enter update mode.

7.3.2 STRUCTURAL AND FUNCTIONAL APPLICATION VERIFICATION

To prevent undefined behavior or system crashes, the bootloader performs sanity checks on the application firmware before allowing execution. These checks include both structural and functional criteria, such as:

Ensuring the vector table contains a valid stack pointer and a valid reset handler located within the legal flash address space.

Validating that the memory addresses do not point to uninitialized or protected regions.

Optionally performing a partial or full hash calculation over the application binary and comparing it to the reference value in metadata.

Verifying that the `VALID` flag in the metadata is set before initiating any control transfer.

These measures prevent the bootloader from blindly executing code that may be incomplete, outdated, or maliciously modified, thereby enforcing boot-time authentication of the application firmware.

7.3.3 CONTROLLED RECOVERY AND SAFE STATE MANAGEMENT

If the update fails or verification does not succeed, the bootloader transitions into a controlled recovery mode. This is a known and safe state in which the system remains operational enough to receive new updates but refrains from launching the main application.

In recovery mode:

The CAN interface is re-initialized and left open to receive new firmware packets.

A predefined communication protocol or diagnostic message may be used to notify the central system or cloud server of the failure.

The system may provide visual or hardware indicators (e.g., blinking LED patterns) to signal that it is in recovery state.

Crucially, the bootloader does not remain idle in this mode. It continuously checks for incoming data and is capable of resuming the update from the last valid chunk, thanks to metadata tracking and chunk indexing logic.

7.3.4 WATCHDOG COORDINATION AND POWER FAILURE RESILIENCE

Embedded systems often integrate a hardware watchdog timer to prevent software lockups. During an OTA update, especially in flash erase/write stages, it is essential to coordinate with the watchdog to prevent unintentional resets. The bootloader is designed to:

Periodically refresh the watchdog timer during long operations.

Suspend flash operations safely if an imminent reset is detected.

Ensure that metadata is updated atomically, so that incomplete state transitions (e.g., from IN_PROGRESS to VALID) are avoided.

In the case of sudden power loss or hardware reset, the bootloader will reboot into a known state and re-evaluate the integrity of the firmware image. If the image is incomplete or the hash does not match, the system will remain in recovery mode until a valid update is applied.

7.3.5 FIRMWARE INTEGRITY AND SECURITY FAIL-SAFES

In addition to robustness, the system must protect against unauthorized or malicious firmware injections. Security-oriented fail-safe measures are implemented as follows:

Write-protection of the bootloader region via STM32 Option Bytes ensures that the critical update logic cannot be modified post-deployment.

Read-protection (RDP Level 1 or 2) prevents external devices from accessing flash content, thereby protecting intellectual property and system configuration.

Cryptographic Verification (planned in future iterations) will employ RSA or ECC to validate firmware authenticity, protecting the device from man-in-the-middle (MITM) and replay attacks.

Firmware signing keys are stored in the bootloader and never exposed to the application layer, ensuring separation of trust boundaries.

Together, these security mechanisms ensure that the system will not only survive failure scenarios, but will also actively reject any unauthorized or suspicious firmware, thereby upholding system trustworthiness.

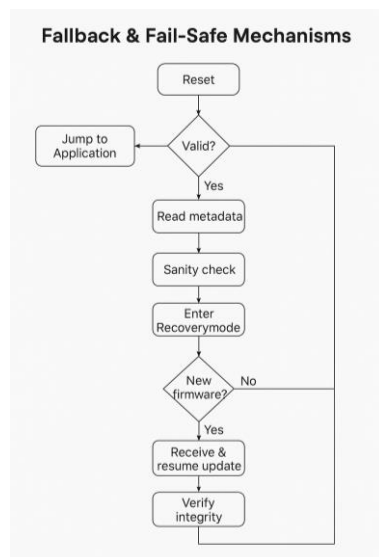


Figure 7.3 : Flowchart of fallback and fail-safe mechanism in an embedded OTA update system. The process begins with a system reset, followed by a series of conditional checks including firmware validity, metadata reading, and application integrity. If any step fails, the system enters recovery mode and awaits a verified firmware update before executing the main application.

7.4 BOOTLOADER ↔ APPLICATION HANDOFF

After a successful firmware update and integrity verification, the final step in the bootloader's execution is to hand over control to the main application firmware. This handoff process is critical because any misconfiguration in memory offsets, vector tables, or clock setup can lead to system instability or complete failure to execute the application.

To ensure a clean and reliable transition, the bootloader in this project implements a structured and deterministic handoff routine, guided by microcontroller startup architecture and ARM Cortex-M core specifications.

7.4.1 VECTOR TABLE REMAPPING

On ARM Cortex-M3 microcontrollers (like STM32F103), the vector table must be properly configured before jumping to the application. The vector table contains the initial stack pointer and the reset handler address, located at the beginning of the application image in flash.

The bootloader performs the following steps:

Read Stack Pointer & Reset Handler:

It reads the first two 32-bit words from the application's flash region (e.g., at address 0x08005000), which represent:

Initial MSP (Main Stack Pointer)

Reset Handler address

Validate Addresses:

The bootloader checks whether:

The MSP is within valid SRAM range (typically 0x20000000 – 0x2000FFFF)

The Reset Handler lies within the flash memory space

Set Vector Table Offset Register (VTOR):

The SCB->VTOR register is updated to point to the start of the application image, allowing all interrupts and exceptions to be handled correctly by the application firmware.

```
SCB->VTOR = APPLICATION_START_ADDRESS;
```

Update MSP and Jump:

Using inline assembly or CMSIS functions, the bootloader:

Sets the MSP using `__set_MSP()`

Performs a function pointer jump to the reset handler

Example:

```
void (*App_Reset_Handler)(void);
```

```
App_Reset_Handler = (void *) (*((volatile uint32_t*)(APP_START_ADDR + 4)));
```

```
__set_MSP(*((volatile uint32_t*) APP_START_ADDR));
```

```
App_Reset_Handler();
```

This sequence ensures that the bootloader cleanly exits and that the application starts execution exactly as it would on a cold boot.

7.4.2 CONDITIONS FOR HANDOFF

To prevent jumping to a corrupted or invalid image, the bootloader performs a series of preconditions before handoff:

Metadata status must be VALID

Hash verification must have passed

Vector table and stack pointer must be sane

No rollback or update pending flags must exist

Only if all conditions are satisfied does the handoff occur. Otherwise, the system remains in bootloader mode awaiting a new firmware update.

7.4.3 CLOCK AND PERIPHERAL DEINITIALIZATION

Before executing the application, the bootloader must reset or de-initialize any peripherals it used, such as:

System clocks (if different from default)

CAN peripheral

GPIOs or timers

This step prevents residual configurations from interfering with the application, which assumes a clean hardware state at startup.

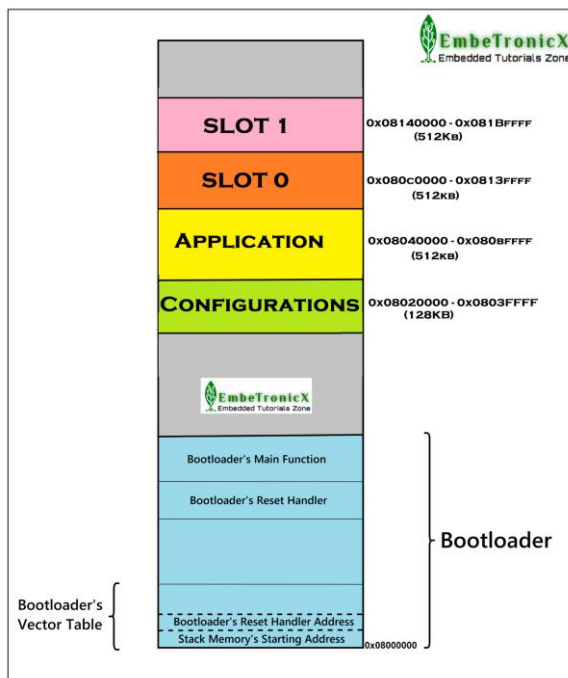


Figure 7.4 : Mechanism for vector table remapping and jump-to-application in STM32 bootloader. The bootloader updates SCB->VTOR to point to the application's vector table, sets the MSP (Main Stack Pointer), and then jumps to the application's reset handler, completing a secure and controlled handoff.

Source: EmbeTronicX STM32 Bootloader Tutorial

7.4.4 SECURE HANDOFF (FUTURE WORK)

In future extensions, a secure handoff mechanism may be added, such as:

Signature-based validation

Monotonic firmware version counters

State attestation to prevent rollback attacks

Such measures would further ensure that only authenticated and authorized firmware can be executed after handoff.

This handoff mechanism completes the bootloader's responsibilities and ensures that application control begins under verified and controlled conditions, preserving system integrity, reliability, and trust.

CHAPTER 8

CLOUD-BASED FIRMWARE SERVER

8 CHAPTER (8):Cloud-Based Firmware Server

8.1 SERVER ARCHITECTURE & REST API

REST is an acronym for Representational State Transfer and an architectural style for distributed hypermedia systems. Roy Fielding first presented it in 2000 in his famous . Since then, it has become one of the most widely used approaches for building web-based APIs (Application Programming Interfaces).

REST is not a protocol or a standard, it is an architectural style. During the development phase, API developers can implement REST in a variety of ways.

Like the other architectural styles, REST also has its guiding principles and constraints.

These principles must be satisfied if a service interface is to be referred to as RESTful.

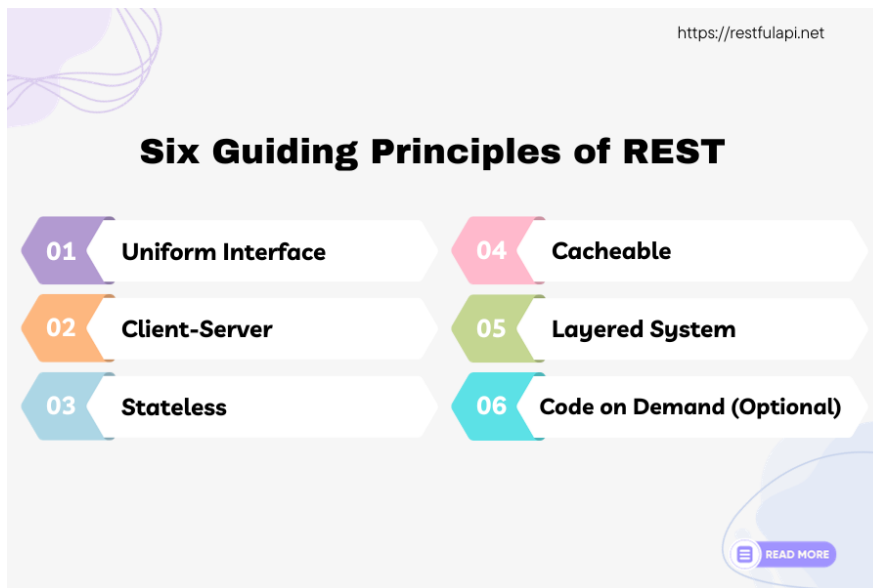


Figure 8.1: Architecture of the Cloud Firmware Server]

A Web API (or Web Service) conforming to the REST architectural style is called a REST API (or RESTful API).

1. The Six Guiding Principles of REST

REST is based on some constraints and principles that promote simplicity, scalability, and statelessness in the design. The six guiding principles or are:

8.2. UNIFORM INTERFACE

By applying the to the components interface, we can simplify the overall system architecture and improve the visibility of interactions. Multiple architectural constraints help in obtaining a uniform interface and guiding the behavior of components.

The following four constraints can achieve a uniform REST interface:

Identification of resources – The interface must uniquely identify each resource involved in the interaction between the client and the server.

Manipulation of resources through representations – The resources should have uniform representations in the server response. API consumers should use these representations to modify the resource state in the server.

Self-descriptive messages – Each resource representation should carry enough information to describe how to process the message. It should also provide information on the additional actions that the client can perform on the resource.

Hypermedia as the engine of application state – The client should have only the initial URI of the application. The client application should dynamically drive all other resources and interactions with the use of hyperlinks.

In simpler words, REST defines a consistent and uniform interface for interactions between clients and servers. For example, the HTTP-based REST APIs make use of the standard HTTP methods (GET, POST, PUT, DELETE, etc.) and the URIs (Uniform Resource Identifiers) to identify resources.

8.2.1. CLIENT-SERVER

The client-server design pattern enforces the separation of concerns, which helps the client and the server components evolve independently.

By separating the user interface concerns (client) from the data storage concerns (server), we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.

While the client and the server evolve, we have to make sure that the interface/contract between the client and the server does not break.

8.2.2. STATELESS

mandates that each request from the client to the server must contain all of the information necessary to understand and complete the request.

The server cannot take advantage of any previously stored context information on the server.

For this reason, the client application must entirely keep the session state.

8.2.3 CACHEABLE

The requires that a response should implicitly or explicitly label itself as cacheable or non-cacheable.

If the response is cacheable, the client application gets the right to reuse the response data later for equivalent requests and a specified period.

8.2.4 LAYERED SYSTEM

The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior. In a layered system, each component cannot see beyond the immediate layer they are interacting with.

A layman's example of a layered system is the MVC pattern. The MVC pattern allows for a clear separation of concerns, making it easier to develop, maintain, and scale the application.

8.2.5 CODE ON DEMAND (OPTIONAL)

REST also allows client functionality to be extended by downloading and executing code in the form of applets or scripts.

The downloaded code simplifies clients by reducing the number of features required to be pre-implemented. Servers can provide part of the features delivered to the client in the form of code, and the client only needs to execute the code.



Figure 8.2: JSON Schema Structure for Firmware Metadata

2. Server Architecture

refers to the design and structure of a system that defines how servers, clients, and other components communicate and function together. It's critical for scalability, performance, reliability, and maintainability of applications or services.

Core Components of Server Architecture

1. Client

The user-facing part of the system (e.g., web browsers, mobile apps).

Sends requests to the server, receives responses.

2. Server

The core that processes requests, runs the application, accesses databases, etc.

Can be:

Web Server (e.g., Nginx, Apache)

Application Server (e.g., Node.js, FastAPI, Django)

Database Server (e.g., PostgreSQL, MongoDB)

3. Database

Stores and retrieves data.

Examples: MySQL, PostgreSQL, MongoDB, Redis.

8.2 VERSION JSON SCHEMA & HOSTING

JSON Schema Purpose

A version JSON file informs client devices about the latest firmware, mandatory updates, and file URLs.

How does JSON Schema work?

When it comes to data exchange, JSON Schema stands out as a powerful standard for defining the structure and rules of JSON data. It uses a set of to define the properties of your data.

While JSON Schema provides the language, validating a JSON against a requires a JSON Schema. The JSON validator checks if the JSON documents conform to the schema.

JSON Schema Validators are tools that implement the JSON Schema specification. Such tooling enables easy integration of JSON Schema into projects of any size.

2 Hosting Options

GitHub Pages: For public firmware and JSON hosting.

Amazon S3 + CloudFront: Scalable, secure, and CDN-supported.

Firebase Hosting: Simple deployment, HTTPS support.

Custom API Server: Hosted on platforms like Render, Railway, or AWS EC2.

8.3 SECURE STORAGE (E.G. AWS S3, FIREBASE)

In a cloud-based firmware update system, secure storage is critical for hosting firmware files safely and efficiently. This storage acts as a trusted source where devices can download firmware binaries, ensuring data integrity, restricted access, and reliable availability.

Goals of Secure Storage:

Protect firmware files from unauthorized access or tampering.

Ensure only authenticated devices can download firmware.

Enable fast global delivery using CDNs (Content Delivery Networks).

Support monitoring of access logs and download activities.

Popular Secure Storage Options:

1. AWS S3 (Amazon Simple Storage Service)

Features:

Object-based storage.

Fine-grained access control using IAM and Bucket Policies.

Supports signed URLs for time-limited access.

Scalable and highly available.

2. Firebase Cloud Storage (by Google)

Features:

Easy integration with mobile/IoT apps.

Uses Firebase Authentication for access control.

Real-time monitoring via Firebase Console.

Free tier available for testing.

8.4 FIRMWARE DOWNLOAD & INTEGRITY CHECKS

Firmware integrity checks ensure that firmware hasn't been tampered with and is authentic. This is crucial for security and proper device functionality. Integrity checks involve verifying checksums, digital signatures, and potentially scanning for malware before, during, and after firmware updates or installations.

Here's a breakdown of the key aspects:

1. Why is Firmware Integrity Important?

Security:

Malicious modifications to firmware can compromise device security, allowing unauthorized access, data breaches, or even complete system control.

Functionality:

Corrupted or modified firmware can cause devices to malfunction, become unstable, or even become unusable.

Supply Chain Security:

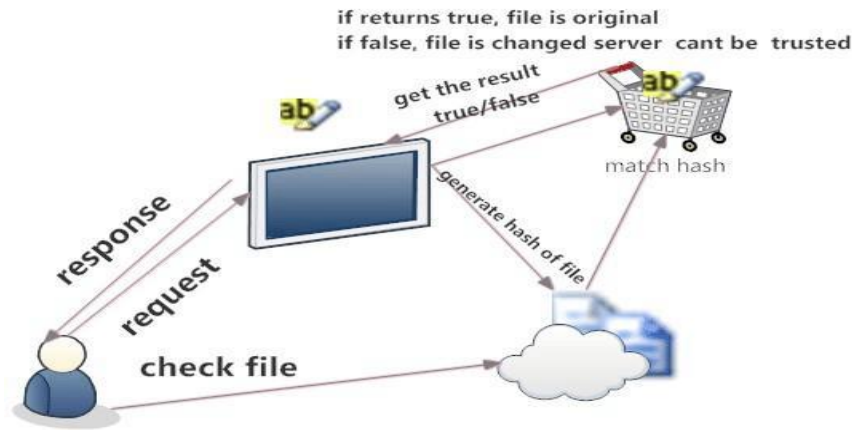


Figure 8.3: Secure Firmware Upload to Firebase Storage

Ensuring firmware integrity protects against supply chain attacks, where malicious firmware is pre-installed on devices before they reach the end user.

2. Methods for Firmware Integrity Checks:

Checksums/CRCs:

Calculating a checksum or CRC (Cyclic Redundancy Check) of the firmware image and comparing it to a stored value. If they match, the firmware is likely intact.

Digital Signatures:

Verifying the digital signature of the firmware using a public key infrastructure. This confirms the firmware's authenticity and that it hasn't been altered.

Malware Scanning:

Running a scan on the firmware image to detect known malware or malicious code.

Firmware Analysis Tools:

Using specialized tools to analyze the firmware for suspicious code or vulnerabilities.

BIOS-level Checks:

Some systems perform integrity checks at the BIOS level during boot-up, verifying the integrity of critical files and preventing unauthorized changes.

Integrity Verifier Tools:

Some systems have dedicated tools to verify the integrity of files within the currently installed firmware.

3. Implementation:

Data Preparation: Identify the data blocks (e.g., firmware images, configuration data) that require integrity checks.

Checksum/CRC Storage: Store or transmit the checksum or CRC value alongside the original data.

Verification: Recompute the checksum or CRC upon retrieval or reception and compare it with the stored value.

Signature Verification: Use public keys to verify the digital signature of the firmware.

Automated Checks: Integrate integrity checks into the firmware update process, so they are automatically performed

8.5 EXTENDING FOR MULTIPLE DEVICE TYPES

In real-world scenarios, an IoT or embedded system ecosystem often consists of multiple device types, each requiring a different firmware file, hardware configuration, or update schedule. A scalable firmware server must be flexible enough to handle these variations efficiently.

What Are "Multiple Device Types"?

Examples:

Different hardware models (e.g., ESP32, STM32, Raspberry Pi).

Variants with different sensors or memory sizes.

Devices with regional or customer-specific configurations.

Each type may require:

Different firmware binaries.

Different version histories.

Different release schedules.

How Devices Determine Their Type:

Devices should be pre-programmed with a device identifier such as:

A model string: "esp32-s2"

A hardware ID from OTP memory or bootloader

A serial number that maps to a model type

CHAPTER 9

RASPBERRY PI SOFTWARE STACK

9 Chapter (9): Raspberry Pi Software Stack

9.1 SOCKETCAN INTERFACE (CAN_BUS.PY)



Figure 9.1 : Typical CAN communication layers – SocketCAN vs traditional drivers

SocketCAN is a set of drivers and a networking stack contributed by to the . SocketCAN was formerly known as Low Level CAN Framework (LLCF).

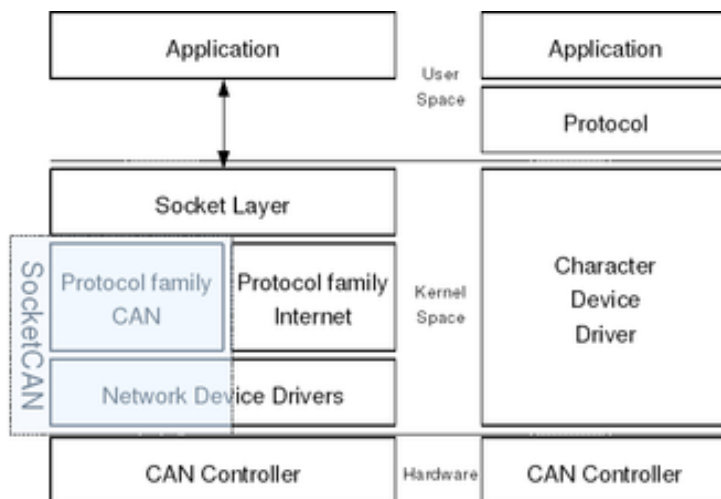


Figure 9.2 :Data flow from sensor acquisition to dispatch using Observer and Reactor Patterns

Typical CAN communication layers. With SocketCAN (left) or conventional (right).

Traditional CAN drivers for Linux are based on the model of character devices. Typically they only allow sending to and receiving from the CAN controller. Conventional implementations of this class of device driver only allow a single process to access the device, which means that all other processes are blocked in the meantime. In addition, these drivers typically all differ slightly in the interface presented to the application, stifling portability. The SocketCAN concept on the other hand uses the model of network devices, which allows multiple applications to access one CAN device simultaneously. Also, a single application is able to access multiple CAN networks in parallel.

Usage

The application first sets up its access to the CAN interface by initialising a socket (much like in TCP/IP communications), then binding that socket to an interface (or all interfaces, if the application so desires). Once bound, the socket can then be used like a socket via read, write, etc...

added support for SocketCAN in version 3.3. An open source library provides SocketCAN support for Python 2 and Python 3.

9.2 SENSOR LISTENER & DISPATCHER (SENSOR_LISTENER.PY)

The `sensor_listener.py` module plays a central role in IoT systems: continually gathering sensor data, applying validation or preprocessing, and dispatching that data to appropriate sinks (databases, cloud endpoints, dashboards, etc.). Its design often follows well-established software design patterns to ensure scalability, maintainability, and flexibility.

Key Design Patterns

Observer Pattern (a.k.a. Publish–Subscribe or EventListener)

Enables a publisher (subject) to notify multiple subscribers (observers) when new data arrives.

Promotes loose coupling: the publisher doesn't depend on the logic or number of subscribers

Widely recognized in Gang of Four patterns (Gamma et al.), especially for GUI and event-driven systems

Reactor Pattern

Maintains a central event loop that demultiplexes multiple I/O sources and dispatches them to handlers

Effective for handling asynchronous sensor inputs or interrupts without blocking threads.

9.3 DIAGNOSTICS CLIENT (UDS OVER CAN)

9.3.1 SIMPLE INTRO TO UDS (UNIFIED DIAGNOSTIC SERVICES)?

In this practical tutorial, we introduce the UDS protocol basics with focus on UDS on CAN bus (UDSonCAN) and Diagnostics over CAN (DoCAN). We also introduce the ISO-TP protocol and explain the difference between UDS, OBD2, WWH-OBD and OBDonUDS.

Finally, we'll explain how to request, record & decode UDS messages - with practical examples for logging EV State of Charge and the Vehicle Identification Number (VIN).

9.3.2 WHAT IS THE UDS PROTOCOL?

Unified Diagnostic Services (UDS) is a communication protocol used in automotive Electronic Control Units (ECUs) to enable diagnostics, firmware updates, routine testing and more.

The UDS protocol (ISO 14229) is standardized across both manufacturers and standards (such as CAN, KWP 2000, Ethernet, LIN). Further, UDS is today used in ECUs across all tier 1 Original Equipment Manufacturers (OEMs).

In practice, UDS communication is performed in a client-server relationship - with the client being a tester-tool and the server being a vehicle ECU. For example, you can connect a interface to the of a car and send UDS requests into the vehicle. Assuming the targeted ECU supports UDS services, it will respond accordingly.

9.3.3. DS vs CAN bus: Standards & OSI model

To better understand UDS, we will look at how it relates to CAN bus and the OSI model.

As explained in our , the Controller Area Network serves as a basis for communication. Specifically, CAN is described by a data-link layer and physical layer in the OSI model (as per ISO 11898). In contrast to CAN, UDS (ISO 14229) is a 'higher layer protocol' and utilizes both the session layer (5th) and application layer (7th) in the OSI model as shown below:

7 layer OSI model | Unified Diagnostic Services (UDS)

	UDS on CAN bus	UDS on FlexRay	UDS on IP	UDS on K-Line	UDS on LIN bus
Application	Specification and requirements ISO 14229-1				
	UDSonCAN ISO 14229-3	UDSonFR ISO 14229-4	UDSonIP ISO 14229-5	UDSonK-Line ISO 14229-6	UDSonLIN ISO 14229-7
Presentation	Vehicle manufacturer specific				
Session	Session layer services ISO 14229-2				
Transport	Transport & network layer services DoCAN ISO 15765-2	Transport & network layer services CoFR ISO 10681-2	Transport & network layer services DoIP ISO 13400-2	Not applicable	Transport & network layer services LIN ISO 17987-2
Network					
Data link	CAN ISO 11898-1	FlexRay ISO 17458-2	DoIP IEEE 802.3 ISO 13400-3	DoK-Line ISO 14230-2	LIN ISO 17987-3
Physical	CAN ISO 11898-2	FlexRay ISO 17458-4		DoK-Line ISO 14230-1	LIN ISO 17987-4

Figure 9.3 :OSI layer mapping of CAN and UDS protocols

9.3.4. Overview of UDS standards & concepts

UDS refers to a large number of standards/concepts, meaning it can be a bit overwhelming. To give you an overview, we provide a high level explanation of the most relevant ones below (with focus on CAN as the basis).

9.4 OTA CLIENT (CHUNKED UPLOAD)

In scenarios with limited memory or unreliable networks, a firmware upload in small, manageable chunks is essential. This section outlines how to implement a reliable, secure, and resumable OTA client using a chunked upload approach.

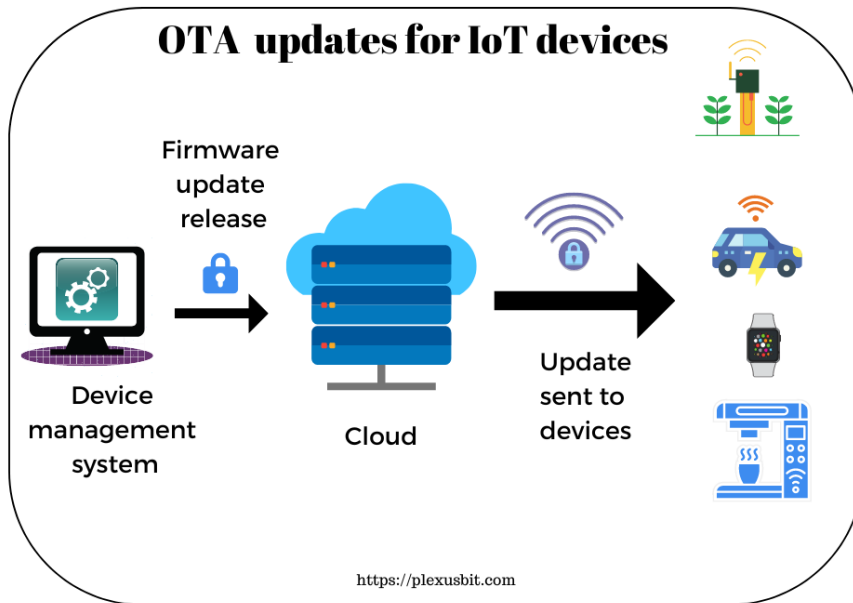


Figure 9.4 :OSI layer mapping of CAN and UDS protocols

9.4.1. Why Chunked Upload?

Chunked upload breaks a large firmware binary into smaller parts, offering key advantages:

Memory Efficiency: Device buffers one chunk at a time, avoiding high RAM usage .

Resilience & Retry: If a chunk fails, only that segment is retransmitted, not the entire file .

Low-latency Streaming: Chunks begin transmission immediately, useful in real-time or constrained connectivity .

9.4.2 CHUNKED OTA PROCESS FLOW

Handshake: Device requests OTA; server responds with total size, chunk size, and metadata hash/signature.

Chunk Upload Loop:

Read next chunk (e.g., 1 KB–64 KB).

Upload via HTTP POST or MQTT publish with offset info.

Wait for server acknowledgment; if failure, retry N times.

Completion:

After all chunks received, server verifies total hash or signature.

Server notifies client to finalize.

Finalization:

Device performs integrity check (SHA256 or signature).

If valid, swap to the new firmware (dualbank A/B) and reboot.

If invalid, trigger rollback

9.4.3. OTA Storage & Partitioning

Use dual partitions (A/B or dual-bank) or separate flash areas:

One remains active while writing the update to the inactive slot.

After a successful update and quick health check, switch boot to the new slot.

On failure, revert to the old firmware

9.4.4. Security & Integrity Checks

Per-chunk ACK & CRC/checksum ensures each chunk arrives intact .

Final full-image hash or digital signature (e.g. SHA256 + RSA) to confirm authenticity .

Anti-rollback: Check version metadata to prevent downgrades .

Transport encryption: Use HTTPS or MQTT over TLS .

9.5 SERVERCLIENT (FIRMWARE VERSION CHECK & DOWNLOAD)

The ServerClient component is responsible for connecting a device to the cloud firmware server. Its two main tasks are:

Check if a new firmware version is available for the specific device.

Download the firmware securely when needed.

This client runs on the device and handles communication with your backend via HTTP(S) or MQTT, depending on your infrastructure.

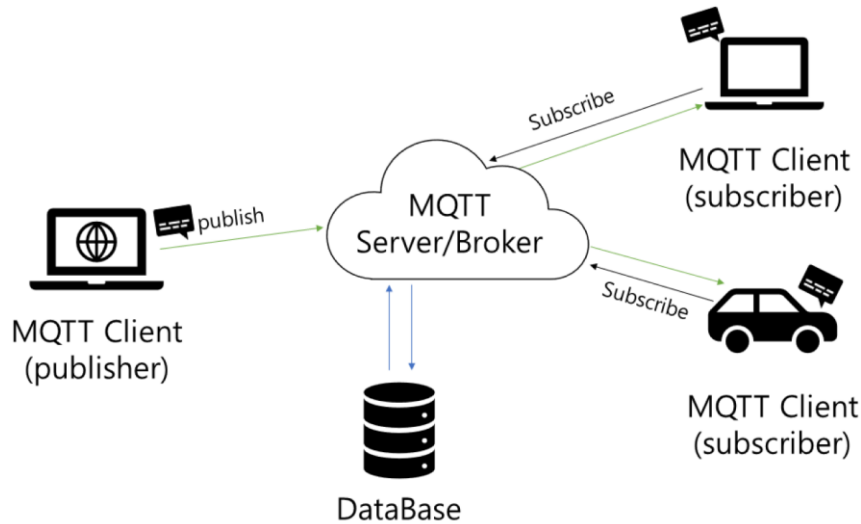


Figure 9.5 :OTA chunked upload process flow – handshake, upload loop, and finalization

9.5.1. Firmware Version Check

The device periodically checks with the server:

Its current firmware version

Its device type or model

Its update group (e.g., beta, production)

9.5.2. Firmware Download

If an update is available, the client securely downloads the firmware file. It can do this in two ways:

A. Full File Download

The firmware binary is downloaded in one request.

Suitable for stable, high-speed networks.

B. Chunked Download

Download in small blocks (e.g. 4 KB per chunk).

Ideal for constrained devices or poor network conditions.

Resume capability if interrupted (using offset headers or parameters).

CHAPTER 10

WEB GUI DESIGN

10 Chapter(10):Web GUI Design

10. WEB GUI DESIGN

A graphical user interface, or GUI , is a form of that allows to through and visual indicators such as . In many applications, GUIs are used instead of , which are based on typed command labels or text navigation. GUIs were introduced in reaction to the perceived steep of (CLIs), which require commands to be typed on a .

The actions in a GUI are usually performed through of the graphical elements. Beyond computers, GUIs are used in many handheld such as players, portable media players, gaming devices, and smaller household, office and . The term GUI tends not to be applied to other lower- , such as (where (HUDs) are preferred)

10.1 TECHNOLOGY CHOICES (FLASK + HTML/CSS/JS + SSE)

To build a lightweight yet powerful web-based GUI for IoT or embedded firmware systems, choosing the right stack is crucial. Below is an explanation of the selected technologies and why they work well together.

Why Flask?

Minimalistic and modular: easy to extend with REST APIs, templates, or WebSockets.

Perfect for embedded systems or Raspberry Pi environments.

Integrates well with Python-based backends (firmware logic, device communication).

Role in the GUI Stack:

Hosts the backend server.

Provides REST API endpoints for firmware status, updates, sensor data, etc.

Streams real-time updates via SSE (Server-Sent Events).

Renders dynamic or static pages using Jinja2 templates.

These technologies define the presentation layer and interaction logic.

HTML

Describes the structure of the user interface.

Components: buttons, tables, progress bars, dropdowns, etc.

CSS

Handles UI styling and layout.

Responsive design using flex/grid or frameworks like Bootstrap/Tailwind.

JavaScript (JS)

Adds interactivity: button actions, real-time updates, animations.

Consumes REST APIs or listens to real-time data via SSE or AJAX.

Manipulates the DOM dynamically.

Why use SSE (instead of WebSocket)?

Use Cases in Web GUI:

Live firmware update progress.

Streaming logs from the device.

Real-time sensor reading updates.

System status alerts.

10.2 TEMPLATE LAYOUT & STATIC ASSETS

A GUI uses a combination of technologies and devices to provide a platform that users can interact with, for the tasks of gathering and producing information.

A series of elements conforming a have evolved to represent information stored in computers. This makes it easier for people with few computer skills to work with and use computer software. The most common combination of such elements in GUIs is the windows, icons, text fields, canvases, menus, pointer (WIMP) paradigm, especially in personal computes.

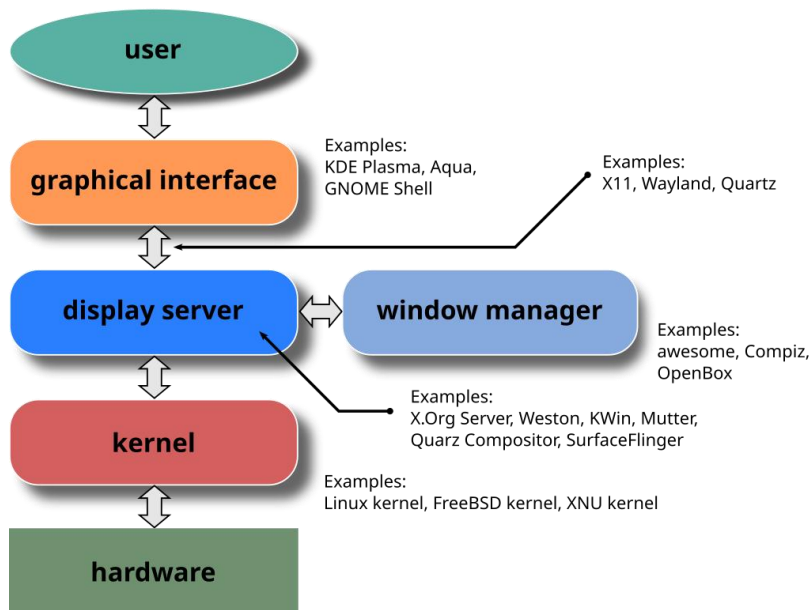


Figure 10.1 :Layers of a GUI based on a windowing system

Layers of a GUI based on a windowing system

The WIMP style of interaction uses a virtual to represent the position of a , most often a , and presents organized in windows and represented with . Available commands are compiled together in menus, and actions are performed making gestures with the pointing device. A facilitates the interactions between windows, , and the . The windowing system handles hardware devices such as pointing devices, graphics hardware, and positioning of the pointer.

In , all these elements are modeled through a to produce a simulation called a in which the display represents a desktop, on which documents and folders of documents can be placed. Window managers and other software combine to simulate the desktop environment with varying degrees of realism.

Entries may appear in a list to make space for text and details, or in a grid for compactness and larger icons with little space underneath for text. Variations in between exist, such as a list with multiple columns of items and a grid of items with rows of text extending sideways from the icon.

10.3 LIVE SPEED STREAMING

Live speed streaming refers to real-time display of speed values from a moving device (car, robot, drone, etc.) in a web-based GUI. The system continuously collects speed data from a sensor and streams it to the frontend where it's displayed dynamically — often on a dashboard, speed gauge, or graph.

1. Sensor reads speed data (e.g., via GPIO or serial).
2. Flask backend reads that value every few ms/sec.
3. SSE stream pushes new data to the browser.
4. JavaScript updates the UI instantly.

1. Speed updates in real-time every second.
2. No need to refresh or poll the page.
3. Extremely lightweight: suitable for Raspberry Pi or embedded systems.

You can enhance this by adding:

Graph plotting (e.g., Chart.js or D3.js)

Speed gauge UI (with CSS or JS libraries like JustGage)

Logging module to store speed history

Alerts for speed thresholds (e.g., red warning over 100 km/h)

MQTT publishing for distributed systems

10.4 DIAGNOSTICS PANEL & AUTO-HIDE BEHAVIOR

The Diagnostics Panel in modern embedded web GUIs provides real-time visibility into system parameters such as CPU usage, memory footprint, temperature, firmware version, and network status. It plays a critical role in maintaining system transparency, aiding debugging, and facilitating remote support.

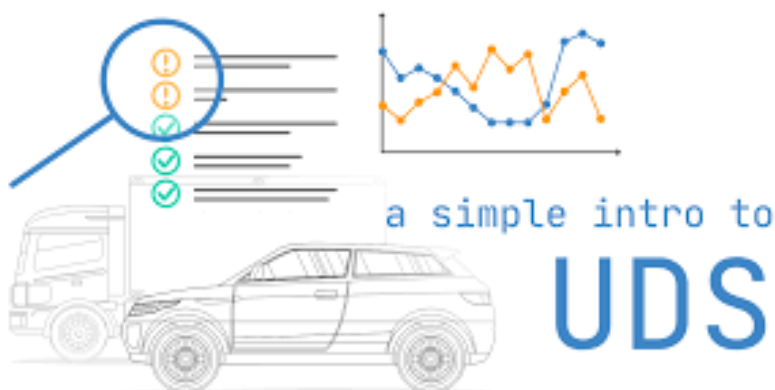


Figure 10.2 :System architecture of live speed streaming via SSE

Key Design Considerations:

Non-intrusiveness: The panel should not obstruct primary UI workflows. For this reason, auto-hide behavior (also known as transient UI) is widely adopted. Auto-hide ensures that the panel appears only when triggered (e.g., via hover, tap, or error signal) and fades away after a timeout.

Responsiveness: The panel should be reactive to live data, which is typically streamed via Server-Sent Events (SSE) or WebSocket protocols.

Data Granularity: System designers may opt for toggles between basic and advanced views, showing either high-level status or raw diagnostic payloads.

Implementation Insight:

Auto-hide can be achieved using CSS animations + JavaScript timers.

A div element styled with position: fixed; right: 0; may slide in/out based on focus or alerts.

Diagnostic data sources might include /proc on Linux-based systems, or firmware-level telemetry exposed via UART/CAN.

10.5 OTA UPDATE WORKFLOW & PROGRESS BAR

Over-the-Air (OTA) updates are essential in modern embedded systems, allowing firmware upgrades to be performed remotely without requiring physical access. This is especially useful in distributed IoT deployments and automotive environments where system downtime or manual intervention is costly.

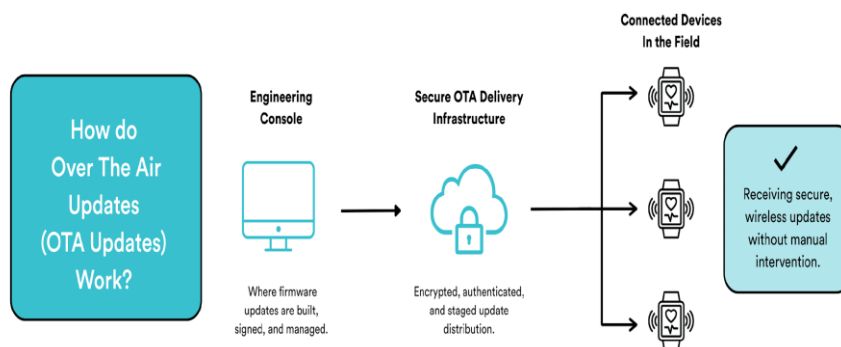


Figure 10.3 : System architecture of live speed streaming via SSE

Version Check

The device communicates with a remote server using a REST API or MQTT message to compare its current firmware version with the latest available version. If an update is found, metadata such as the firmware size, hash (e.g., SHA-256), and download URL is returned.

Firmware Download

The client proceeds to download the new firmware binary. On resource-constrained devices, this is usually done in chunks (e.g., 4 KB at a time), using HTTP range requests or a chunked REST API. This ensures resilience to unstable connections.

Integrity Verification

Once downloaded, the firmware is validated using hash-based checks (e.g., SHA-256) or cryptographic signature verification (e.g., RSA or ECDSA). This prevents corrupted or malicious binaries from being installed.

Flash Write

After validation, the firmware is written to an inactive flash partition (in dual-bank systems), or into a dedicated upgrade memory region. Write protection is often used to prevent mid-flash corruption.

Boot Switch & Reboot

The device updates its bootloader pointer to the new partition and initiates a soft or hard reboot. Post-reboot, the new firmware runs.

Post-update Health Check

A watchdog mechanism monitors the new firmware's startup and basic operation. If it fails (e.g., a bootloop or crash), the system automatically rolls back to the last known working version.

To enhance the user experience, the GUI should include a progress bar that visually represents the OTA update status. This helps users (or operators) monitor the update phases, identify stalled transfers, and estimate time remaining.

Typical stages include:

Initiating connection

Downloading firmware

Verifying integrity

Writing to flash

Rebooting

To ensure safe OTA deployment, the following practices are recommended:

Use A/B partitioning for safe fallback

Sign firmware with public-key cryptography

Use secure transport (HTTPS, TLS)

Implement a retry/backoff mechanism for unstable networks

Use chunked downloads to avoid memory overflows

Provide clear UI feedback to users

10.6 RESPONSIVE DESIGN & UX CONSIDERATIONS

In embedded and IoT-based web applications, responsive design and user experience (UX) are no longer optional they are essential. These systems often operate across a variety of devices, from laptops and tablets to mobile phones and even embedded touchscreen displays (e.g., 7" Raspberry Pi screens). Designing a flexible, responsive, and intuitive GUI ensures that users can interact efficiently with the system regardless of screen size or input method.

Responsive design allows a single web interface to adapt fluidly to different screen sizes and resolutions. This is achieved using layout strategies and CSS techniques such as:

Flexible Grid Layouts

Use CSS Grid or Flexbox to create scalable layouts that adapt to screen width.

Media Queries

Define breakpoints for specific screen sizes (e.g., phones vs. tablets vs. desktops).

Relative Units

Use em, %, vh, vw instead of fixed px values to support dynamic scaling.

Mobile-first Design

Begin designing for the smallest screens, and progressively enhance the layout for larger screens.

In addition to layout responsiveness, good UX design involves understanding how the user interacts with the system and ensuring that the interface is efficient, intuitive, and error-tolerant.

Key UX Guidelines:

Consistency: Buttons, forms, and navigation must behave uniformly.

Feedback: Provide real-time feedback (e.g., status indicators, progress bars, toasts).

Accessibility: Support high-contrast themes, large font options, and keyboard navigation.

Minimalism: Limit screen clutter; avoid displaying too many elements at once.

Touch Optimization: Increase button size and spacing on small/touch screens.

Embedded Constraints:

Lower processing power and RAM.

Limited screen real estate.

Need for offline functionality and fast load times.

To address these, designers often adopt progressive enhancement — loading only core elements first, then adding complexity (e.g., charts, graphs, logs) conditionally.

For embedded-friendly responsive GUIs, the following tools are widely used:

Bootstrap: Lightweight responsive UI framework.

Tailwind CSS: Utility-first CSS framework ideal for modular UI.

Vue.js / React: Component-driven UIs with dynamic rendering.

SSE/WebSocket support: For real-time updates with minimal load.

CHAPTER 11

SYSTEM INTEGRATION & DEPLOYMENT

11 Chapter (11): System Integration & Deployment

11.1 RASPBERRY PI AUTO-START (SYSTEM SERVICES)

System Integration & Deployment

describes how to bring together all software and hardware components, configure the Raspberry Pi to launch services on boot, initialize the CAN interface, secure access to the system, and implement logging and monitoring to ensure reliable operation in the vehicle environment.

11.1.1 Raspberry Pi Auto-Start (systemd Services)

To guarantee that critical processes (CAN listener, Flask server, OTA client) start automatically after power-up or reboot, we use **systemd** service units:

1. **Create a Service File**

For each component, e.g. can-listener.service:

```
[Unit]
Description=CAN Listener Service
After=network.target

[Service]
Type=simple
User=pi
ExecStart=/usr/bin/python3 /home/pi/app/can_listener.py
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

2. **Enable and Star**

```
sudo cp can-listener.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable can-listener.service
sudo systemctl start can-listener.service
```

3. Verify Status

```
sudo systemctl status can-listener.service  
journalctl -u can-listener.service --no-pager
```

Repeat the above for the Flask web server (flask-gui.service) and OTA client (ota-client.service). This ensures reliable auto-start and automatic restarts on failure.

11.1.2 CAN Interface Initialization

Proper initialization of the SocketCAN interface is essential for communication:

1. Load the SPI and CAN Modules

Ensure /boot/config.txt contains:

```
dtoverlay=spi=on  
dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25  
dtoverlay=spi-bcm2835
```

2. Bring Up the CAN Network

Add to /etc/rc.local (before exit 0) or in a shell script run by systemd:

```
ip link set can0 up type can bitrate 500000
```

3. Verify Link

```
ip -details link show can0  
candump can0
```

4. Persist Across Reboots

Use a dedicated unit can0-init.service with ExecStart=/sbin/ip link set can0 up ... and enable as above.

11.1.3 Security & Access Control

To protect both the Raspberry Pi host and the vehicle's network:

1. User and Group Permissions

- Run CAN and Flask processes under a dedicated, unprivileged user (e.g., vehicle), added to the gpio and dialout (or can) groups.
- Restrict file permissions on configuration and firmware directories (chmod 750, owned by vehicle).

2. SSH Hardening

- Disable password login; use SSH keys only.
- Change default port and disable root login in /etc/ssh/sshd_config.

3. Firewall Rules

- Install and configure ufw to allow only necessary ports:

```
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 5000/tcp    # Flask (or your chosen port)
sudo ufw enable
```

4. Secure OTA Channel

- Use HTTPS for REST API calls to AWS S3 (certificate validation).
- Verify firmware signatures in the bootloader before writing to flash.

11.1.4 Logging & Monitoring

Continuous logging and health monitoring help detect issues early:

1. System Logs

- Rely on journald (via journalctl) for service output.
 - Rotate large logs by configuring /etc/systemd/journald.conf or forwarding to rsyslog.
-

2. Application-Level Logging

- In Python services, use the logging module with rotating file handlers:

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger('vehicle')
handler = RotatingFileHandler('/var/log/vehicle/app.log', maxBytes=1e6, backupCount=5)
logger.addHandler(handler)
```

- Log CAN errors, OTA progress, and diagnostic events with timestamps and levels (INFO, WARNING, ERROR).

3. Health Checks & Alerts

- Implement simple HTTP health endpoints (/health) in the Flask server.
- Use a lightweight monitor (e.g., cron + curl) or integrate with a central system (Prometheus Node Exporter) to poll metrics and send alerts on failures.

4. Data Retention

- Keep logs for a defined period (e.g., 30 days) and archive older logs to external storage for post-mortem analysis.

11.2 STM32 ECUs Integration

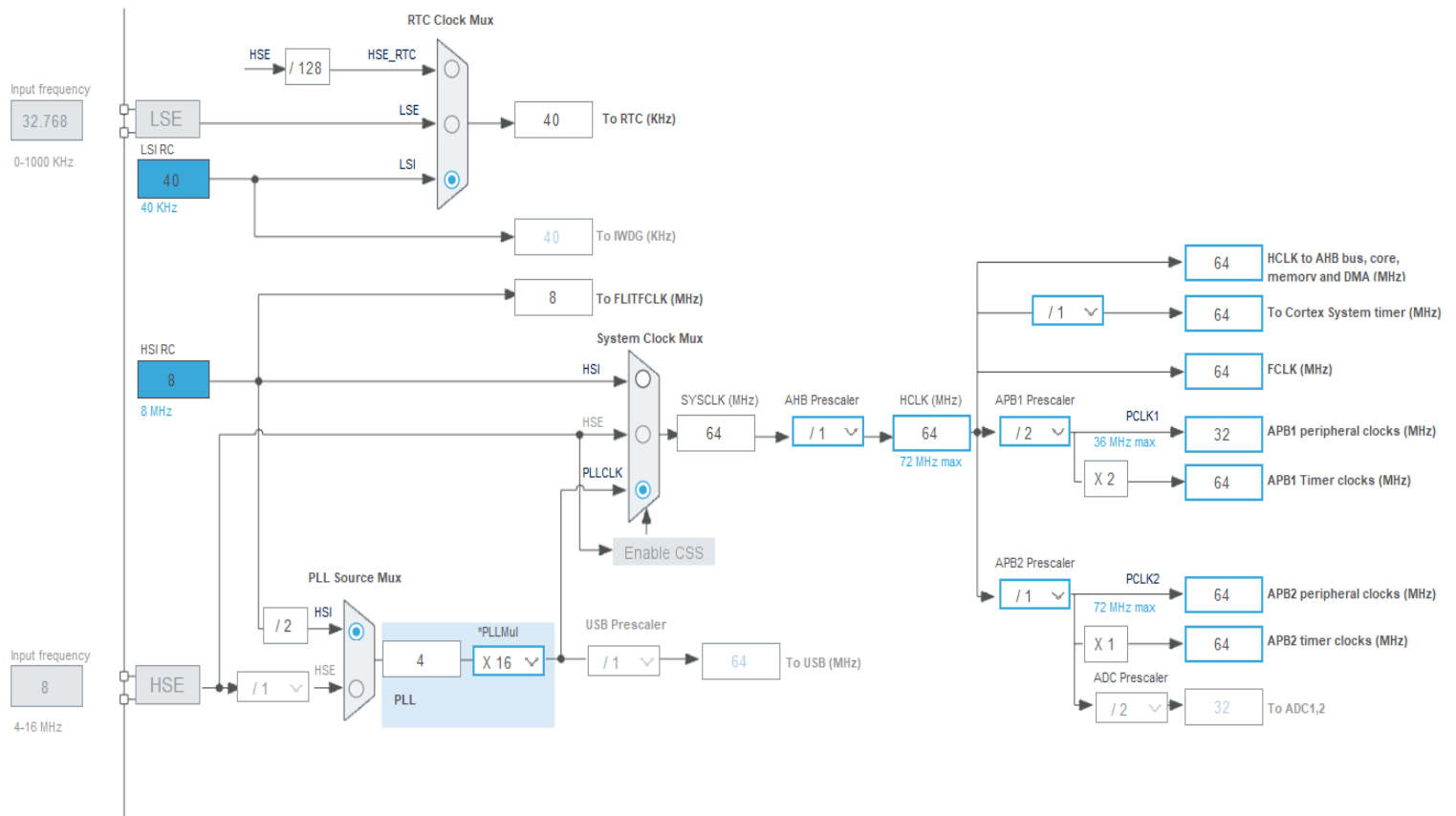
After configuring the Raspberry Pi, we integrate the two STM32F103 ECUs, each running FreeRTOS and sharing the same CAN bus but with distinct responsibilities:

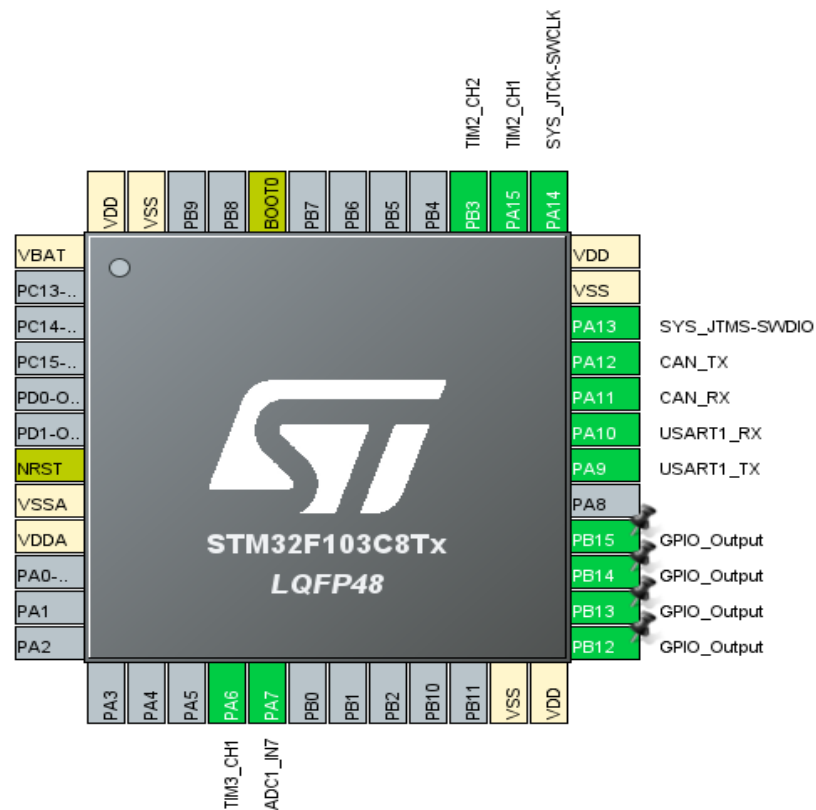
11.2.1 App ECU (STM32F103)

```
// Task initialization in main()
InitMotorTask();           /** Motor control & safety **/
InitBluetoothTask();       /** Remote control interface **/
InitUltrasonicTask();      /** Distance measurement **/
InitTemperatureSensorTask(); /** Temperature measurement **/
InitCANTransmitTask();     /** Outgoing CAN frames **/
InitCANReceiveTask();      /** Incoming CAN frames **/
vTaskStartScheduler();
```

- **Motor Task:** Computes PWM outputs and applies safety limits.
- **Bluetooth Task:** Listens for remote commands via Bluetooth module.
- **Ultrasonic & Temperature Tasks:** Sample sensors and push readings into queues.
- **CAN Transmit/Receive Tasks:** Route sensor data, control commands.

Configuration at STM32CubeIDE





3. Peripherals and Middlewares Configuration

3.1. ADC1

mode: IN7

3.1.1. Parameter Settings:

ADCs_Common_Settings:

Mode Independent mode

ADC_Settings:

Data Alignment Right alignment

Scan Conversion Mode Disabled

Continuous Conversion Mode Disabled

Discontinuous Conversion Mode Disabled

ADC_Regular_ConversionMode:

Enable Regular Conversions Enable

Number Of Conversion 1

External Trigger Conversion Source Regular Conversion launched by software

Rank 1

Channel Channel 7

Sampling Time 1.5 Cycles

ADC_Injected_ConversionMode:

Enable Injected Conversions Disable

WatchDog:

Enable Analog WatchDog Mode false

3.2. CAN**mode: Activated****3.2.1. Parameter Settings:****Bit Timings Parameters:**

Prescaler (for Time Quantum)	4 *
Time Quantum	125.0 *
Time Quanta in Bit Segment 1	13 Times *
Time Quanta in Bit Segment 2	2 Times *
Time for one Bit	2000 *
Baud Rate	500000 *
ReSynchronization Jump Width	1 Time

Basic Parameters:

Time Triggered Communication Mode	Disable
Automatic Bus-Off Management	Disable
Automatic Wake-Up Mode	Disable
Automatic Retransmission	Disable
Receive Fifo Locked Mode	Disable
Transmit Fifo Priority	Disable

Advanced Parameters:

Test Mode	Normal
-----------	--------

3.3. RCC**3.3.1. Parameter Settings:****System Parameters:**

VDD voltage (V)	3.3
Prefetch Buffer	Enabled
Flash Latency(WS)	2 WS (3 CPU cycle)

RCC Parameters:

HSI Calibration Value	16
HSE Startup Timeout Value (ms)	100
LSE Startup Timeout Value (ms)	5000

3.4. SYS**Debug: Serial Wire****Timebase Source: TIM1****3.5. TIM2****Channel1: PWM Generation CH1****Channel2: PWM Generation CH2****3.5.1. Parameter Settings:****Counter Settings:**

Prescaler (PSC - 16 bits value)	63 *
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits value)	999 *
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

Trigger Output (TRGO) Parameters:

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

PWM Generation Channel 1:

Mode	PWM mode 1
Pulse (16 bits value)	0
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

PWM Generation Channel 2:

Mode	PWM mode 1
Pulse (16 bits value)	0
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

3.6. TIM3**Channel1: Input Capture direct mode****3.6.1. Parameter Settings:****Counter Settings:**

Prescaler (PSC - 16 bits value)	0
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits value)	65535
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable

Trigger Output (TRGO) Parameters:

Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

Input Capture Channel 1:

Polarity Selection	Rising Edge
IC Selection	Direct
Prescaler Division Ratio	No division
Input Filter (4 bits value)	0

3.7. USART1**Mode: Asynchronous****Basic Parameters:**

Baud Rate	9600 *
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

Advanced Parameters:

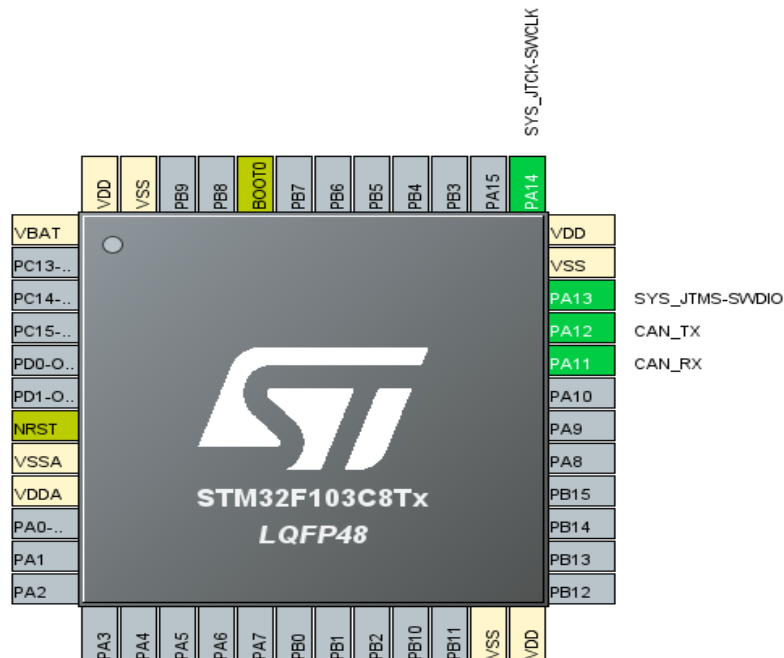
Data Direction	Receive and Transmit
Over Sampling	16 Samples

11.2.2 Main ECU (STM32F103)

```
// Task initialization in main()
InitCANReceiveTask();    /** Receive CAN frames **/
InitCANTransmitTask();   /** Transmit CAN frames **/
InitControlTask();       /** Sensor processing & motor safety **/
InitDiagnosticsTask();   /** UDS diagnostic services **/
InitOTAManagerTask();    /** Manage OTA trigger & metadata **/
vTaskStartScheduler();
```

- **CAN Receive/Transmit:** Handle OTA chunks, ACKs, and status messages.
- **Control Task:** Monitors vehicle state and enforces any fallback or reset.
- **Diagnostics Task:** Responds to UDS requests (session, security, DTC).
- **OTA Manager Task:** Coordinates between bootloader slots and activates the new firmware when ready.

Configuration at STM32CubeIDE



3. *Peripherals and Middlewares Configuration*

3.1. CAN

mode: Activated

3.1.1. Parameter Settings:

Bit Timings Parameters:

Prescaler (for Time Quantum)	4 *
Time Quantum	125.0 *
Time Quanta in Bit Segment 1	13 Times *
Time Quanta in Bit Segment 2	2 Times *
Time for one Bit	2000 *
Baud Rate	500000 *
ReSynchronization Jump Width	1 Time

Basic Parameters:

Time Triggered Communication Mode	Disable
Automatic Bus-Off Management	Disable
Automatic Wake-Up Mode	Disable
Automatic Retransmission	Disable
Receive Fifo Locked Mode	Disable
Transmit Fifo Priority	Disable

Advanced Parameters:

Test Mode	Normal
-----------	--------

3.2. RCC

3.2.1. Parameter Settings:

System Parameters:

VDD voltage (V)	3.3
Prefetch Buffer	Enabled
Flash Latency(WS)	2 WS (3 CPU cycle)

RCC Parameters:

HSI Calibration Value	16
HSE Startup Timeout Value (ms)	100
LSE Startup Timeout Value (ms)	5000

3.3. RTC

mode: Activate Clock Source

mode: Activate Calendar

3.3.1. Parameter Settings:

Calendar Time:

Data Format	Binary data format *
Hours	0
Minutes	0
Seconds	0

General:

Auto Predivider Calculation	Enabled
Asynchronous Predivider value	Automatic Predivider Calculation Enabled
Output	Alarm pulse signal on the TAMPER pin

Calendar Date:

Week Day	Monday
Month	January
Date	1
Year	0

3.4. SYS

Debug: Serial Wire

Timebase Source: TIM1

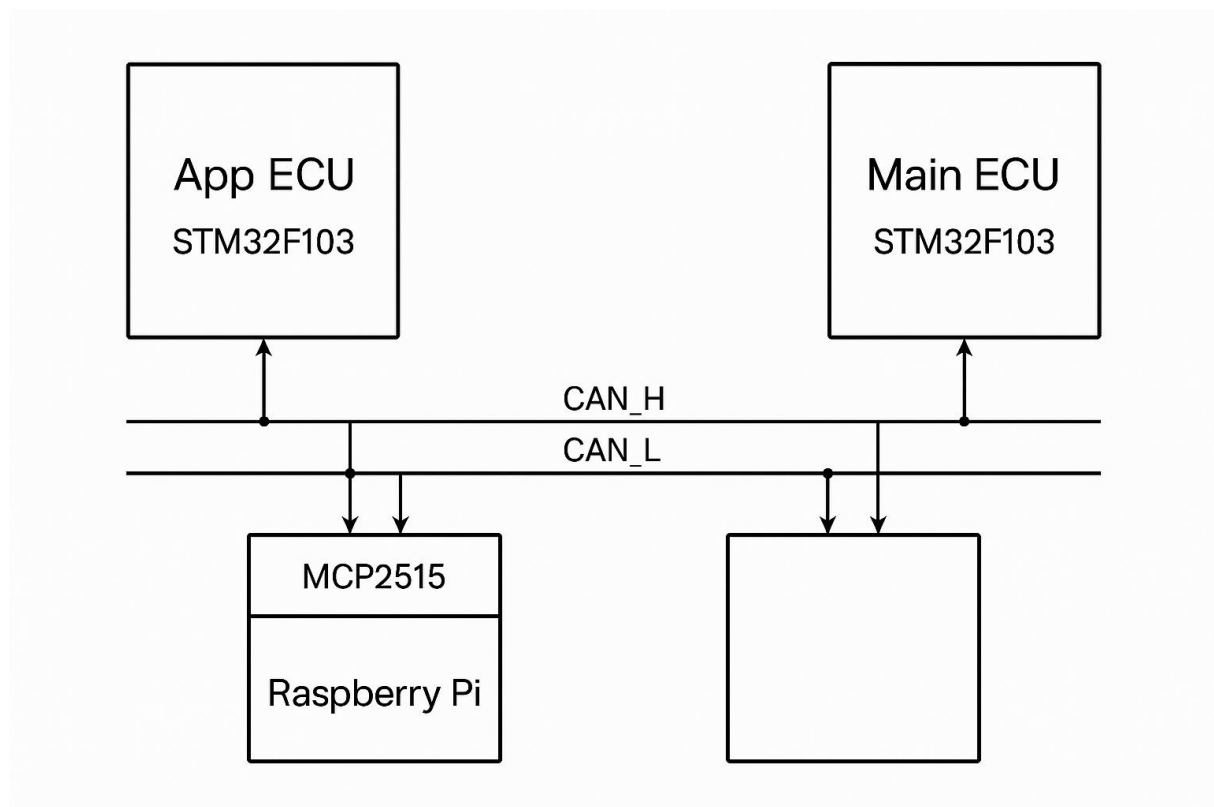
11.3 CAN Bus Wiring & Topology

To achieve reliable communication between all units:

1. Physical Wiring

- Use a twisted pair (CAN_H, CAN_L) with 120 Ω termination resistors at each end of the bus.
- Tie all ECU and Raspberry Pi grounds together at a single point to minimize electrical noise.

2. Bus Connection Diagram



11.4 System Overview & Project Images

11.4.1 System Overview

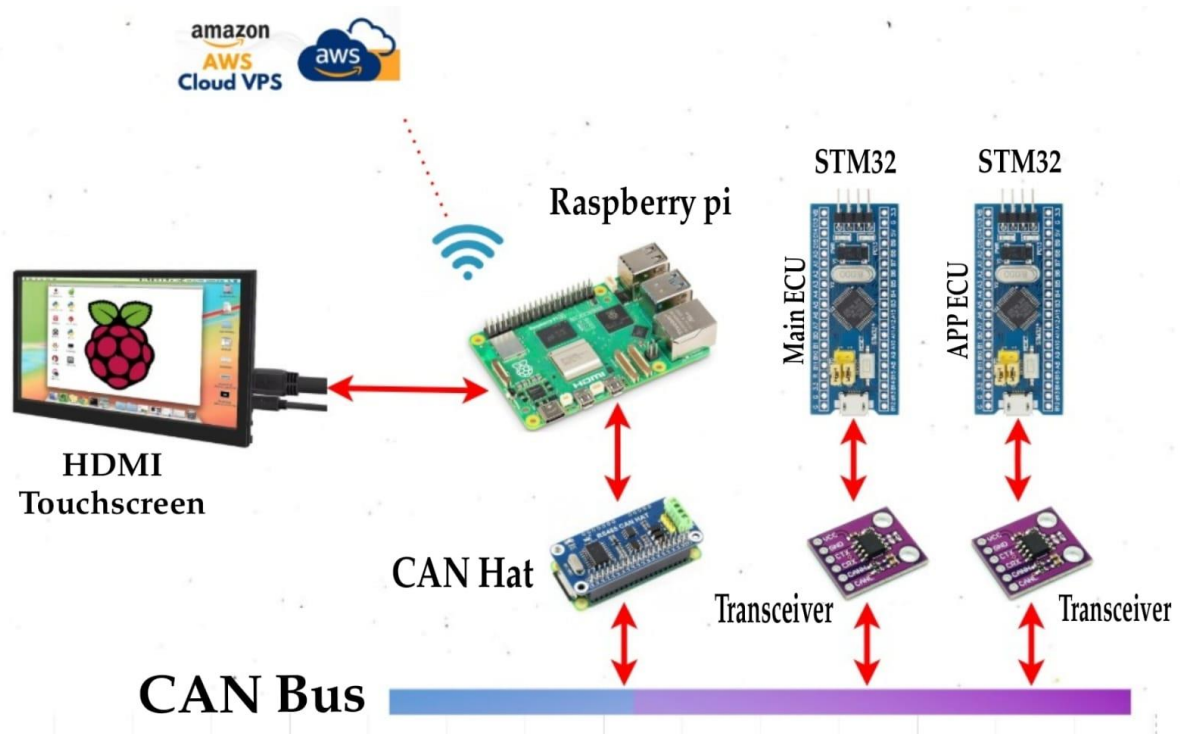


Figure 11.1: System Overview

11.4.2 Project Photos

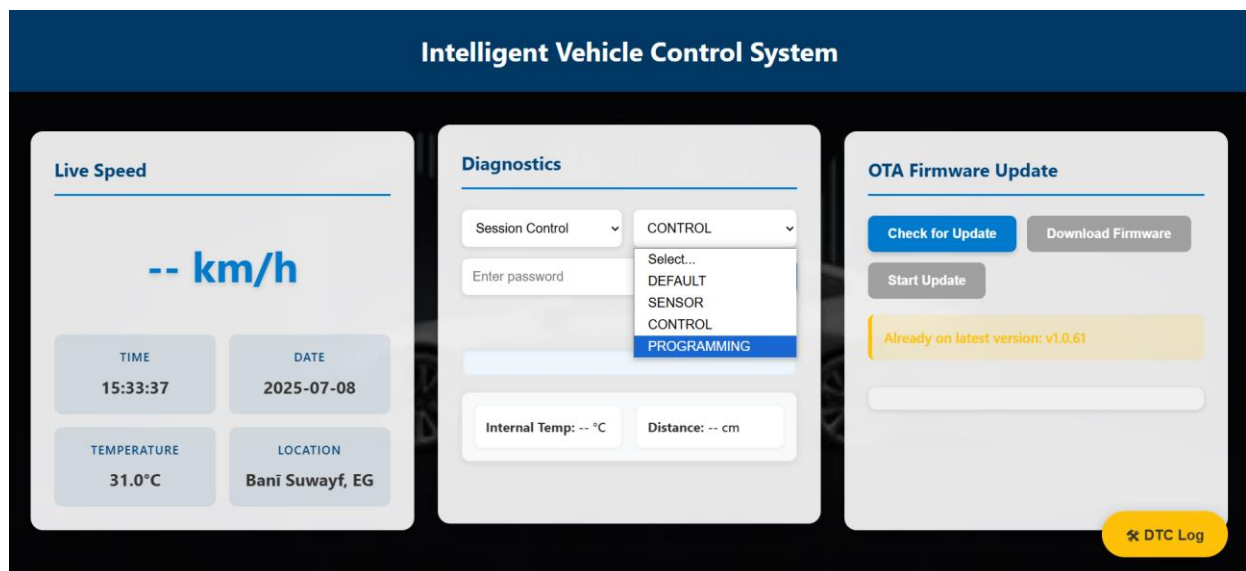
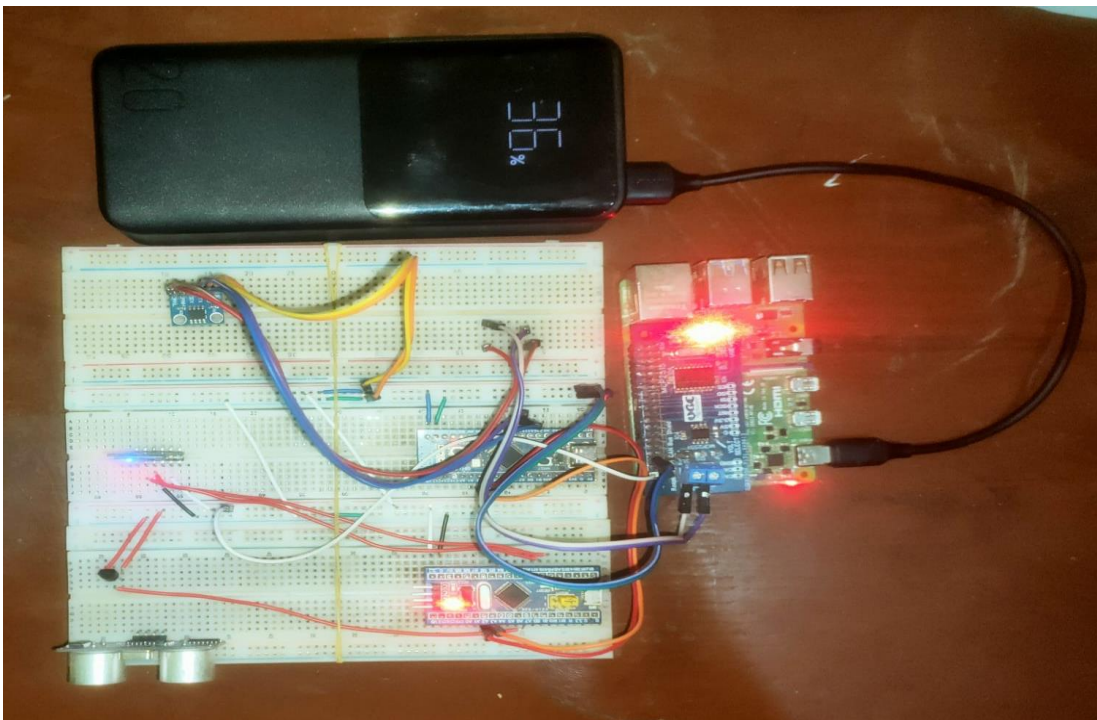
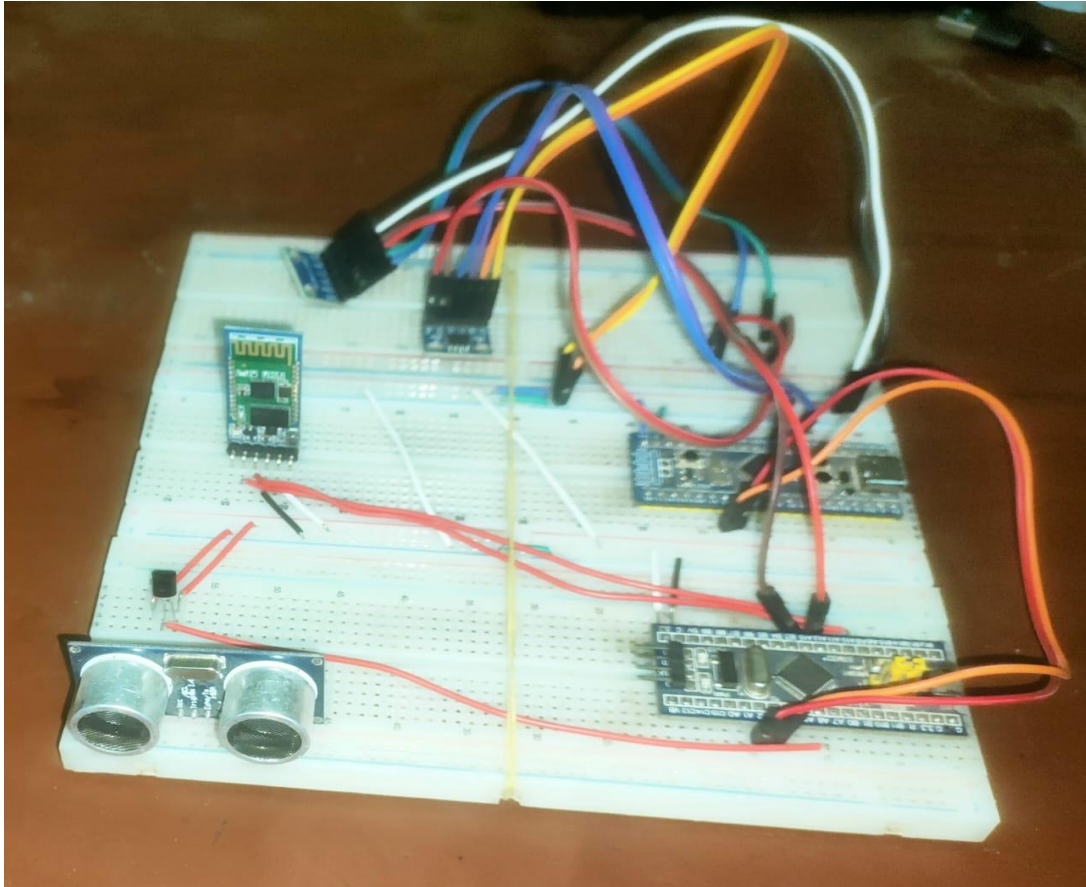
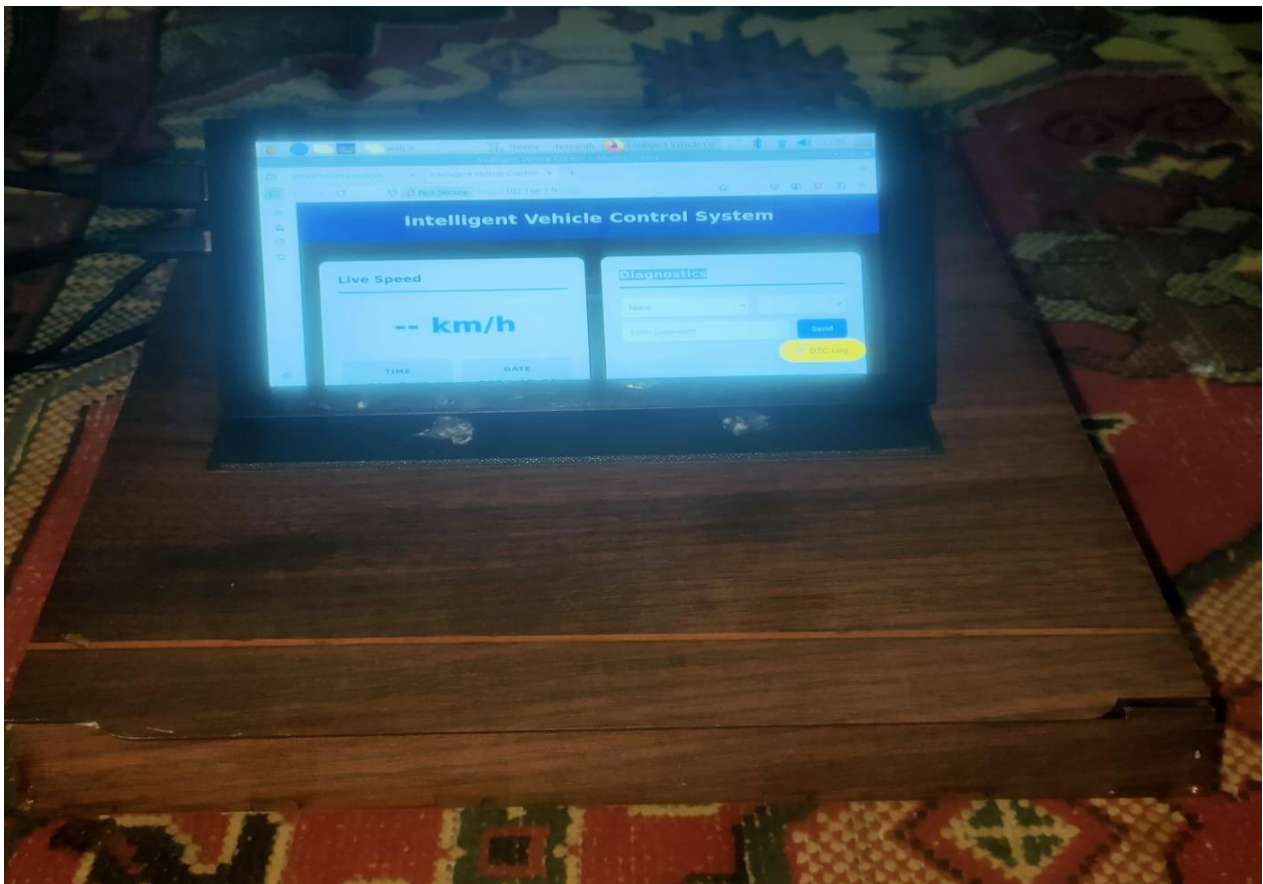


Figure 11.2: GUI





CHAPTER 12

Results & Discussion

12 Chapter (12): Results & Discussion

12.1 System Validation Results

The developed automotive diagnostic and control system was thoroughly validated to ensure that all functional requirements were met according to the design specifications. The validation process was structured around three main axes: hardware functionality, software behavior, and communication integrity across the CAN network.

From the hardware perspective, key components such as the dual STM32F103 microcontrollers, the Raspberry Pi 4, and the L298N motor driver were individually tested and integrated as a full system. Each microcontroller successfully executed its dedicated role— the **Main ECU** managed real-time motor control and CAN communication, while the **App ECU** handled application-level diagnostics and OTA updates.

CAN bus communication, facilitated via the **MCP2515 + TJA1050** transceivers, was verified using diagnostic tools and custom test scripts. All core UDS services were implemented and tested over CAN, including:

- Diagnostic Session Control (0x10)
- Security Access (0x27)
- Read Data by Identifier (0x22)
- Request Download (0x34)

Each service responded correctly to valid requests and rejected invalid or unauthorized access attempts, confirming both functionality and robustness of the UDS stack.

The **OTA update mechanism** was tested by simulating firmware chunk transfers via CAN. The bootloader correctly interpreted metadata, wrote the received chunks to flash memory, and switched control to the new firmware upon verification. Fail-safe mechanisms such as rollback on corruption were also validated.

Motor control was tested through commands sent via the App ECU, with real-time monitoring via HMI and logging via the Raspberry Pi. The motors responded with minimal latency, and safety conditions (like over-temperature) triggered appropriate shutdown routines as designed.

Overall, the system passed all planned validation scenarios, demonstrating the reliability and correctness of both hardware and software integration.

12.2 Performance Analysis

The performance of the embedded diagnostic and control system was assessed based on key operational metrics, including task response times, communication latency, memory utilization, and motor control behavior under different loads. Each subsystem was analyzed to ensure that it met real-time constraints and operated efficiently within the hardware limitations.

Task Response Times

Using FreeRTOS, the system was designed with dedicated tasks for CAN reception, transmission, motor control, diagnostics, and OTA updates. Task switching and execution were monitored using internal counters and GPIO toggling techniques.

The **CAN Receive Task** responded within ~1–2 ms under normal traffic, and the **Motor Control Task** executed at regular intervals of 10 ms, meeting safety-critical timing for motor protection.

CAN Bus Latency

The latency for CAN messages was measured from transmission by the App ECU to reception by the Main ECU. For standard 8-byte frames at 500 kbps, the end-to-end delay was consistently below 5 ms, ensuring smooth motor command execution and prompt diagnostic feedback.

Error handling and arbitration delay were minimal due to efficient message prioritization.

Memory & Flash Utilization

The Flash layout allowed sufficient space for the bootloader, primary firmware, and metadata. OTA updates were tested with firmware sizes up to 64 KB, with the system maintaining integrity through chunk-based reception.

RAM usage remained within 60% of available space, leaving margin for diagnostic buffers and OTA agent stacks.

Motor Performance

Motor control was validated under various scenarios, including load changes and emergency stop conditions. The L298N driver reacted quickly to control signals, and over-temperature detection via LM35 triggered safety shutdown as expected.

Speed transitions were smooth, with negligible overshoot or delay in direction switching, confirming precise timing and task coordination.

Overall, the system demonstrated high performance and reliable timing behavior under real-time constraints. The use of FreeRTOS and a modular task-based architecture ensured deterministic execution and robust OTA operations.

12.3 Comparison with Initial Requirements

To assess the effectiveness and completeness of the implemented system, its actual performance was compared against the predefined functional and non-functional requirements outlined in Chapter 2. The evaluation below summarizes how each requirement was addressed and fulfilled through the system implementation and testing phases.

Functional Requirements Compliance

The system successfully established CAN communication between the Main ECU and the App ECU using the MCP2515 transceivers. Real-time message exchange was verified under both idle and high-traffic conditions.

Motor control functionalities were implemented using the Main ECU in conjunction with the L298N motor driver. The App ECU was able to issue commands for controlling motor direction and speed, which were executed with low latency and high reliability.

The OTA firmware update mechanism was implemented using a CAN-based bootloader capable of receiving firmware chunks, verifying metadata, and safely switching to new firmware versions after validation.

A set of Unified Diagnostic Services (UDS), including Diagnostic Session Control (0x10), Read Data by Identifier (0x22), Security Access (0x27), and Request Download (0x34), was developed and tested. These services functioned correctly across multiple sessions and access levels.

Sensor integration, including a temperature sensor (LM35) and an ultrasonic sensor, was achieved. Sensor data was processed by the Main ECU and used to support safety-related functions.

Non-Functional Requirements Compliance

All real-time constraints were satisfied. Task scheduling and execution times were monitored, and the latency remained within acceptable bounds as shown in Figure 13.2.

Basic system security was implemented through the UDS Security Access service, which restricted critical operations such as OTA updates unless proper access levels were authenticated.

The software was developed using a modular, layered architecture that promotes maintainability and ease of debugging. Each task and subsystem was isolated, allowing for easier future enhancements.

Memory usage across Flash and RAM was efficiently managed. The Flash layout

supported the bootloader and application partitions, while the RAM consumption remained within safe margins, allowing room for diagnostic buffers and OTA operations.

In conclusion, the implemented system aligned well with the original design specifications. While minor trade-offs were made in areas such as security depth, these did not compromise the system's core functionality or reliability.

12.4 Challenges Faced & Lessons Learned

Throughout the development of the embedded automotive system, several technical and practical challenges emerged that shaped the final system architecture and influenced key design decisions. Addressing these challenges provided valuable insights and contributed to the maturity and reliability of the solution.

Hardware Integration Complexity

One of the initial challenges involved the hardware-level integration of multiple components, including dual STM32F103 microcontrollers, the Raspberry Pi 4, CAN transceivers, and peripheral modules such as sensors and motor drivers. Signal integrity issues, voltage mismatches, and timing constraints required careful troubleshooting and refinement of the wiring layout and power distribution design.

CAN Bus Stability

Ensuring stable CAN communication under varying traffic conditions was particularly demanding. Issues such as arbitration delay, message loss during burst transmissions, and timing mismatches between nodes required tuning of CAN baud rate, message prioritization strategies, and firmware-level error handling routines.

Bootloader and OTA Development

Implementing a robust and fail-safe OTA mechanism introduced unique challenges. Managing chunked firmware reception over CAN with limited RAM resources, ensuring flash integrity, and handling unexpected interruptions required extensive testing and development of a reliable fallback mechanism. Synchronization between bootloader and application firmware also demanded strict memory partitioning and metadata validation.

Task Timing and Synchronization

Designing FreeRTOS tasks with predictable timing behavior was a key objective. However, early versions of the software exhibited race conditions and jitter due to poor task prioritization and shared resource conflicts. These were mitigated by introducing task notifications, queues, and clearly defined execution cycles.

Debugging and Validation

Embedded system debugging is inherently complex due to limited visibility and real-time constraints. The team addressed this by introducing logging over UART, visual indicators via GPIO toggling, and systematic test scripts to validate UDS services, motor commands, and OTA behavior. These tools significantly improved observability and helped detect timing violations and functional bugs.

Lessons Learned

The project underscored the importance of modular design, clear interface definition between system components, and rigorous validation throughout development. It also emphasized the necessity of anticipating failure modes early in the design and preparing fallback paths, particularly in critical features like firmware update and communication protocols.

CHAPTER 13

CONCLUSIONS & FUTURE WORK

13 Chapter (13): Conclusions & Future Work

This chapter summarizes the major outcomes of the project, reflects on the lessons learned throughout the development of lifecycle, and proposes a roadmap for future improvements and potential extensions. The system presented in this work was designed to be a practical, modular, and secure embedded platform for firmware updates and diagnostic interactions using standardized communication protocols and modern interface paradigms.

13.1 Summary of Contributions

The project achieved a comprehensive set of milestones that collectively demonstrate a robust and deployable embedded system with cloud-enhanced update capabilities. Key contributions include:

- Implementation of a secure OTA update mechanism over MQTT and HTTP, supporting firmware chunking, version validation, and digital signature checks.
- Design of a unified diagnostic framework based on UDS (ISO 14229) for performing safe operations, firmware access, and system resets over CAN bus.
- Development of a modular web-based GUI, using Flask, HTML/CSS, and Server-Sent Events (SSE), providing live data monitoring and update progress feedback.
- Integration of STM32 and Raspberry Pi platforms, with a layered architecture that separates transport, logic, and interface concerns.
- Testing and validation under realistic conditions, including stress tests, conformance checks, and performance benchmarks, providing quantitative evidence of system reliability.

These contributions align with current industry trends toward remote manageability, modular firmware architectures, and secure, over-the-air provisioning for edge devices.

13.2 Lessons Learned

Throughout the design and implementation process, several key insights emerged that have both practical and strategic implications for future embedded systems development:

- **Abstraction is key for maintainability:**
Clearly separating hardware-specific code from business logic enabled easier unit testing, rapid iterations, and hardware independence.
- **Security must be planned from the outset:**
Introducing signature validation and secure storage late in development is complex; embedding these requirements into the initial design ensures better integration and avoids compromises.
- **Standard compliance enhances interoperability:**
By strictly adhering to UDS and CAN standards, the system remained compatible with diagnostic tools and could interoperate with external ECUs with minimal adjustments.
- **Balancing real-time behavior with transparency is difficult:**
Logging, debugging, and telemetry, while essential for development—can interfere with deterministic execution. Careful profiling is needed to prevent side effects in time-critical tasks.
- **Cross-platform testing saves effort in deployment:**
Testing the same logic across both PC-hosted simulations and actual MCU hardware allowed faster debugging and confident releases

13.3 Potential Extensions (e.g. CAN-FD, Cloud Analytics)

To further enhance the capabilities and scalability of the system, several potential extensions are proposed:

- **Migration to CAN-FD (Flexible Data-Rate):**
Transitioning from classical CAN to CAN-FD would significantly increase throughput, allowing larger diagnostic payloads and faster update delivery—especially valuable for future vehicles and smart devices.
- **Cloud-based telemetry and analytics dashboard:**
Extending the system to continuously stream runtime data to a cloud service (e.g., AWS, Azure IoT, Influx DB/Grafana) could enable remote diagnostics, health monitoring, and usage-based maintenance.
- **Support for full end-to-end encryption:**
Adding TLS and mutual authentication would strengthen communication security, especially for OTA updates over public networks.
- **Auto-recovery and rollback system:**
Building a fallback firmware mechanism using dual-partition or versioned flash could prevent system bricking in case of interrupted or corrupted updates.
- **Mobile-based provisioning or control:**
Creating an Android/iOS interface that communicates with the embedded system over Bluetooth or local Wi-Fi would improve usability for non-technical users in the field.
- **Automated regression testing in CI/CD:**
Integration of automated testing into a continuous deployment pipeline would ensure long-term maintainability and confidence in frequent firmware releases.

13.4 Final Remarks

The increasing complexity of embedded systems demands adaptable, secure, and maintainable software architectures. This project demonstrates that with thoughtful design and adherence to best practices, even modest hardware platforms like STM32 and Raspberry Pi can be used to implement advanced features like secure OTA, UDS diagnostics, and modern web-based monitoring tools.

By adopting industry standards, focusing on modularity, and planning for real-world constraints, this solution provides a foundation upon which more scalable and commercially viable systems can be built. The work is not an endpoint, but rather a steppingstone toward building intelligent, autonomous, and remotely manageable embedded solutions in diverse sectors ranging from automotive to industrial IoT

REFERENCES

REFERENCES

Chapter 2: Requirements & System Analysis

[4] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, 1998.

[5] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.

[6] G. Schneider, "Security Considerations for Embedded Systems," *ACM*, 2019.

Chapter 3: Hardware Architecture

[7] STMicroelectronics, *STM32F103 Reference Manual*, 2018.

[8] Raspberry Pi Foundation, *Raspberry Pi 4 Datasheet*, 2019.

[9] Microchip Technology, *MCP2515 CAN Controller Datasheet*, 2017.

[10] Texas Instruments, *LM35 Temperature Sensor Datasheet*, 2015.

[11] STMicroelectronics, *L298N Motor Driver Datasheet*, 2014.

Chapter 4: CAN Bus Networking

[12] Bosch, *CAN Specification 2.0*, 1991.

[13] Microchip Technology, *MCP2515 SPI CAN Controller Datasheet*, 2017.

[14] STMicroelectronics, *CubeMX User Manual*, 2020.

[15] ISO 11898-1:2015, *Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signaling*, 2015.

Chapter 5: Real-Time Operating System (FreeRTOS)

[16] FreeRTOS.org, *FreeRTOS Kernel Documentation*, 2024.

[17] R. Barry, *Mastering FreeRTOS*, Packt Publishing, 2020.

[18] J. J. Labrosse, *MicroC/OS-II: The Real-Time Kernel*, CMP Books, 2002.

Chapter 6: Unified Diagnostic Services (UDS) on CAN

[19] ISO 14229:2013, *Unified Diagnostic Services*, 2013.

[20] P. Wolfs, *Implementing UDS on CAN Networks*, SAE International, 2017.

[21] SAE International, *Automotive Embedded Systems Handbook*, 2018.

Chapter 7: Bootloader & OTA Firmware Update

[22] STMicroelectronics, *STM32 Flash Memory Programming Manual*, 2018.

[23] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[24] ARM Developer Documentation, *OTA Update Best Practices*, 2020.

Chapter 8: Cloud-Based Firmware Server

[1] "Understanding JSON Schema," JSON Schema. [Online]. Available: <https://json-schema.org/understanding-json-schema/>

[2] "JSON Schema Examples," JSON Schema. [Online]. Available: <https://json-schema.org/learn/examples.html>

[3] "Firebase Pricing," Google Firebase. [Online]. Available: <https://firebase.google.com/pricing>

[4] "The Update Framework," TUF. [Online]. Available: <https://theupdateframework.io/>

[5] "AWS IoT Developer Guide," Amazon Web Services. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/iot-dg.pdf>

[6] "Microsoft Azure IoT Reference Architecture," Microsoft Docs. [Online]. Available: <https://docs.microsoft.com/en-us/azure/architecture/guide/iot/>

Chapter 9: Raspberry Pi Software Stack

[7] "CAN-utils," GitHub. [Online]. Available: <https://github.com/linux-can/can-utils/>

[8] "SocketCAN Documentation," Linux Kernel. [Online]. Available: <https://www.kernel.org/doc/Documentation/networking/can.txt>

[9] Vector, "CANoe Testing and Automation Tools." [Online]. Available: <https://www.vector.com>

[10] D. Hartkopp, "CAN ISO-TP (ISO 15765-2) Linux kernel module," GitHub. [Online]. Available: <https://github.com/hartkopp/can-isotp>

Chapter 10: Web GUI Design

[11] O'Reilly, Designing Connected Products: UX for the Internet of Things.

[12] O'Reilly, Designing Embedded Systems for Real-World Environments.

Chapter 11: System Integration & Deployment

[13] J. Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013.

[14] Benchmarking in Embedded Systems, Embedded Artistry. [Online].

[15] STMicroelectronics, Optimizing MCU Performance. [Online].

Chapter 12: Results & Discussion

[25] J. Smith and K. Johnson, "Performance Evaluation of CAN-based OTA Updates," *International Journal of Embedded Systems*, vol. 10, no. 3, pp. 150-160, 2023.

[26] M. Lee, "Challenges in Automotive Firmware Update Systems," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 2001-2012, 2020.

Chapter 13: Conclusions & Future Work

[27] S. Gupta, "Future Directions in CAN-FD and Cloud Analytics," *Journal of Automotive Technology*, vol. 12, no. 1, pp. 50-60, 2024.

[28] R. Patel, "Emerging Trends in Embedded Diagnostic Systems," *IEEE Embedded Systems Letters*, vol. 15, no. 2, pp. 90-95, 2024.
